# ELE456
# Lecture Notes

Lectures by Paolo Stegagno

Fall 2026

# About these notes

These notes develop the mathematical foundations of robotics, from linear algebra to models of robot motion, sensing, and planning. Each chapter introduces a topic through definitions, explanations, and worked examples. The material builds progressively, with an emphasis on understanding the assumptions behind each calculation and interpreting the resulting relationships.

**Preparation.** These notes are prepared with AI assistance from Paolo Stegagno's lecture transcripts and board material and reviewed by the instructor. Errors may remain; please report suspected mistakes.

**License.** Copyright © 2026 Paolo Stegagno, to the extent copyright applies. Unless otherwise noted, these notes, diagrams, and their LaTeX/TikZ sources are licensed under Creative Commons Attribution-NonCommercial 4.0 International (CC BY-NC 4.0). Sharing and adaptation are permitted for noncommercial purposes with appropriate attribution, a license link, and an indication of changes. Commercial uses outside the license require separate permission. Uses otherwise permitted by law and public-domain material are unaffected. The full terms are available at `https://creativecommons.org/licenses/by-nc/4.0/legalcode`.

# Contents


**1 Introduction and linear algebra** **1**

1.1 Matrices and dimensions . . . . . 1

1.2 Matrix addition . . . . . 1

1.3 Matrix multiplication . . . . . 2

1.4 The transpose . . . . . 3

1.5 Identity matrices and inverses . . . . . 3

1.6 Orthogonal matrices . . . . . 4

1.7 Key identities . . . . . 5

1.8 Additional exercises . . . . . 6

**2 Reference frames and planar rotations** **7**

2.1 Coordinates depend on a reference frame . . . . . 7

2.2 Constructing the rotation matrix . . . . . 8

2.3 Transforming the coordinates of a point . . . . . 9

2.4 Reversing a coordinate transformation . . . . . 11

2.5 Composing rotations through several frames . . . . . 11

2.6 The limit of a rotation-only model . . . . . 12

2.7 Key identities . . . . . 13

2.8 Additional exercises . . . . . 14

**3 Planar rigid transformations and homogeneous coordinates** **15**

3.1 Rotation and translation between two frames . . . . . 15

3.2 Reversing the affine relationship . . . . . 17

3.3 Composing transformations through a sensor frame . . . . . 18

3.4 Homogeneous coordinates and transformation matrices . . . . . 18

3.5 Composition by matrix multiplication . . . . . 19

3.6 Inverting a homogeneous transformation . . . 20
3.7 Key identities . . . 21
3.8 Additional exercises . . . 22

**4 Three-dimensional rotations and orientation representations 23**
4.1 Spatial frames and rotation matrices . . . 23
4.2 Elementary rotations . . . 24
4.3 What makes a matrix a valid spatial rotation? . . . 25
4.4 Roll, pitch, and yaw as a sequence of rotations . . . 26
4.5 Worked example . . . 27
4.6 Recovering roll, pitch, and yaw . . . 28
4.7 Choosing and maintaining an orientation representation . . . 29
4.8 Key identities . . . 30
4.9 Additional exercises . . . 31

**5 Three-dimensional rigid transformations and pose 32**
5.1 Combining spatial rotation and translation . . . 32
5.2 The reverse coordinate transformation . . . 33
5.3 Homogeneous transformations in space . . . 34
5.4 Inverting a homogeneous transformation . . . 34
5.5 What rounding changes . . . 35
5.6 Position, orientation, and pose . . . 36
5.7 Additional exercises . . . 37

**6 Robot systems and sensing 38**
6.1 Sensing, processing, and action . . . 38
6.2 Sensors and the environment . . . 38
6.3 Actuators and physical interaction . . . 39
6.4 Connecting measurements to commands . . . 39
6.5 Proprioceptive and exteroceptive sensing . . . 40
6.6 From a physical quantity to a measurement . . . 40
6.7 Gain, bias, noise, and calibration . . . 41
6.8 Distance sensing with ultrasound . . . 43
6.9 Infrared reflection and laser ranging . . . 44

6.10 Representing a planar laser scan . . . 44
6.11 From ranges to obstacle coordinates . . . 46
6.12 Additional exercises . . . 47

**7 Cameras and perspective projection** **48**
7.1 Images as arrays of measurements . . . 48
7.2 The pinhole camera model . . . 50
7.3 From metric image coordinates to pixels . . . 51
7.4 Worked projection example . . . 52
7.5 Why one image point does not determine depth . . . 53
7.6 Additional information for recovering position . . . 54
7.7 Connecting camera geometry to robot frames . . . 55
7.8 Additional exercises . . . 56

**8 Motors, transmissions, and speed control** **57**
8.1 From a command to motion . . . 57
8.2 Pulse-width modulation . . . 58
8.3 Other forms of actuation . . . 59
8.4 Transmissions and reduction ratio . . . 59
8.5 Encoders and speed measurement . . . 60
8.6 From encoder counts to wheel speed . . . 61
8.7 From desired wheel speed to desired motor speed . . . 62
8.8 Closing the speed-control loop . . . 62
8.9 Additional exercises . . . 64

**9 Differential-drive kinematics and odometry** **65**
9.1 Geometry and assumptions . . . 65
9.2 From wheel velocities to robot motion . . . 66
9.3 The unicycle model . . . 67
9.4 Odometry as integration of measured motion . . . 69
9.5 Which values belong in an integration step? . . . 69
9.6 Sampling interval and missed motion . . . 72
9.7 From wheel counts to an odometry update . . . 73
9.8 Why odometry drifts . . . 74

9.9 Additional exercises . . . 75

**10 Robotic arms and direct kinematics** **76**

10.1 Links, joints, and the end effector . . . 76
10.2 Joint variables and degrees of freedom . . . 77
10.3 Direct and inverse kinematics . . . 77
10.4 Building direct kinematics from local transformations . . . 79
10.5 A planar two-link manipulator . . . 79
10.6 Task dimension, reachability, and redundancy . . . 81
10.7 The Denavit–Hartenberg frame convention . . . 82
10.8 Four parameters for one adjacent-frame transformation . . . 84
10.9 Deriving the standard DH matrix . . . 85
10.10 From a robot drawing to direct kinematics . . . 87
10.11 Additional exercises . . . 88

**A Rotation representation conversions** **90**

# Chapter 1

# Introduction and linear algebra

Robotics brings together mathematical models of motion, measurements from sensors, and algorithms that determine how a robot should act. Describing a robot's position, relating measurements to its surroundings, and planning its motion all require a precise mathematical language. Linear algebra provides many of the tools used to express these relationships.

This chapter introduces matrices and the operations needed to work with them: addition, multiplication, transposition, and inversion. It then examines orthogonal matrices, whose inverses can be obtained simply by transposing. The examples develop two habits that will remain essential throughout the study of robotics: checking dimensions before performing an operation and preserving the order of factors in a matrix product.

## 1.1 Matrices and dimensions

A *matrix* is an array of numbers arranged in rows and columns. Its dimensions are written as

$$\text{number of rows} \times \text{number of columns}.$$

Consider the matrix

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 1 \\ 0 & 1 \end{bmatrix} \in \mathbb{R}^{3\times 2}. \tag{1.1}$$

It has three rows and two columns. In general, $a_{ij}$ denotes the entry in row $i$, column $j$; for example, $a_{21} = 3$ and $a_{32} = 1$.

## 1.2 Matrix addition

Two matrices can be added only if they have the same dimensions. Their entries are then added in corresponding positions:

$$(A + C)_{ij} = a_{ij} + c_{ij}.$$

Consider two additional matrices,

$$B = \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 2}, \qquad C = \begin{bmatrix} 0 & 1 \\ 0 & 0 \\ 1 & 0 \end{bmatrix} \in \mathbb{R}^{3\times 2}.$$

The sum $A + B$ is undefined because the dimensions differ. The sum $A + C$ is defined, and its calculation is

$$A + C = \begin{bmatrix} 1+0 & 2+1 \\ 3+0 & 1+0 \\ 0+1 & 1+0 \end{bmatrix} = \begin{bmatrix} 1 & 3 \\ 3 & 1 \\ 1 & 1 \end{bmatrix}. \tag{1.2}$$

Addition is commutative when defined: $A + C = C + A$.

## 1.3 Matrix multiplication

### 1.3.1 Dimension rule and row-by-column calculation

For matrices $M \in \mathbb{R}^{m\times n}$ and $N \in \mathbb{R}^{n\times p}$, the product $MN$ is defined and has dimensions $m \times p$:

$$\underbrace{(m \times n)(n \times p)}_{\text{inner dimensions must match}} \qquad \longrightarrow \quad m \times p.$$

The entry in row $i$, column $j$ is the sum of the products of corresponding entries of row $i$ of $M$ and column $j$ of $N$:

$$(MN)_{ij} = \sum_{k=1}^{n} m_{ik} n_{kj}. \tag{1.3}$$

Matrix multiplication is therefore a row-by-column operation, rather than entrywise multiplication.

For the matrices $A$ and $B$ above, the dimensions are $(3 \times 2)(2 \times 2)$, so $AB$ is a $3 \times 2$ matrix:

$$AB = \begin{bmatrix} 1 & 2 \\ 3 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix} \tag{1.4}$$

$$= \begin{bmatrix} 1\cdot 1 + 2\cdot 1 & 1\cdot 2 + 2\cdot 1 \\ 3\cdot 1 + 1\cdot 1 & 3\cdot 2 + 1\cdot 1 \\ 0\cdot 1 + 1\cdot 1 & 0\cdot 2 + 1\cdot 1 \end{bmatrix} = \begin{bmatrix} 3 & 4 \\ 4 & 7 \\ 1 & 1 \end{bmatrix}. \tag{1.5}$$

### 1.3.2 The order of the factors matters

Reversing the factors gives the dimensions $(2 \times 2)(3 \times 2)$. The inner dimensions do not match, so $BA$ is undefined, even though $AB$ exists.

Even when both orders are defined, the results need not agree. With

$$E = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix},$$

direct multiplication gives

$$BE = \begin{bmatrix} 7 & 10 \\ 4 & 6 \end{bmatrix}, \qquad EB = \begin{bmatrix} 3 & 4 \\ 7 & 10 \end{bmatrix}.$$

Thus $BE \neq EB$. Matrix multiplication is not commutative in general. Some particular matrices do commute, but factors cannot be exchanged without a reason that applies to the matrices in question.

## 1.4 The transpose

Transposing a matrix interchanges its rows and columns. In entry notation, $(M^\mathsf{T})_{ij} = m_{ji}$. If $M$ has dimensions $m \times n$, then $M^\mathsf{T}$ has dimensions $n \times m$. For the running example,

$$A^\mathsf{T} = \begin{bmatrix} 1 & 3 & 0 \\ 2 & 1 & 1 \end{bmatrix} \in \mathbb{R}^{2\times 3}. \tag{1.6}$$

The transpose of a product reverses the order of its factors:

$$\boxed{(AB)^\mathsf{T} = B^\mathsf{T} A^\mathsf{T}.} \tag{1.7}$$

The dimensions provide a quick check. Here $B^\mathsf{T} A^\mathsf{T}$ has dimensions $(2 \times 2)(2 \times 3)$, producing a $2 \times 3$ matrix as required. The incorrect expression $A^\mathsf{T} B^\mathsf{T}$ would have dimensions $(2 \times 3)(2 \times 2)$ and is undefined.

Using the matrices defined above, this identity can be verified directly:

$$B^\mathsf{T} A^\mathsf{T} = \begin{bmatrix} 1 & 1 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 1 & 3 & 0 \\ 2 & 1 & 1 \end{bmatrix} = \begin{bmatrix} 3 & 4 & 1 \\ 4 & 7 & 1 \end{bmatrix} = (AB)^\mathsf{T}.$$

Dimension checks can rule out an invalid expression, but matching dimensions alone do not prove that an identity is correct.

## 1.5 Identity matrices and inverses

The identity matrix $I_n$ has ones on its main diagonal and zeros elsewhere. For a square matrix $M \in \mathbb{R}^{n\times n}$, an inverse $M^{-1}$ satisfies

$$MM^{-1} = M^{-1}M = I_n. \tag{1.8}$$

This ordinary, two-sided inverse is defined only for square matrices, and not every square matrix has one. The condition for invertibility is

$$\boxed{\det(M) \neq 0.} \tag{1.9}$$

A square matrix with zero determinant is called *singular*.

### 1.5.1 A one-by-one example

For $H = \begin{bmatrix} 5 \end{bmatrix}$, the inverse is $H^{-1} = \begin{bmatrix} 1/5 \end{bmatrix}$, since

$$HH^{-1} = H^{-1}H = \begin{bmatrix} 1 \end{bmatrix} = I_1.$$

This is the familiar reciprocal operation for a nonzero scalar.

### 1.5.2 A diagonal example and its determinant

The second example is

$$G = \begin{bmatrix} 1 & 0 \\ 0 & 2 \end{bmatrix}, \qquad G^{-1} = \begin{bmatrix} 1 & 0 \\ 0 & 1/2 \end{bmatrix}.$$

Multiplying verifies the inverse:

$$GG^{-1} = \begin{bmatrix} 1\cdot 1 + 0\cdot 0 & 1\cdot 0 + 0\cdot \frac{1}{2} \\ 0\cdot 1 + 2\cdot 0 & 0\cdot 0 + 2\cdot \frac{1}{2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2.$$

The reverse product also equals $I_2$. For a general $2 \times 2$ matrix,

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc.$$

Hence $\det(G) = 1 \cdot 2 - 0 \cdot 0 = 2 \neq 0$, as required. Reciprocating the diagonal entries works here because $G$ is diagonal with nonzero diagonal entries; matrix inversion is not generally an entrywise reciprocal operation.

## 1.6 Orthogonal matrices

A real square matrix $L$ is *orthogonal* if its inverse equals its transpose:

$$\boxed{L^{-1} = L^{\mathsf{T}}.} \tag{1.10}$$

Equivalently, $LL^{\mathsf{T}} = L^{\mathsf{T}}L = I$. This gives a practical way to check orthogonality: form the transpose, multiply, and verify the identity matrix. For an orthogonal matrix, an inverse can be obtained by transposing instead of performing a general inverse calculation.

### 1.6.1 Verifying orthogonality

Consider

$$L = \begin{bmatrix} \frac{1}{2} & \frac{\sqrt{3}}{2} \\ -\frac{\sqrt{3}}{2} & \frac{1}{2} \end{bmatrix}, \qquad L^{\mathsf{T}} = \begin{bmatrix} \frac{1}{2} & -\frac{\sqrt{3}}{2} \\ \frac{\sqrt{3}}{2} & \frac{1}{2} \end{bmatrix}.$$

Their product is

$$LL^{\mathsf{T}} = \begin{bmatrix} \frac{1}{4} + \frac{3}{4} & -\frac{\sqrt{3}}{4} + \frac{\sqrt{3}}{4} \\ -\frac{\sqrt{3}}{4} + \frac{\sqrt{3}}{4} & \frac{3}{4} + \frac{1}{4} \end{bmatrix} \tag{1.11}$$

$$= \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2. \tag{1.12}$$

Thus $L$ is orthogonal and the displayed $L^{\mathsf{T}}$ is its inverse.

### 1.6.2 The trigonometric pattern

The preceding example belongs to the family

$$Q(\theta) = \begin{bmatrix} \sin\theta & \cos\theta \\ -\cos\theta & \sin\theta \end{bmatrix}. \tag{1.13}$$

Multiplying by the transpose gives

$$\begin{aligned} Q(\theta)Q(\theta)^{\mathsf{T}} &= \begin{bmatrix} \sin^2\theta + \cos^2\theta & -\sin\theta\cos\theta + \cos\theta\sin\theta \\ -\cos\theta\sin\theta + \sin\theta\cos\theta & \cos^2\theta + \sin^2\theta \end{bmatrix} \\ &= I_2. \end{aligned}$$

The identity $\sin^2\theta + \cos^2\theta = 1$ makes this family orthogonal for every real $\theta$.

The numerical example corresponds to $L = Q(\pi/6)$, since $\sin(\pi/6) = 1/2$ and $\cos(\pi/6) = \sqrt{3}/2$. An equivalent parameterization uses cosine on the diagonal:

$$P(\varphi) = \begin{bmatrix} \cos\varphi & \sin\varphi \\ -\sin\varphi & \cos\varphi \end{bmatrix}.$$

The relation $Q(\theta) = P(\pi/2 - \theta)$ follows from the complementary-angle identities. Thus the same numerical matrix is $L = P(\pi/3)$. In either parameterization, orthogonality follows from the same trigonometric identity. Such matrices provide a foundation for describing changes of orientation in robotics.

## 1.7 Key identities

For matrices of compatible dimensions, the operations developed in this chapter can be collected as follows:

| Operation or property | Rule |
|---|---|
| Addition | $(M+N)_{ij} = m_{ij} + n_{ij}$ |
| Multiplication | $(MN)_{ij} = \sum_{k=1}^{n} m_{ik}n_{kj}$ |
| Transpose | $(M^{\mathsf{T}})_{ij} = m_{ji}$ |
| Transpose of a product | $(MN)^{\mathsf{T}} = N^{\mathsf{T}}M^{\mathsf{T}}$ |
| Inverse | $MM^{-1} = M^{-1}M = I$ |
| Invertibility of a square matrix | $\det(M) \neq 0$ |
| Orthogonality | $M^{-1} = M^{\mathsf{T}}$ |

In the multiplication formula, $n$ is the number of columns of $M$ and the number of rows of $N$. Addition requires equal dimensions, whereas a product requires only that these inner dimensions match. Inverse and orthogonality conditions apply to square matrices. These distinctions determine which algebraic manipulations are valid before any numerical calculation begins.

## 1.8 Additional exercises

Show your calculations and justify each conclusion. Check dimensions before performing an operation. These exercises can be completed by hand.

1. **Dimensions and valid operations.** Let $A \in \mathbb{R}^{3\times 2}$, $B \in \mathbb{R}^{2\times 3}$, and $C \in \mathbb{R}^{3\times 3}$. Determine which expressions are defined and give the dimensions of each valid result:

$$A+B, \qquad AB, \qquad BA, \qquad CA, \qquad AC, \qquad A^{\mathsf{T}}+C, \qquad B+A^{\mathsf{T}}.$$

2. **Addition and multiplication.** Consider

$$M = \begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix}, \qquad N = \begin{bmatrix} 1 & 2 \\ -1 & 0 \end{bmatrix}, \qquad v = \begin{bmatrix} 1 \\ -2 \end{bmatrix}.$$

   Compute $M+N$, $MN$, $NM$, and $Mv$. Do $M$ and $N$ commute? Verify numerically that $(M+N)v = Mv + Nv$.

3. **Transposing a product.** Let

$$U = \begin{bmatrix} 1 & 0 & -1 \\ 2 & 1 & 3 \end{bmatrix}, \qquad V = \begin{bmatrix} 1 & 2 \\ 0 & -1 \\ 2 & 0 \end{bmatrix}.$$

   Compute $UV$ and $(UV)^{\mathsf{T}}$, then independently compute $V^{\mathsf{T}}U^{\mathsf{T}}$ and compare the results. Is $U^{\mathsf{T}}V^{\mathsf{T}}$ defined? Explain why it cannot equal $(UV)^{\mathsf{T}}$ in this example.

4. **Determinants and inverses.** For

$$D = \begin{bmatrix} 4 & 0 \\ 0 & -2 \end{bmatrix}, \qquad F = \begin{bmatrix} 1 & 2 \\ 2 & 4 \end{bmatrix}, \qquad K = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix},$$

   use determinants to decide which matrices are invertible. Find $D^{-1}$. Verify that $\begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix}$ is the inverse of $K$ by multiplying in both orders.

5. **Testing orthogonality.** Consider

$$R = \begin{bmatrix} \frac{3}{5} & -\frac{4}{5} \\ \frac{4}{5} & \frac{3}{5} \end{bmatrix}, \qquad S = \begin{bmatrix} 1 & 0 \\ 0 & -1 \end{bmatrix}, \qquad T = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix}.$$

   Test each matrix for orthogonality using its transpose. For each orthogonal matrix, write its inverse. Is an invertible matrix necessarily orthogonal? Support your answer with one of these examples.

6. **A product of orthogonal matrices.** Suppose $P$ and $Q$ are real orthogonal $n \times n$ matrices. Use the transpose-of-a-product rule to show that $(PQ)(PQ)^{\mathsf{T}} = I_n$. Deduce that $PQ$ is orthogonal and express $(PQ)^{-1}$ in terms of $P^{\mathsf{T}}$ and $Q^{\mathsf{T}}$, paying attention to their order.

# Chapter 2

# Reference frames and planar rotations

A robot must relate what its sensors observe to a description of the surrounding world. An obstacle may be directly ahead of the robot, yet lie in a different direction when described on a map. Neither description is ambiguous once the reference frame is specified. The task is to express the same geometric point in different coordinate systems.

This chapter develops that change of coordinates for planar frames with a common origin. Their relative orientation is described by one angle, or equivalently by a rotation matrix. The matrix formulation makes it possible to transform coordinates, reverse a transformation, and compose transformations through several frames using the linear algebra of Chapter 1.

## 2.1 Coordinates depend on a reference frame

A planar Cartesian reference frame consists of an origin and two perpendicular unit directions, denoted $\boldsymbol{x}$ and $\boldsymbol{y}$. A world frame $W$ provides a fixed reference for a map, while a robot frame $R$ is attached to the robot. A sensor can have its own frame $S$.

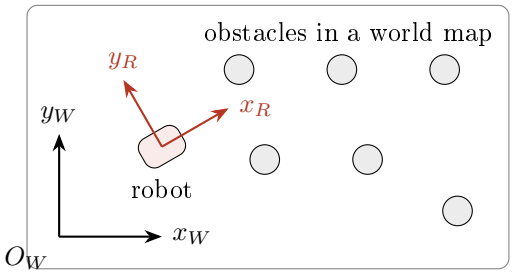



Figure 2.1: A map and a robot use different reference frames. In a general setting their origins and orientations both differ.

If the robot's $x_R$ axis points forward and $y_R$ points left, an obstacle one unit ahead has robot coordinates $[1\ 0]^\mathsf{T}$. An obstacle one unit to the left has coordinates $[0\ 1]^\mathsf{T}$. These descriptions alone do not specify where the obstacles belong on the world map; the relation between the frames is also needed.

### 2.1.1 Notation and the common-origin assumption

Let $B$ denote a geometric point. Its coordinates are column vectors,

$$B^W = \begin{bmatrix} x_B^W \\ y_B^W \end{bmatrix}, \qquad B^R = \begin{bmatrix} x_B^R \\ y_B^R \end{bmatrix}. \tag{2.1}$$

The superscript states the frame in which the coordinates are expressed. The point $B$ does not change when its coordinate description changes.

For now, assume that $W$ and $R$ share an origin $O$ and have the same orientation convention for their axes. Let $\theta = \theta_R^W$ be the signed angle from $x_W$ to $x_R$, positive counterclockwise. Only a rotation separates the two frames. Figure 2.2 shows how projections onto their axes produce different coordinates for the same point.

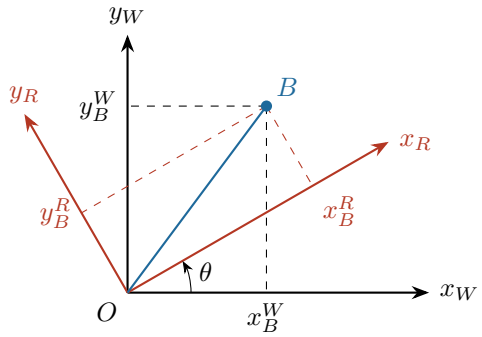



Figure 2.2: A point expressed in two frames with a common origin. Black axes belong to $W$ and red axes to $R$; dashed lines give the coordinate projections.

## 2.2 Constructing the rotation matrix

### 2.2.1 The columns are the rotated basis vectors

The direction $\boldsymbol{x}_R$ makes angle $\theta$ with $x_W$. Because it is a unit vector, its components in the world frame are

$$[\boldsymbol{x}_R]^W = \begin{bmatrix} \cos\theta \\ \sin\theta \end{bmatrix}. \tag{2.2}$$

The direction $\boldsymbol{y}_R$ is a further $\pi/2$ counterclockwise, so

$$[\boldsymbol{y}_R]^W = \begin{bmatrix} \cos(\theta + \pi/2) \\ \sin(\theta + \pi/2) \end{bmatrix} = \begin{bmatrix} -\sin\theta \\ \cos\theta \end{bmatrix}. \tag{2.3}$$

Here brackets with a superscript mean the components of a geometric vector in the indicated frame. The negative sign in the first component of $[\boldsymbol{y}_R]^W$ is essential: for a small positive angle, $y_R$ points to the left of $y_W$.

Putting these two column vectors side by side defines the rotation matrix:

$$\boxed{R_R^W = \begin{bmatrix} [\boldsymbol{x}_R]^W & [\boldsymbol{y}_R]^W \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}}. \tag{2.4}$$

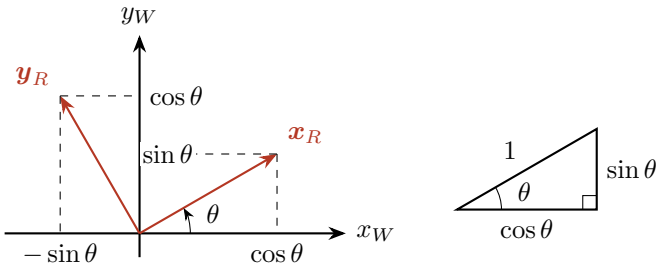



Figure 2.3: Components of the robot's unit basis vectors in the world frame. The right triangle explains the sine and cosine components of $\boldsymbol{x}_R$.

The subscript identifies the frame whose orientation is being described, and the superscript identifies the reference frame. As a coordinate map, $R_R^W$ takes components expressed in $R$ and returns components expressed in $W$. An angle is a compact description of orientation; the matrix provides a linear map on coordinate vectors once that angle is fixed.

### 2.2.2 Orthogonality

Write $c = \cos\theta$ and $s = \sin\theta$. Direct multiplication gives

$$(R_R^W)^{\mathsf{T}} R_R^W = \begin{bmatrix} c & s \\ -s & c \end{bmatrix} \begin{bmatrix} c & -s \\ s & c \end{bmatrix} = \begin{bmatrix} c^2+s^2 & 0 \\ 0 & c^2+s^2 \end{bmatrix} = I_2.$$

Thus $R_R^W$ is orthogonal and

$$(R_R^W)^{-1} = (R_R^W)^{\mathsf{T}} = \begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}. \tag{2.5}$$

Its determinant is $c^2 + s^2 = 1$. This last property distinguishes a rotation from an orthogonal reflection, whose determinant is $-1$.

## 2.3 Transforming the coordinates of a point

The vector from the common origin to $B$ can be decomposed along the robot axes:

$$\overrightarrow{OB} = x_B^R \boldsymbol{x}_R + y_B^R \boldsymbol{y}_R.$$

Expressing both basis vectors in $W$ gives

$$\boxed{B^W = R_R^W B^R.} \tag{2.6}$$

In scalar form, this is

$$x_B^W = \cos\theta\, x_B^R - \sin\theta\, y_B^R, \tag{2.7}$$
$$y_B^W = \sin\theta\, x_B^R + \cos\theta\, y_B^R. \tag{2.8}$$

The dimensions are $(2\times 2)(2\times 1) = (2\times 1)$, and the input frame $R$ agrees with the matrix subscript. Both checks are useful: correct matrix dimensions alone do not establish that the intended frames match.

### 2.3.1 A geometric derivation

For $B \neq O$, let $d$ be the distance from $O$ to $B$, let $\alpha$ be the angle from $x_R$ to $\overrightarrow{OB}$, and let $\beta$ be the angle from $x_W$ to that vector. The angles satisfy $\beta = \alpha + \theta$ (modulo $2\pi$). Consequently,

$$x_B^R = d\cos\alpha, \quad y_B^R = d\sin\alpha, \qquad x_B^W = d\cos\beta, \quad y_B^W = d\sin\beta.$$

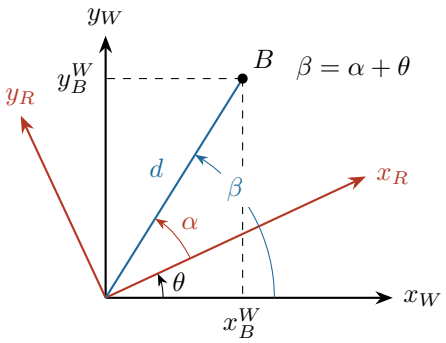



Figure 2.4: The same position vector has angle $\alpha$ in $R$ and angle $\beta = \alpha + \theta$ in $W$. Its length $d$ is independent of the frame.

Expanding the angle sums gives

$$\begin{aligned} x_B^W &= d\cos(\alpha+\theta) = d\cos\alpha\cos\theta - d\sin\alpha\sin\theta \\ &= \cos\theta\, x_B^R - \sin\theta\, y_B^R, \\ y_B^W &= d\sin(\alpha+\theta) = d\cos\alpha\sin\theta + d\sin\alpha\cos\theta \\ &= \sin\theta\, x_B^R + \cos\theta\, y_B^R. \end{aligned}$$

These are precisely the entries of (2.6). The matrix formula also applies at $B = O$, where both coordinate vectors are zero and a direction angle is unnecessary.

### 2.3.2 Worked example: a thirty-degree orientation

Suppose

$$B^R = \begin{bmatrix} 0.8 \\ 0.5 \end{bmatrix}, \qquad \theta = \frac{\pi}{6}.$$

Then

$$B^W = \begin{bmatrix} \sqrt{3}/2 & -1/2 \\ 1/2 & \sqrt{3}/2 \end{bmatrix} \begin{bmatrix} 0.8 \\ 0.5 \end{bmatrix} \tag{2.9}$$

$$= \begin{bmatrix} 0.4\sqrt{3} - 0.25 \\ 0.4 + 0.25\sqrt{3} \end{bmatrix} \approx \begin{bmatrix} 0.4428 \\ 0.8330 \end{bmatrix}. \tag{2.10}$$

The coordinates change, but the point's distance from the common origin does not. Indeed, orthogonality implies

$$(B^W)^{\mathsf{T}} B^W = (B^R)^{\mathsf{T}} (R_R^W)^{\mathsf{T}} R_R^W B^R = (B^R)^{\mathsf{T}} B^R = 0.8^2 + 0.5^2 = 0.89.$$

This is a change of coordinate description of a fixed point, rather than a physical motion of that point.

## 2.4 Reversing a coordinate transformation

To recover robot coordinates from world coordinates, multiply Equation (2.6) on the left by the inverse:

$$(R_R^W)^{-1} B^W = (R_R^W)^{-1} R_R^W B^R = B^R.$$

Using orthogonality,

$$\boxed{B^R = (R_R^W)^\mathsf{T} B^W = R_W^R B^W, \qquad R_W^R = (R_R^W)^\mathsf{T}.} \tag{2.11}$$

Reversing the frame relation exchanges the frame indices and negates the relative angle. If $R(\theta)$ denotes the matrix in Equation (2.4), then $R(\theta)^\mathsf{T} = R(-\theta)$. For arbitrary frames $A$ and $F$, the same rule reads

$$R_F^A = (R_A^F)^\mathsf{T}. \tag{2.12}$$

For example, applying the transpose to the exact vector in Equation (2.10) recovers $[0.8\ 0.5]^\mathsf{T}$.

## 2.5 Composing rotations through several frames

A measurement may begin in a sensor frame $S$, pass through a robot frame $R$, and finally be expressed in $W$. Assume again that all origins coincide. Let the orientation of $R$ relative to $W$ be $\theta$, and the orientation of $S$ relative to $R$ be $\varphi$.

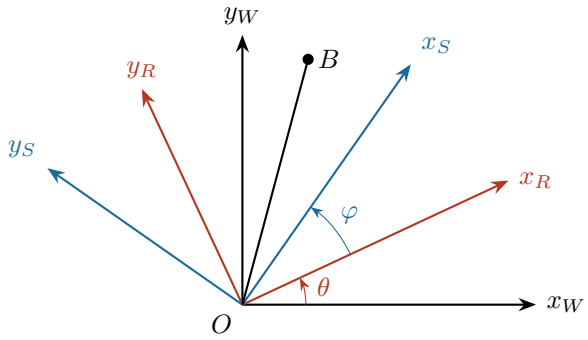



Figure 2.5: Three frames with a common origin. The sensor frame (blue) is rotated by $\varphi$ relative to the robot frame (red), which is rotated by $\theta$ relative to the world frame (black).

The two successive coordinate transformations are

$$B^R = R_S^R B^S, \qquad B^W = R_R^W B^R.$$

Substitution gives

$$B^W = R_R^W R_S^R B^S, \qquad \boxed{R_S^W = R_R^W R_S^R.} \tag{2.13}$$

The rightmost matrix acts first. Its output is expressed in $R$, precisely the frame expected by the matrix to its left. In two dimensions,

$$R_R^W = R(\theta), \qquad R_S^R = R(\varphi), \qquad R_S^W = R(\theta + \varphi),$$

as can be verified with the sine and cosine addition identities.

### 2.5.1 Reading the frame indices

The general composition rule is

$$R_C^A = R_B^A R_C^B. \tag{2.14}$$

The subscript of the left factor matches the superscript of the next factor. The remaining indices identify the overall output and input frames. This is a rule about the meanings of the maps, not ordinary cancellation of numbers.

For example,

$$R_2^3 R_4^2 R_5^4 R_1^5 = R_1^3. \tag{2.15}$$

Reading from right to left, the coordinates pass through frames $1 \to 5 \to 4 \to 2 \to 3$. By contrast, in the written product $R_2^3 R_3^4$, the inner frame labels 2 and 4 do not match. Its factors cannot be composed as that sequence of coordinate maps merely by cancelling the repeated 3.

Planar rotation matrices happen to commute numerically: $R(\theta)R(\varphi) = R(\varphi)R(\theta)$. This special property does not change the input and output frame labels of an individual map. Keeping the frame chain explicit also prepares the notation for three-dimensional rotations, where commutativity does not hold in general.

### 2.5.2 Simplifying chains containing transposes

First apply the transpose rules, then inspect the frame chain. Consider

$$R_2^3 (R_2^4)^{\mathsf{T}} (R_5^1 R_4^5)^{\mathsf{T}}.$$

Transposing the last product reverses its factor order, so

$$\begin{aligned} R_2^3 (R_2^4)^{\mathsf{T}} (R_5^1 R_4^5)^{\mathsf{T}} &= R_2^3 R_4^2 (R_4^5)^{\mathsf{T}} (R_5^1)^{\mathsf{T}} \\ &= R_2^3 R_4^2 R_5^4 R_1^5 \\ &= R_1^3. \end{aligned}$$

Two different operations are involved: a transpose exchanges the frame indices of one rotation matrix, and transposing a product reverses the order of its factors. Both are necessary to obtain a consistent chain.

## 2.6 The limit of a rotation-only model

The formula $B^W = R_R^W B^R$ describes point coordinates when the origins coincide. In a general robot configuration, the origin $O_R$ is displaced from $O_W$, as in Figure 2.6. A rotation matrix describes the relative orientation, but contains no information about this displacement.

To see the limitation, take the point $B = O_R$. Its robot coordinates are zero, so $R_R^W B^R = 0$ for every rotation matrix. Its world coordinates, however, are the nonzero displacement of $O_R$ from $O_W$. A complete point-coordinate transformation must therefore include translation as well as rotation. The rotation rules remain applicable to direction vectors, which do not depend on the choice of origin. Distinguishing point positions from directions is essential when extending the model.

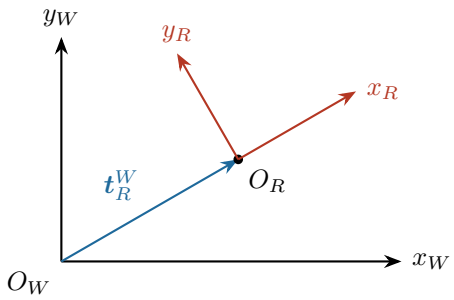



Figure 2.6: Different origins introduce a translation $\boldsymbol{t}_R^W$. Orientation alone cannot determine the world coordinates of a point measured relative to $O_R$.

## 2.7 Key identities

| Operation | Identity |
|---|---|
| Planar rotation | $R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}$ |
| Point coordinates, common origin | $B^W = R_R^W B^R$ |
| Reverse transformation | $R_W^R = (R_R^W)^\mathsf{T} = R(-\theta)$ |
| Composition | $R_S^W = R_R^W R_S^R$ |
| Planar angle composition | $R(\theta)R(\varphi) = R(\theta + \varphi)$ |

## 2.8 Additional exercises

Use column vectors, show your calculations, and label the input and output frames. Unless stated otherwise, all frames share an origin.

1. **Sketching frames and coordinates.** Draw world axes $x_W, y_W$ and robot axes rotated counterclockwise by $\pi/2$. A point has $B^R = [2\ 1]^\mathsf{T}$. Locate it on your sketch, compute $B^W$, and check that your answer agrees with its position relative to the world axes.

2. **Constructing a rotation matrix.** The robot frame is rotated by $-\pi/4$ relative to the world frame. Write $[\boldsymbol{x}_R]^W$ and $[\boldsymbol{y}_R]^W$, and use them to construct $R_R^W$. Verify $(R_R^W)^\mathsf{T} R_R^W = I_2$ and $\det(R_R^W) = 1$.

3. **Forward and reverse transformations.** Let $\theta_R^W = \pi/6$ and $B^R = [1\ 2]^\mathsf{T}$. Find $B^W$ in exact form. Construct $R_W^R$ and use it to recover $B^R$. Verify that the squared distance from the common origin is the same in both coordinate descriptions.

4. **From a sensor to the world.** Suppose $\theta_R^W = \pi/6$, $\theta_S^R = \pi/3$, and $B^S = [2\ -1]^\mathsf{T}$. Compute $R_S^W$ and $B^W$. Check your result by first finding $B^R$ and then transforming it to $W$. What is the orientation of $S$ relative to $W$?

5. **Frame chains and transposes.** Simplify the first two expressions to a single rotation matrix. For the third, explain why the written frame chain is inconsistent, even though the numerical matrix dimensions permit multiplication:

$$R_B^A R_C^B R_D^C, \qquad (R_A^B)^\mathsf{T} (R_C^D R_B^C)^\mathsf{T}, \qquad R_B^A R_D^C.$$

   Assume distinct letters denote distinct frames. State the input and output frames of each valid chain.

6. **Why an origin matters.** The robot axes are parallel to the world axes, but the robot origin has world coordinates $[3\ 1]^\mathsf{T}$. A point has robot coordinates $[2\ 0]^\mathsf{T}$. Use a sketch to find its world coordinates. What would the rotation-only formula predict, and what information would it omit?

7. **Length preservation.** Let $v^W = R_R^W v^R$ be two coordinate descriptions of a direction vector. Using orthogonality, show that $(v^W)^\mathsf{T} v^W = (v^R)^\mathsf{T} v^R$. Explain geometrically why a change of reference frame cannot change the length of the vector.

# Chapter 3

# Planar rigid transformations and homogeneous coordinates

Rotation describes the relative orientation of two frames, but a mobile robot also changes position. Its origin generally differs from the origin of the world map. A sensor mounted on the robot may have yet another origin and orientation. Relating point coordinates between these frames requires both rotation and translation.

This chapter extends the rotation model of Chapter 2 to planar rigid transformations. The geometric construction leads first to an affine coordinate relationship, then to a homogeneous matrix representation that combines rotation and translation in a single multiplication. The same representation makes composition and inversion systematic.

## 3.1 Rotation and translation between two frames

Let $O_W$ and $O_R$ be the origins of world frame $W$ and robot frame $R$. Define

$$p_R^W = \begin{bmatrix} p_{Rx}^W \\ p_{Ry}^W \end{bmatrix}$$

as the position of $O_R$ relative to $O_W$, expressed in world coordinates. The rotation matrix $R_R^W$ describes the orientation of $R$ relative to $W$. Together, $p_R^W$ and $R_R^W$ specify the robot frame's planar pose.

For a point $B$, the geometric displacement vectors obey

$$\overrightarrow{O_W B} = \overrightarrow{O_W O_R} + \overrightarrow{O_R B}. \tag{3.1}$$

This triangle relation is independent of coordinates. To turn it into a numerical calculation, however, every vector being added must be expressed in the same frame.

The column $B^R$ contains the components of $\overrightarrow{O_R B}$ along the robot axes. The column $p_R^W$ uses the world axes. Adding these columns directly would combine components measured along different directions. First rotate $B^R$ into world-aligned components; then add the translation:

$$\boxed{B^W = p_R^W + R_R^W B^R.} \tag{3.2}$$

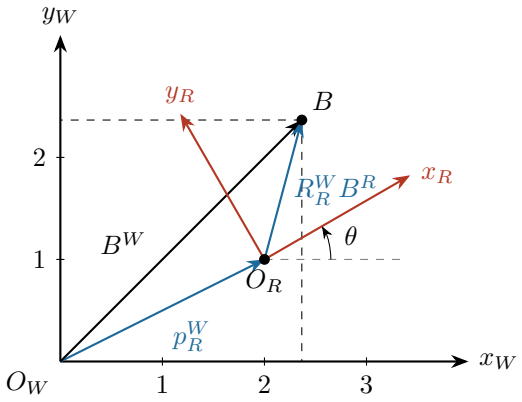



Figure 3.1: The displacement to $B$ is the sum of the displacement to the robot origin and the displacement from that origin to $B$. All three arrow labels give components in $W$. The geometry uses $p_R^W = [2\ 1]^\mathsf{T}$, $\theta = \pi/6$, and $B^R = [1\ 1]^\mathsf{T}$.

Here $R_R^W B^R$ describes the displacement from $O_R$ to $B$ using world directions. It is not, by itself, the position of $B$ relative to $O_W$.

### 3.1.1 An intermediate frame makes the geometry explicit

Introduce a frame $A$ with origin $O_A = O_R$ and axes parallel to those of $W$. Frames $R$ and $A$ differ only by rotation, so the previous chapter gives

$$B^A = R_R^A B^R.$$

Since $A$ and $W$ have the same orientation, $R_R^A = R_R^W$. Moreover, the components in $A$ use exactly the directions needed for addition to $p_R^W$. Thus $B^W = p_R^W + B^A$, recovering Equation (3.2).

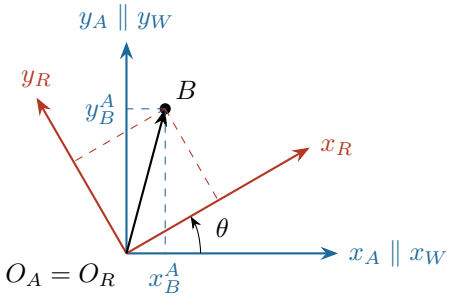



Figure 3.2: The auxiliary frame $A$ shares the robot origin but is aligned with the world. Rotating from $R$ to $A$ resolves the displacement into components that can be added to the world position of $O_R$.

### 3.1.2 Why the relationship is affine

A map $f(b) = Rb + p$ is called affine: it consists of a linear map followed by an offset. With a nonzero translation, it is not a linear map of the original two coordinates, since $f(0) = p \neq 0$. In

the present setting $R$ is a rotation, so the transformation is rigid: it preserves distances between points. Distances to the coordinate origin need not be preserved when the origin changes.

### 3.1.3 Worked example

Suppose the given quantities are

$$B^R = \begin{bmatrix}1\\1\end{bmatrix}\text{ m}, \qquad p_R^W = \begin{bmatrix}2\\1\end{bmatrix}\text{ m}, \qquad \theta_R^W = \frac{\pi}{6}.$$

The rotation matrix is dimensionless. Applying the affine relationship gives

$$B^W = \begin{bmatrix}2\\1\end{bmatrix} + \begin{bmatrix}\sqrt{3}/2 & -1/2\\ 1/2 & \sqrt{3}/2\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix} \tag{3.3}$$

$$= \begin{bmatrix}(3+\sqrt{3})/2\\(3+\sqrt{3})/2\end{bmatrix} \approx \begin{bmatrix}2.3660\\2.3660\end{bmatrix}\text{ m}. \tag{3.4}$$

The coordinate arithmetic uses metres throughout. The result agrees with Figure 3.1: the point lies above and slightly to the right of the robot origin. Keeping the exact expressions until the final step avoids rounding errors when reversing the calculation.

## 3.2 Reversing the affine relationship

Suppose $B^W$ is known and $B^R$ is required. Subtract the world-coordinate translation before applying the inverse rotation:

$$B^W - p_R^W = R_R^W B^R, \tag{3.5}$$

$$\boxed{B^R = (R_R^W)^\mathsf{T}(B^W - p_R^W).} \tag{3.6}$$

The difference in parentheses is expressed in $W$; the transpose maps it to $R$. The parentheses matter: the rotation acts on both terms, not just on $B^W$. Equivalently,

$$B^R = -(R_R^W)^\mathsf{T} p_R^W + (R_R^W)^\mathsf{T} B^W. \tag{3.7}$$

This has the same affine structure as the forward relationship, with

$$R_W^R = (R_R^W)^\mathsf{T}, \qquad p_W^R = -(R_R^W)^\mathsf{T} p_R^W. \tag{3.8}$$

The reversed translation is not generally just $-p_R^W$: it must also be expressed along the axes of the reverse transformation's output frame.

For the exact coordinates in Equation (3.4),

$$B^W - p_R^W = \begin{bmatrix}(\sqrt{3}-1)/2\\(1+\sqrt{3})/2\end{bmatrix}\text{ m}.$$

Multiplying by $\begin{bmatrix}\sqrt{3}/2 & 1/2\\ -1/2 & \sqrt{3}/2\end{bmatrix}$ recovers $B^R = [1\ 1]^\mathsf{T}$ m. If the rounded value $[2.37\ 2.37]^\mathsf{T}$ is used instead, the recovered result is approximately $[1.0054\ 1.0015]^\mathsf{T}$ m, rather than exactly $[1\ 1]^\mathsf{T}$.

## 3.3 Composing transformations through a sensor frame

A sensor frame $S$ may be displaced and rotated relative to the robot frame. Given the sensor's pose in $R$ and the robot's pose in $W$, the two coordinate relationships are

$$B^R = p_S^R + R_S^R B^S, \qquad B^W = p_R^W + R_R^W B^R.$$

Substitution yields

$$B^W = p_R^W + R_R^W \left(p_S^R + R_S^R B^S\right) \tag{3.9}$$

$$= \underbrace{p_R^W + R_R^W p_S^R}_{p_S^W} + \underbrace{R_R^W R_S^R}_{R_S^W} B^S. \tag{3.10}$$

The overall rotation and translation are therefore

$$\boxed{R_S^W = R_R^W R_S^R, \qquad p_S^W = p_R^W + R_R^W p_S^R.} \tag{3.11}$$

The sensor translation must be rotated into $W$ before it can be added to the robot translation. Frame labels explain why the expression is valid and why simply adding $p_R^W + p_S^R$ would be incorrect in general.

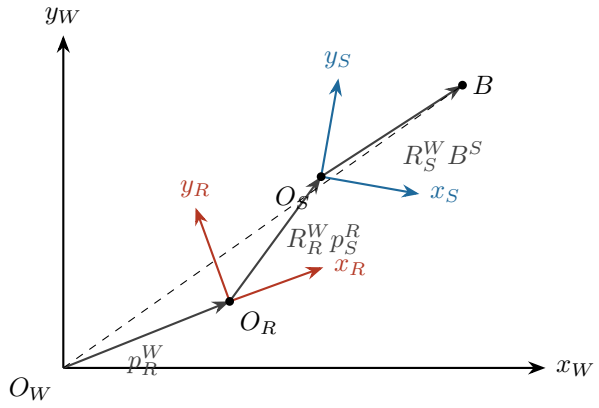



Figure 3.3: A world–robot–sensor chain with distinct origins. Each displacement along the path to $B$ is labeled in world components so that the arrows can be added numerically. The dashed line is the direct displacement to $B$.

Repeated substitution works for any number of frames, but each additional frame introduces another translation term. A representation that absorbs these additions into matrix multiplication makes longer chains easier to manage.

## 3.4 Homogeneous coordinates and transformation matrices

### 3.4.1 Adding one coordinate

For a point with coordinates $B^R$, define its homogeneous representation by appending a dimensionless one:

$$\widetilde{B}^R = \begin{bmatrix} B^R \\ 1 \end{bmatrix} = \begin{bmatrix} x_B^R \\ y_B^R \\ 1 \end{bmatrix}. \tag{3.12}$$

The additional coordinate is bookkeeping for translation, not a third spatial coordinate. Define the planar homogeneous transformation matrix as

$$\boxed{H_R^W = \begin{bmatrix} R_R^W & p_R^W \\ 0_{1\times 2} & 1 \end{bmatrix}.} \tag{3.13}$$

Its blocks have dimensions $2 \times 2$, $2 \times 1$, $1 \times 2$, and $1 \times 1$, respectively. Written entry by entry, it is

$$H_R^W = \begin{bmatrix} \cos\theta & -\sin\theta & p_{Rx}^W \\ \sin\theta & \cos\theta & p_{Ry}^W \\ 0 & 0 & 1 \end{bmatrix}.$$

The upper-left block contains dimensionless rotation coefficients; the first two entries of the last column carry the chosen length unit. Block divisions are visual aids, not extra mathematical operations.

### 3.4.2 One product performs the full transformation

Block multiplication gives

$$H_R^W \widetilde{B}^R = \begin{bmatrix} R_R^W & p_R^W \\ 0_{1\times 2} & 1 \end{bmatrix} \begin{bmatrix} B^R \\ 1 \end{bmatrix} = \begin{bmatrix} R_R^W B^R + p_R^W \\ 1 \end{bmatrix} = \widetilde{B}^W. \tag{3.14}$$

Thus the affine transformation becomes a single matrix multiplication:

$$\boxed{\widetilde{B}^W = H_R^W \widetilde{B}^R.} \tag{3.15}$$

The first two output entries are the desired point coordinates. For the normalized point representation used here, the last entry remains one. The fixed bottom row [0 0 1] is what preserves this property.

For the running example, using metres for the position entries,

$$\begin{bmatrix} \sqrt{3}/2 & -1/2 & 2 \\ 1/2 & \sqrt{3}/2 & 1 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} (3+\sqrt{3})/2 \\ (3+\sqrt{3})/2 \\ 1 \end{bmatrix} \approx \begin{bmatrix} 2.3660 \\ 2.3660 \\ 1 \end{bmatrix}.$$

The physical point coordinates are the first two entries in metres; the final one has no length unit.

## 3.5 Composition by matrix multiplication

Using homogeneous coordinates for the sensor chain,

$$\widetilde{B}^R = H_S^R \widetilde{B}^S, \qquad \widetilde{B}^W = H_R^W \widetilde{B}^R.$$

Combining the two maps gives

$$\boxed{H_S^W = H_R^W H_S^R, \qquad \widetilde{B}^W = H_R^W H_S^R \widetilde{B}^S.} \tag{3.16}$$

Multiplying the blocks verifies that this contains both composition rules:

$$H_R^W H_S^R = \begin{bmatrix} R_R^W & p_R^W \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R_S^R & p_S^R \\ 0 & 1 \end{bmatrix} \tag{3.17}$$

$$= \begin{bmatrix} R_R^W R_S^R & p_R^W + R_R^W p_S^R \\ 0 & 1 \end{bmatrix} = H_S^W. \tag{3.18}$$

Here each 0 in the bottom-left block denotes a $1 \times 2$ zero row. The product is another homogeneous rigid transformation with bottom row [0 0 1]. As before, the rightmost map acts first and adjacent frame indices must match. For example,

$$H_1^2 H_3^1 H_4^3 H_5^4 = H_5^2$$

maps coordinates through $5 \to 4 \to 3 \to 1 \to 2$.

Unlike pure planar rotations, planar transformations with translation do not generally commute. The term $R_R^W p_S^R$ shows why: a preceding rotation changes the direction in which a subsequent local translation acts.

## 3.6 Inverting a homogeneous transformation

### 3.6.1 Why the transpose is insufficient

Expanding the determinant along the bottom row gives

$$\det(H_R^W) = \det(R_R^W) = 1.$$

Every matrix of this rigid-transformation form is therefore invertible. However, it is generally not orthogonal. If its translation is nonzero, its transpose has bottom row $[(p_R^W)^\mathsf{T}\ 1]$ rather than [0 0 1]. Consequently, transposing the full matrix does not produce its inverse. When the translation is zero, the full matrix is orthogonal and the transpose does coincide with the inverse.

### 3.6.2 Deriving the inverse from the affine relationship

Equation (3.7) already identifies the reverse rotation and translation. Place these in their corresponding blocks:

$$\boxed{H_W^R = (H_R^W)^{-1} = \begin{bmatrix} (R_R^W)^\mathsf{T} & -(R_R^W)^\mathsf{T} p_R^W \\ 0_{1\times 2} & 1 \end{bmatrix}.} \tag{3.19}$$

This gives $\widetilde{B}^R = H_W^R \widetilde{B}^W$ without a general-purpose $3 \times 3$ inversion. Transpose the rotation block, compute the rotated negative translation, and retain the bottom row.

For a compact verification, write $R = R_R^W$ and $p = p_R^W$. Then

$$\begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \begin{bmatrix} R^\mathsf{T} & -R^\mathsf{T} p \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} RR^\mathsf{T} & -RR^\mathsf{T} p + p \\ 0 & 1 \end{bmatrix} = I_3.$$

The reverse product also equals $I_3$, since its translation block is $R^\mathsf{T} p - R^\mathsf{T} p = 0$.

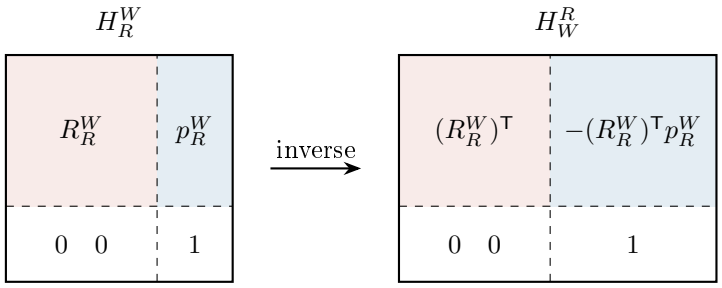



Figure 3.4: Block structure of a transform and its inverse. Red blocks contain rotation; blue blocks contain translation. The inverse translation must be rotated as well as negated. Block widths are schematic.

### 3.6.3 Worked inverse

For $p_R^W = [2\ 1]^\mathsf{T}$ m and $\theta_R^W = \pi/6$,

$$\begin{aligned} -(R_R^W)^\mathsf{T} p_R^W &= -\begin{bmatrix} \sqrt{3}/2 & 1/2 \\ -1/2 & \sqrt{3}/2 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} \\ &= \begin{bmatrix} -\sqrt{3} - 1/2 \\ 1 - \sqrt{3}/2 \end{bmatrix} \approx \begin{bmatrix} -2.2321 \\ 0.1340 \end{bmatrix} \text{ m}. \end{aligned}$$

Hence, with translation entries in metres,

$$H_W^R = \begin{bmatrix} \sqrt{3}/2 & 1/2 & -\sqrt{3} - 1/2 \\ -1/2 & \sqrt{3}/2 & 1 - \sqrt{3}/2 \\ 0 & 0 & 1 \end{bmatrix}. \tag{3.20}$$

Its last column locates the world origin in robot coordinates. Its upper-left block describes the world axes in the robot frame. These meanings provide geometric checks on the inverse, in addition to verifying the matrix product.

## 3.7 Key identities

| Operation | Identity |
|---|---|
| Forward point transformation | $B^W = p_R^W + R_R^W B^R$ |
| Reverse point transformation | $B^R = (R_R^W)^\mathsf{T}(B^W - p_R^W)$ |
| Homogeneous point transformation | $\widetilde{B}^W = H_R^W \widetilde{B}^R$ |
| Composition | $H_S^W = H_R^W H_S^R$ |
| Reverse rotation | $R_W^R = (R_R^W)^\mathsf{T}$ |
| Reverse translation | $p_W^R = -(R_R^W)^\mathsf{T} p_R^W$ |

Always specify the frame of a translation, express displacements in a common frame before adding them, and distinguish the inverse of a rotation matrix from the inverse of a full homogeneous transformation.

## 3.8 Additional exercises

Use metres for all position coordinates and counterclockwise-positive angles. Show your calculations and keep exact values until the final numerical step.

1. **A translated and rotated robot.** Let $p_R^W = [3\ 1]^\mathsf{T}$, $\theta_R^W = \pi/2$, and $B^R = [2\ -1]^\mathsf{T}$. Sketch the two frames and the point. Compute $B^W$ using the affine relationship and explain why $p_R^W + B^R$ gives a different result.

2. **Recovering robot coordinates.** Suppose $p_R^W = [1\ -2]^\mathsf{T}$, $\theta_R^W = -\pi/2$, and $B^W = [4\ -3]^\mathsf{T}$. Find $B^R$. Substitute it into the forward relationship to verify your answer. State the frame of every vector being added.

3. **Building a homogeneous matrix.** Construct $H_R^W$ for $p_R^W = [2\ 0]^\mathsf{T}$ and $\theta_R^W = \pi/4$. Use it to transform $B^R = [\sqrt{2}\ 0]^\mathsf{T}$. Identify the physical coordinates in the output and explain the role of its last entry.

4. **World–robot–sensor composition.** Let $p_R^W = [2\ 1]^\mathsf{T}$, $\theta_R^W = \pi/2$, $p_S^R = [1\ 0]^\mathsf{T}$, and $\theta_S^R = -\pi/2$. Find $H_S^W = H_R^W H_S^R$ and use it to transform $B^S = [1\ 2]^\mathsf{T}$. Check your result by applying the two affine transformations separately.

5. **Inversion without a general matrix inverse.** For

$$H_R^W = \begin{bmatrix} 0 & -1 & 2 \\ 1 & 0 & 3 \\ 0 & 0 & 1 \end{bmatrix},$$

   compute $H_W^R$ using its rotation and translation blocks. Verify $H_W^R H_R^W = I_3$. Compare your result with $(H_R^W)^\mathsf{T}$ and identify why the transpose fails.

6. **Order of operations.** Consider the two maps, expressed using a fixed coordinate system,

$$T = \begin{bmatrix} 1 & 0 & 1 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad Q = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

   Compute $QT$ and $TQ$ and apply each to $[0\ 0\ 1]^\mathsf{T}$. Explain geometrically why rotating after translating differs from translating after rotating.

7. **Distances between points.** Two points obey $B^W = p + RB^R$ and $C^W = p + RC^R$, with $R$ orthogonal. Show that $B^W - C^W = R(B^R - C^R)$ and use this to prove that their distance is preserved. Why need the distance of a single point from its coordinate origin not be preserved?

# Chapter 4

# Three-dimensional rotations and orientation representations

A robot that moves outside a plane requires a spatial description of orientation. One angle no longer suffices: a body can turn about three independent directions. The construction of a rotation matrix remains the same, however. Its columns are the unit axes of one frame expressed in another. In three dimensions, these columns have three components, giving a $3 \times 3$ matrix.

This chapter constructs rotations about individual axes, combines them into a yaw–pitch–roll sequence, recovers those angles from a rotation matrix, and introduces unit quaternions as a practical orientation representation. All frames used for point-coordinate transformations here share an origin; translation is set aside so that the orientation conventions can be developed explicitly.

## 4.1 Spatial frames and rotation matrices

Let $W$ be a right-handed world frame and $R$ a right-handed robot frame. Each has three mutually perpendicular unit axes. Write the robot's axes in world components as $[\boldsymbol{x}_R]^W$, $[\boldsymbol{y}_R]^W$, and $[\boldsymbol{z}_R]^W$. Then

$$R_R^W = \begin{bmatrix} [\boldsymbol{x}_R]^W & [\boldsymbol{y}_R]^W & [\boldsymbol{z}_R]^W \end{bmatrix} \in \mathbb{R}^{3\times 3}. \tag{4.1}$$

As in the planar case, the subscript gives the input frame and the superscript gives the output frame of the coordinate map. For a point $B$ and common origins,

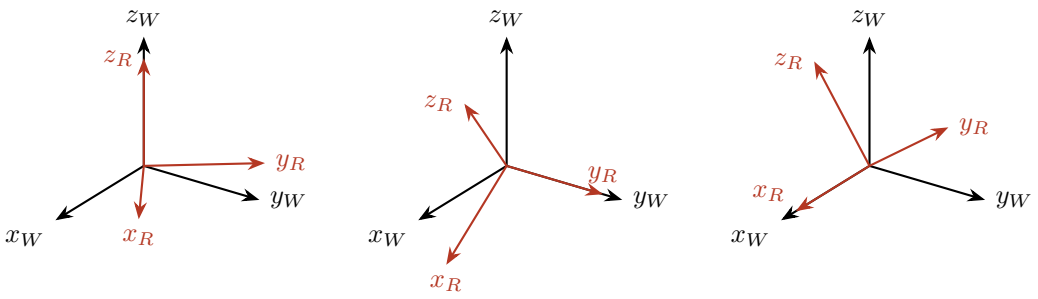
$$B^W = R_R^W B^R, \qquad B^R = \begin{bmatrix} x_B^R \\ y_B^R \\ z_B^R \end{bmatrix}. \tag{4.2}$$

The same rotation maps the components of a direction vector between frames regardless of the locations of their origins.

Positive elementary angles follow the right-hand rule: point the right thumb along the positive rotation axis and curl the fingers in the positive direction. Viewed from the positive end of that axis toward the origin, the positive rotation appears counterclockwise. A perspective drawing may distort lengths and visible angles; it does not change this convention.

## 4.2 Elementary rotations

An elementary rotation leaves one axis fixed and rotates the perpendicular plane. The three basic cases are shown in Figure 4.1. Each is constructed by expressing the rotated basis vectors in the original frame, then placing those vectors in columns.




$R_z$: rotation about $z$ $R_y$: rotation about $y$ $R_x$: rotation about $x$

Figure 4.1: Elementary rotations in perspective. Black axes are the world frame; red axes are the rotated frame. The rotation axis is shared by the two frames. The three panels are separate constructions, each with a positive angle.

### 4.2.1 Rotation about the z axis

Let $\psi$ denote a rotation about $z_W$. The $z$ axis remains unchanged, while the $x$ and $y$ axes rotate in the $xy$ plane:

$$[\boldsymbol{x}_R]^W = \begin{bmatrix} \cos\psi \\ \sin\psi \\ 0 \end{bmatrix}, \quad [\boldsymbol{y}_R]^W = \begin{bmatrix} -\sin\psi \\ \cos\psi \\ 0 \end{bmatrix}, \quad [\boldsymbol{z}_R]^W = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}.$$

Therefore,

$$\boxed{R_z(\psi) = \begin{bmatrix} \cos\psi & -\sin\psi & 0 \\ \sin\psi & \cos\psi & 0 \\ 0 & 0 & 1 \end{bmatrix}}. \tag{4.3}$$

The upper-left block is exactly the planar rotation matrix. The last coordinate is unchanged because the rotation takes place in the plane perpendicular to $z$.

### 4.2.2 Rotation about the y axis

For a positive angle $\theta$ about $y_W$, the $x$ axis turns toward negative $z$ and the $z$ axis turns toward positive $x$. The basis columns are

$$[\boldsymbol{x}_R]^W = \begin{bmatrix} \cos\theta \\ 0 \\ -\sin\theta \end{bmatrix}, \quad [\boldsymbol{y}_R]^W = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}, \quad [\boldsymbol{z}_R]^W = \begin{bmatrix} \sin\theta \\ 0 \\ \cos\theta \end{bmatrix}.$$

Thus

$$\boxed{R_y(\theta) = \begin{bmatrix} \cos\theta & 0 & \sin\theta \\ 0 & 1 & 0 \\ -\sin\theta & 0 & \cos\theta \end{bmatrix}.} \tag{4.4}$$

The signs follow from the right-hand rule. The two-dimensional pattern is visible in the ordered plane $(z, x)$, where positive rotation carries $z$ toward $x$; using the order $(x, z)$ without accounting for orientation would reverse the signs.

### 4.2.3 Rotation about the x axis

For a positive angle $\varphi$ about $x_W$, the $x$ axis is fixed and the $y$ axis turns toward positive $z$. Hence

$$[\boldsymbol{x}_R]^W = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}, \quad [\boldsymbol{y}_R]^W = \begin{bmatrix} 0 \\ \cos\varphi \\ \sin\varphi \end{bmatrix}, \quad [\boldsymbol{z}_R]^W = \begin{bmatrix} 0 \\ -\sin\varphi \\ \cos\varphi \end{bmatrix},$$

and

$$\boxed{R_x(\varphi) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\varphi & -\sin\varphi \\ 0 & \sin\varphi & \cos\varphi \end{bmatrix}.} \tag{4.5}$$

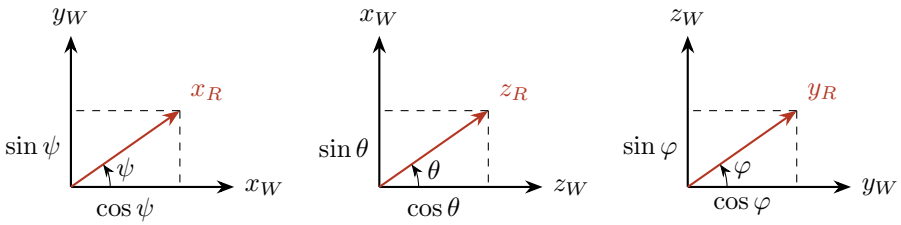



Figure 4.2: Plane views of the elementary rotations. From left to right, the ordered planes are $(x_W, y_W)$, $(z_W, x_W)$, and $(y_W, z_W)$. Each red vector has unit length; its projections explain the sine and cosine entries.

A quick sign check is to use a quarter-turn: $R_z(\pi/2)$ maps the first basis vector to the second, $R_y(\pi/2)$ maps the third to the first, and $R_x(\pi/2)$ maps the second to the third. At zero angle, all three matrices reduce to $I_3$.

## 4.3 What makes a matrix a valid spatial rotation?

Write $R = [u\ v\ w]$. Its columns must be unit vectors and mutually perpendicular:

$$u^\mathsf{T}u = v^\mathsf{T}v = w^\mathsf{T}w = 1, \qquad u^\mathsf{T}v = u^\mathsf{T}w = v^\mathsf{T}w = 0. \tag{4.6}$$

Together these six scalar constraints give $R^\mathsf{T}R = I_3$. They constrain the nine matrix entries to three continuous degrees of freedom. To describe a rotation between right-handed frames, rather than a reflection, one also requires $\det R = +1$. The determinant condition selects the proper-rotation component; it does not remove another continuous degree of freedom.

For a valid rotation,

$$R^{-1} = R^{\mathsf{T}}, \qquad R_F^A = (R_A^F)^{\mathsf{T}}. \tag{4.7}$$

Composition still follows the frame chain, $R_C^A = R_B^A R_C^B$, and lengths are preserved. These are the same structural properties used in two dimensions.

### 4.3.1 An illustrative set of components

To illustrate how columns are assembled, consider the rough component array

$$M = \begin{bmatrix} .8 & -.3 & .4 \\ .2 & .7 & .15 \\ -.1 & .2 & .7 \end{bmatrix}. \tag{4.8}$$

Its first column describes a direction with a large positive $x$ component, a smaller positive $y$ component, and a negative $z$ component. The other columns can be read in the same way. However, these illustrative numbers do *not* define a rotation matrix. For example,

$$u^{\mathsf{T}}u = .8^2 + .2^2 + (-.1)^2 = .69 \neq 1, \qquad u^{\mathsf{T}}v = .8(-.3) + .2(.7) + (-.1)(.2) = -.12 \neq 0.$$

Even normalizing each column separately would not make the columns mutually perpendicular. A valid frame must satisfy all the orthonormality conditions and the right-handedness condition, not just have plausible-looking components.

## 4.4 Roll, pitch, and yaw as a sequence of rotations

A general spatial orientation can be represented by a sequence of three elementary rotations. Many sequences are possible, and the convention must be specified before interpreting three angle values. Here the convention is the *intrinsic Z–Y–X sequence*: each successive rotation is about an axis of the frame obtained after the previous rotation.

Starting from frame $W$, define intermediate frames 1 and 2 and final frame 3:

1. Rotate by yaw $\psi$ about $z_W$ to obtain frame 1.
2. Rotate by pitch $\theta$ about the new axis $y_1$ to obtain frame 2.
3. Rotate by roll $\varphi$ about the new axis $x_2$ to obtain frame 3.

Although the angles are often listed as roll, pitch, yaw, the construction above proceeds through yaw, pitch, then roll.

The elementary frame relationships are

$$R_1^W = R_z(\psi), \qquad R_2^1 = R_y(\theta), \qquad R_3^2 = R_x(\varphi).$$

Applying the frame-chain rule gives

$$\boxed{R_3^W = R_1^W R_2^1 R_3^2 = R_z(\psi)R_y(\theta)R_x(\varphi).} \tag{4.9}$$

This product maps coordinates from frame 3 to $W$. When it multiplies $B^3$, the rightmost factor acts first: $B^3 \to B^2 \to B^1 \to B^W$. This algebraic order is consistent with constructing the frames successively about their moving axes; it should not be confused with three rotations about fixed world axes in the same chronological order.

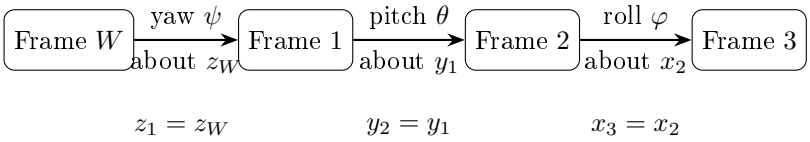



Figure 4.3: The intrinsic Z–Y–X construction. The axis used at each stage stays fixed during that stage. Arrows show the sequence of frame construction; coordinate maps from frame 3 back to $W$ act in the reverse traversal.

### 4.4.1 Order matters in three dimensions

Unlike planar rotations, rotations about different spatial axes do not generally commute. With $e_x = [1\ 0\ 0]^{\mathsf{T}}$,

$$R_z(\pi/2)R_x(\pi/2)e_x = e_y, \qquad R_x(\pi/2)R_z(\pi/2)e_x = e_z.$$

The first product leaves $e_x$ unchanged in its first step, then turns it toward $e_y$. The second turns it toward $e_y$ first, then toward $e_z$. Thus changing the factor order generally changes the orientation.

### 4.4.2 Expanded form

For compactness, write $c_a = \cos a$ and $s_a = \sin a$. Multiplying the three elementary matrices yields

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\varphi - s_\psi c_\varphi & c_\psi s_\theta c_\varphi + s_\psi s_\varphi \\ s_\psi c_\theta & s_\psi s_\theta s_\varphi + c_\psi c_\varphi & s_\psi s_\theta c_\varphi - c_\psi s_\varphi \\ -s_\theta & c_\theta s_\varphi & c_\theta c_\varphi \end{bmatrix}. \tag{4.10}$$

The factored form is often easier to evaluate and check. The expanded form is particularly useful for identifying which entries contain the information needed to recover the angles.

## 4.5 Worked example

Let the roll, pitch, and yaw angles be

$$\varphi = \frac{\pi}{4}, \qquad \theta = \frac{\pi}{3}, \qquad \psi = \frac{\pi}{2}.$$

Then

$$R_3^W = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1/2 & 0 & \sqrt{3}/2 \\ 0 & 1 & 0 \\ -\sqrt{3}/2 & 0 & 1/2 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 \\ 0 & \sqrt{2}/2 & -\sqrt{2}/2 \\ 0 & \sqrt{2}/2 & \sqrt{2}/2 \end{bmatrix} \tag{4.11}$$

$$= \begin{bmatrix} 0 & -\sqrt{2}/2 & \sqrt{2}/2 \\ 1/2 & \sqrt{6}/4 & \sqrt{6}/4 \\ -\sqrt{3}/2 & \sqrt{2}/4 & \sqrt{2}/4 \end{bmatrix}. \tag{4.12}$$

For instance, the first column has squared length $1/4 + 3/4 = 1$ and has zero dot product with the second column. More generally, the product of these three proper rotation matrices is itself a proper rotation matrix.

If $B^3 = [1\ 0\ 0]^{\mathsf{T}}$, its world coordinates are the first column of $R_3^W$. To return to frame 3, use $(R_3^W)^{\mathsf{T}}$. Thus the orientation representation serves directly as a coordinate transformation.

## 4.6 Recovering roll, pitch, and yaw

Let a valid rotation matrix be written as

$$R = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}.$$

For the convention in Equation (4.9), choose the principal pitch range $-\pi/2 < \theta < \pi/2$. In this nonsingular range,

$$\boxed{\begin{aligned} &\varphi = \operatorname{atan2}(r_{32}, r_{33}), \\ &\theta = \operatorname{atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right), \\ &\psi = \operatorname{atan2}(r_{21}, r_{11}). \end{aligned}} \tag{4.13}$$

The entries in Equation (4.10) explain these formulas:

$$(r_{32}, r_{33}) = c_\theta(s_\varphi, c_\varphi), \qquad (r_{21}, r_{11}) = c_\theta(s_\psi, c_\psi).$$

Since $c_\theta > 0$ on the chosen range, each pair has the same direction as the corresponding sine–cosine pair. Also,

$$-r_{31} = s_\theta, \qquad \sqrt{r_{32}^2 + r_{33}^2} = |c_\theta| = c_\theta.$$

Applied to Equation (4.12), the formulas give

$$\varphi = \operatorname{atan2}(\sqrt{2}/4, \sqrt{2}/4) = \pi/4, \quad \theta = \operatorname{atan2}(\sqrt{3}/2, 1/2) = \pi/3,$$

and $\psi = \operatorname{atan2}(1/2, 0) = \pi/2$, recovering the original angles.

### 4.6.1 Why atan2 takes two arguments

The function $\operatorname{atan2}(y, x)$ returns the angle of the pair $(x, y)$, using both signs to identify the quadrant. The argument order here is *vertical component first, horizontal component second.* The numeral 2 is part of the function's name, not a multiplication or exponent.

The ratio $y/x$ alone loses information: $(1, 1)$ and $(-1, -1)$ both have ratio one, but point in opposite directions. Ordinary $\arctan(y/x)$ returns the same principal angle for both, whereas

$$\operatorname{atan2}(1, 1) = \pi/4, \qquad \operatorname{atan2}(-1, -1) = -3\pi/4.$$

Unlike division followed by arctangent, atan2 also handles $x = 0$ with $y \neq 0$, giving $\pi/2$ or $-\pi/2$.

Tangent has period $\pi$, while a full revolution has length $2\pi$. The principal arctangent takes values in $(-\pi/2, \pi/2)$; a conventional principal range for atan2 spans $2\pi$, such as $(-\pi, \pi]$. At the negative horizontal axis, $\pi$ and $-\pi$ represent the same direction; endpoint handling can vary between implementations. At $(x, y) = (0, 0)$ there is no geometric direction, whatever numerical value a software routine returns.

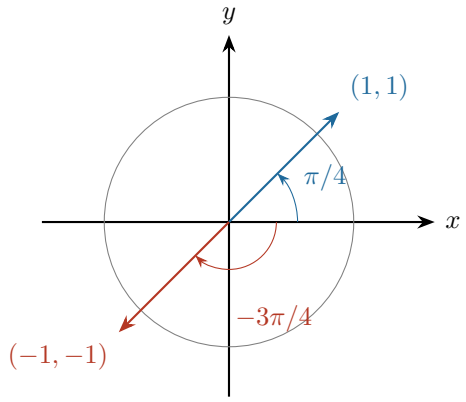



Figure 4.4: Equal ratios do not imply equal directions. The signs of both components distinguish the first and third quadrants. The circle indicates direction only; the two labeled vectors need not have unit length.

### 4.6.2 Singular orientations and angle ranges

The three-angle representation covers all orientations, but is not globally unique or nonsingular. At $\theta = \pm\pi/2$, the pairs used to extract yaw and roll both vanish:

$$r_{32} = r_{33} = r_{21} = r_{11} = 0.$$

Consequently, Equation (4.13) cannot determine yaw and roll separately. At $\theta = \pi/2$, for example, the matrix depends on their difference $\varphi - \psi$. This is the Euler-angle singularity often called *gimbal lock*. It is a limitation of the angle coordinates, not a loss of information in the rotation matrix.

An implementation must choose a convention for the singular case rather than treat $\operatorname{atan2}(0, 0)$ as a uniquely meaningful angle. Near this case, small matrix perturbations can produce large changes in extracted yaw and roll. Away from it, the principal ranges are typically roll and yaw in a $2\pi$ interval, and pitch in $[-\pi/2, \pi/2]$.

## 4.7 Choosing and maintaining an orientation representation

Angles make individual turns easy to describe, while matrices make coordinate transformation and composition direct. The map $B^W = R_R^W B^R$ is linear in the coordinates of $B$ for a fixed rotation; the dependence of $R_R^W$ on the angles remains trigonometric.

Finite-precision arithmetic can cause repeatedly multiplied rotation matrices to drift from orthogonality. Useful checks are whether $R^\mathsf{T} R$ remains close to $I_3$ and whether $\det R$ remains close to $+1$. Restoring unit column lengths alone is insufficient: perpendicularity and handedness must also be retained. A correction procedure must restore a proper orthonormal frame.

Angle representations have their own difficulties, including wrapping at the ends of the chosen intervals and singular orientations. Unit quaternions provide another representation used in robotics software. No representation removes the need to specify conventions and manage numerical error.

### 4.7.1 Unit quaternions in practice

A unit quaternion represents orientation using four real components. With the scalar component first, write

$$q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^{\mathsf{T}}, \qquad q^{\mathsf{T}} q = q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1. \tag{4.14}$$

Four components with one unit-norm constraint leave three continuous degrees of freedom, just as for a rotation matrix. The quaternions $q$ and $-q$ represent the same orientation; this discrete ambiguity does not add a degree of freedom. Component order must always be checked when exchanging data: some software places the scalar component last.

For a nonzero quaternion affected by small numerical drift, restoring unit norm is straightforward:

$$q \leftarrow \frac{q}{\|q\|}, \qquad \|q\| = \sqrt{q_w^2 + q_x^2 + q_y^2 + q_z^2}.$$

A zero quaternion cannot be normalized and does not represent an orientation. Normalization enforces the representation's constraint; it does not undo every numerical error in the represented orientation.

The four components are less immediately intuitive than roll, pitch, and yaw, but are convenient for storing and exchanging orientations. A useful workflow is to retain a unit quaternion in software, convert to a rotation matrix for coordinate calculations, and extract angles when interpreting the result. To compose two orientations using familiar matrix algebra, convert both to matrices, multiply in the appropriate frame order, and convert back if needed. Direct quaternion composition is also possible and efficient; its algebra is outside the scope of this introduction.

Appendix A gives all six conversion directions between matrices, RPY angles, and scalar-first unit quaternions, including their validity conditions. For example, a quaternion can be converted to RPY angles to inspect yaw, while remembering that the chosen angle representation still has its pitch singularities.

## 4.8 Key identities

| Property or operation | Rule |
|---|---|
| Proper rotation | $R^{\mathsf{T}} R = I_3$, $\det R = 1$ |
| Coordinate transformation, common origin | $B^W = R_R^W B^R$ |
| Reverse transformation | $R_W^R = (R_R^W)^{\mathsf{T}}$ |
| Frame composition | $R_C^A = R_B^A R_C^B$ |
| Intrinsic Z–Y–X convention | $R = R_z(\psi) R_y(\theta) R_x(\varphi)$ |
| Nonsingular principal pitch range | $-\pi/2 < \theta < \pi/2$ |

## 4.9 Additional exercises

Use right-handed frames, column vectors, and the intrinsic Z–Y–X convention. Show your calculations and keep exact trigonometric values where possible.

1. **Elementary rotations.** Write $R_z(\pi/2)$, $R_y(\pi/2)$, and $R_x(\pi/2)$. For each matrix, identify its unchanged axis and determine the images of all three standard basis vectors. Check the signs using the right-hand rule.

2. **Checking a candidate matrix.** For the matrix $M$ in Equation (4.8), compute the squared lengths of its three columns and all three pairwise dot products. Explain why normalizing the columns individually cannot produce a valid rotation matrix.

3. **A valid spatial orientation.** Consider

$$R = \begin{bmatrix} 0 & -1 & 0 \\ 1 & 0 & 0 \\ 0 & 0 & 1 \end{bmatrix}, \qquad B^R = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}.$$

   Compute $B^W = RB^R$, recover $B^R$ using the transpose, and verify length preservation. State which elementary rotation $R$ represents.

4. **Noncommutativity.** Compute $R_x(\pi/2)R_y(\pi/2)$ and $R_y(\pi/2)R_x(\pi/2)$. Apply both to $e_z = [0\ 0\ 1]^\mathsf{T}$. Explain why the two results differ.

5. **From angles to a matrix and back.** Take $\varphi = \pi/4$, $\theta = 0$, and $\psi = \pi/2$. Compute $R = R_z(\psi)R_y(\theta)R_x(\varphi)$, then recover all three angles with Equation (4.13). Identify the frame axis used at each stage of the intrinsic construction.

6. **Quadrants and atan2.** Evaluate $\operatorname{atan2}(1, -1)$, $\operatorname{atan2}(-1, -1)$, and $\operatorname{atan2}(-1, 0)$. Sketch the corresponding directions. Explain what goes wrong if each result is computed using only $\arctan(y/x)$.

7. **A singular orientation.** Set $\theta = \pi/2$ in Equation (4.10). Show that the result depends on $\varphi - \psi$. Verify that the triples $(\varphi, \theta, \psi) = (0, \pi/2, 0)$ and $(\pi/4, \pi/2, \pi/4)$ give the same matrix. Why can the extraction formulas not recover the two triples as distinct orientations?

# Chapter 5

# Three-dimensional rigid transformations and pose

A spatial reference frame can differ from another in both its origin and its orientation. Combining translation with the rotations developed in Chapter 4 gives a description of the pose of a rigid body. The geometry follows the planar construction: rotate a point's coordinates into the destination frame, then add the displacement between origins. The vectors now have three components, and homogeneous transformations have four rows and four columns.

## 5.1 Combining spatial rotation and translation

Let $W$ be a world frame and $R$ a frame attached to a rigid body. Define

$$p_R^W \in \mathbb{R}^3, \qquad R_R^W \in \mathbb{R}^{3\times 3},$$

where $p_R^W$ is the position of the origin $O_R$ expressed in $W$, and $R_R^W$ maps vector components from $R$ to $W$. The matrix is a proper rotation: $(R_R^W)^{\mathsf{T}} R_R^W = I_3$ and $\det R_R^W = 1$.

If a sensor reports a point $B$ using coordinates $B^R$, its world coordinates are obtained by adding two displacements expressed in the same frame:

$$\boxed{B^W = R_R^W B^R + p_R^W .} \tag{5.1}$$

The first term describes the displacement from $O_R$ to $B$ in world components; the second locates $O_R$ relative to $O_W$.

The dimensions provide a useful check:

$$\underbrace{B^W}_{3\times 1} = \underbrace{R_R^W}_{3\times 3}\underbrace{B^R}_{3\times 1} + \underbrace{p_R^W}_{3\times 1}.$$

Rotation alone maps direction vectors between frames. Translation is required for point coordinates because the two origins can differ. For example, subtracting the transformed coordinates of two points cancels $p_R^W$, leaving only the rotation of their displacement.

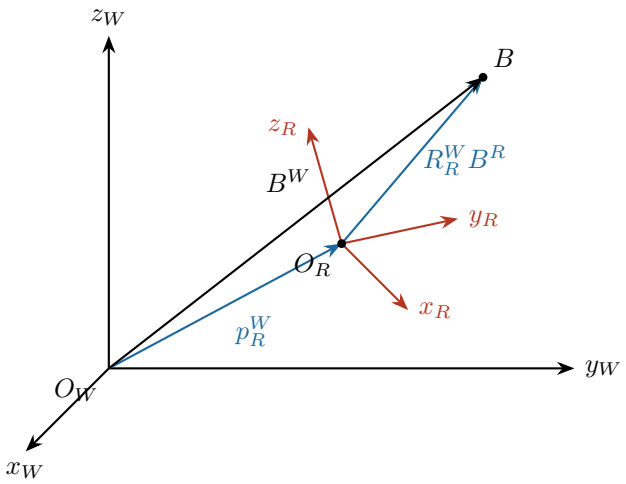



Figure 5.1: Spatial coordinate transformation. The displacement from $O_W$ to $B$ is the sum of the displacement to $O_R$ and the displacement from $O_R$ to $B$. All three labeled displacement vectors use world components. The perspective sketch illustrates the geometry, not the numerical example.

### 5.1.1 Worked example

Take

$$p_R^W = \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix}, \qquad B^R = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}, \qquad R_R^W = R_z(\pi/4) = \begin{bmatrix} c & -c & 0 \\ c & c & 0 \\ 0 & 0 & 1 \end{bmatrix}, \quad c = \frac{\sqrt{2}}{2}.$$

The rotation is about $z$, chosen to simplify the arithmetic; the transformation law applies equally to a general spatial rotation. Substitution gives

$$B^W = \begin{bmatrix} 2c \\ 2c \\ 1 \end{bmatrix} + \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} = \begin{bmatrix} 1+\sqrt{2} \\ \sqrt{2} \\ 3 \end{bmatrix}. \tag{5.2}$$

## 5.2 The reverse coordinate transformation

To recover the coordinates in $R$, first subtract the origin's world position and then apply the inverse rotation:

$$\boxed{B^R = (R_R^W)^{\mathsf{T}}(B^W - p_R^W).} \tag{5.3}$$

The order matters. The subtraction takes place in $W$; the transpose then converts that displacement into $R$ components. For the example,

$$B^R = \begin{bmatrix} c & c & 0 \\ -c & c & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \sqrt{2} \\ \sqrt{2} \\ 1 \end{bmatrix} = \begin{bmatrix} 2 \\ 0 \\ 1 \end{bmatrix}.$$

## 5.3 Homogeneous transformations in space

Introduce homogeneous point coordinates by appending a one:

$$\widetilde{B}^R = \begin{bmatrix} B^R \\ 1 \end{bmatrix} \in \mathbb{R}^4.$$

The spatial homogeneous transformation is

$$H_R^W = \begin{bmatrix} R_R^W & p_R^W \\ 0_{1\times 3} & 1 \end{bmatrix} \in \mathbb{R}^{4\times 4}, \qquad \boxed{\widetilde{B}^W = H_R^W \widetilde{B}^R.} \tag{5.4}$$

The lower-left block contains three zeros. Block multiplication reproduces Equation (5.1) in the first three rows and preserves the final one. Lines drawn between blocks are visual guides, not extra operations.

For the worked example,

$$H_R^W = \begin{bmatrix} c & -c & 0 & 1 \\ c & c & 0 & 0 \\ 0 & 0 & 1 & 2 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \qquad H_R^W \begin{bmatrix} 2 \\ 0 \\ 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 1+\sqrt{2} \\ \sqrt{2} \\ 3 \\ 1 \end{bmatrix}.$$

The last entry is a consistency check for this representation of a point under a rigid transformation. A direction vector is instead augmented with zero; then multiplication rotates the direction without adding a translation.

### 5.3.1 Composition follows the frame chain

The advantage of homogeneous coordinates remains the same as in the planar case. For an intermediate frame $R$ and a sensor frame $S$,

$$H_S^W = H_R^W H_S^R. \tag{5.5}$$

Expanding the blocks gives

$$H_S^W = \begin{bmatrix} R_R^W R_S^R & R_R^W p_S^R + p_R^W \\ 0_{1\times 3} & 1 \end{bmatrix}.$$

Thus the rotation composes by multiplication, while the sensor-origin displacement must be rotated into $W$ before adding $p_R^W$. The rightmost map acts first: $S \to R \to W$. This is the spatial version of the earlier frame-chain rule.

## 5.4 Inverting a homogeneous transformation

Rearranging Equation (5.3) separates the reverse rotation and translation:

$$B^R = (R_R^W)^\mathsf{T} B^W - (R_R^W)^\mathsf{T} p_R^W.$$

Consequently,

$$\boxed{H_W^R = (H_R^W)^{-1} = \begin{bmatrix} (R_R^W)^\mathsf{T} & -(R_R^W)^\mathsf{T} p_R^W \\ 0_{1\times 3} & 1 \end{bmatrix}.} \tag{5.6}$$

This uses the structure of a rigid transformation instead of a general $4 \times 4$ inversion. Multiplying the original and inverse blocks gives $R_R^W (R_R^W)^\mathsf{T} = I_3$ and translation $-R_R^W (R_R^W)^\mathsf{T} p_R^W + p_R^W = 0$, verifying the result.

### 5.4.1 Why reversing the translation requires a rotation

The upper-right block of the inverse is

$$\boxed{p_W^R = -(R_R^W)^\mathsf{T} p_R^W .} \tag{5.7}$$

Geometrically, the displacement from $O_R$ to $O_W$ is opposite to the displacement from $O_W$ to $O_R$. However, $p_R^W$ uses world components, whereas $p_W^R$ uses robot components. Negating the former reverses the physical displacement but leaves its components expressed in $W$. The transpose is needed to express them in $R$.

In the example,

$$p_W^R = -\begin{bmatrix} c & c & 0 \\ -c & c & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 2 \end{bmatrix} = \begin{bmatrix} -c \\ c \\ -2 \end{bmatrix},$$

which differs from $-p_R^W = [-1\ 0\ -2]^\mathsf{T}$. The full reverse transformation is

$$H_W^R = \begin{bmatrix} c & c & 0 & -c \\ -c & c & 0 & c \\ 0 & 0 & 1 & -2 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

Applying it to $[1+\sqrt{2}\ \sqrt{2}\ 3\ 1]^\mathsf{T}$ returns $[2\ 0\ 1\ 1]^\mathsf{T}$, as required.

## 5.5 What rounding changes

Replacing $c = \sqrt{2}/2$ by 0.7 makes the forward calculation especially simple. Denote the resulting approximate rotation by

$$\widehat{R} = \begin{bmatrix} .7 & -.7 & 0 \\ .7 & .7 & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

With the same translation and input point, it gives

$$\widehat{B}^W = \widehat{R} B^R + p_R^W = \begin{bmatrix} 2.4 \\ 1.4 \\ 3 \end{bmatrix}.$$

If we now use the transpose as though it were the inverse, the return calculation produces

$$\widehat{R}^\mathsf{T} (\widehat{B}^W - p_R^W) = \begin{bmatrix} .7 & .7 & 0 \\ -.7 & .7 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1.4 \\ 1.4 \\ 1 \end{bmatrix} = \begin{bmatrix} 1.96 \\ 0 \\ 1 \end{bmatrix}.$$

The discrepancy has a precise explanation:

$$\widehat{R}^{\mathsf{T}}\widehat{R} = \operatorname{diag}(.98, .98, 1) \neq I_3. \tag{5.8}$$

After rounding, $\widehat{R}$ is not an exact rotation matrix, and its transpose is not its exact inverse. The return calculation scales the first two input components by 0.98. The exact radical form returns the original point without this error.

This deliberately coarse rounding illustrates why orthogonality matters. Using the actual algebraic inverse of $\widehat{R}$ would reverse its own forward map exactly in exact arithmetic, but that map would still fail to be a rigid rotation. In numerical work, keep adequate precision and monitor the rotation constraints rather than assuming that a matrix labeled as a rotation automatically satisfies them.

## 5.6 Position, orientation, and pose

*Position* specifies the location of a chosen body-fixed origin relative to a reference frame. In space it has three degrees of freedom. *Orientation* specifies how the body-fixed axes are aligned relative to the reference axes and has three more continuous degrees of freedom. Together, position and orientation define the body's *pose*. An unconstrained rigid body in space therefore has six degrees of freedom.

The number of stored components need not equal the number of degrees of freedom. A rotation matrix contains nine entries subject to six independent orthonormality constraints, with determinant $+1$ selecting proper rotations. A unit quaternion has four components subject to one norm constraint. Both describe three continuous degrees of freedom. RPY angles use three parameters directly, with the singularity and nonuniqueness discussed in Chapter 4.

| Pose representation | Stored entries | Main condition or limitation |
|---|---|---|
| $H_R^W$ | 16 | Rigid-transform block structure |
| Position and RPY | 6 | Angle ranges and singularities |
| Position and quaternion | 7 | Unit quaternion; $q$ and $-q$ equivalent |

For example, two vector representations are

$$\begin{bmatrix} p_R^W \\ \varphi_R^W \\ \theta_R^W \\ \psi_R^W \end{bmatrix} \in \mathbb{R}^6, \qquad \begin{bmatrix} p_R^W \\ q_R^W \end{bmatrix} \in \mathbb{R}^7,$$

where $q_R^W$ represents the same orientation as $R_R^W$ under the scalar-first convention. Stacking the components does not make pose composition ordinary vector addition. To combine poses, use the appropriate rotation and translation rules, conveniently encoded by homogeneous matrices.

Once both the position and orientation of a body-fixed frame are specified, the locations of all points fixed to that rigid body follow from their constant coordinates in that frame. This is why a single pose describes the placement of the entire rigid body.

## 5.7 Additional exercises

Use right-handed frames and exact values unless rounding is explicitly requested.

1. **A spatial point transformation.** Let $R_R^W = R_x(\pi/2)$, $p_R^W = [1\ -2\ 3]^\mathsf{T}$, and $B^R = [2\ 1\ 0]^\mathsf{T}$. Find $B^W$ and recover $B^R$ using the reverse map.

2. **Homogeneous form.** Build $H_R^W$ for Exercise 1 and use it to transform $\widetilde{B}^R$. Build $(H_R^W)^{-1}$ using its blocks and verify that their product is $I_4$.

3. **Reversing a displacement.** For $R_R^W = R_z(\pi/2)$ and $p_R^W = [2\ 1\ 3]^\mathsf{T}$, compute $p_W^R$. Compare it with $-p_R^W$ and explain the frame difference. Verify that $R_R^W p_W^R + p_R^W = 0$.

4. **A sensor frame.** Suppose $R_R^W = R_z(\pi/2)$, $p_R^W = [1\ 0\ 2]^\mathsf{T}$, $R_S^R = I_3$, and $p_S^R = [1\ 0\ 1]^\mathsf{T}$. Find $H_S^W$ and the world coordinates of $B^S = [0\ 2\ 0]^\mathsf{T}$. Check your result by transforming first to $R$ and then to $W$.

5. **Points and directions.** Using the transformation in Exercise 1, transform the points $A^R = [0\ 0\ 0]^\mathsf{T}$ and $B^R = [2\ 1\ 0]^\mathsf{T}$. Show that $B^W - A^W = R_R^W(B^R - A^R)$. Explain why a homogeneous direction uses a final zero instead of a one.

6. **Rounding and orthogonality.** Replace $c$ in the worked example by $a = 0.707$. Derive $\widehat{R}^\mathsf{T}\widehat{R} = \mathrm{diag}(2a^2, 2a^2, 1)$. Find the result of the forward map followed by the transpose-based reverse map for $B^R = [2\ 0\ 1]^\mathsf{T}$, and compare with the $a = 0.7$ result.

7. **Components versus degrees of freedom.** Explain why position plus a unit quaternion stores seven numbers but describes six continuous degrees of freedom. Normalize $q = [2\ 0\ 0\ 2]^\mathsf{T}$. Use Appendix A to find its rotation matrix, then build a homogeneous transformation with position $[1\ 0\ 2]^\mathsf{T}$.

# Chapter 6

# Robot systems and sensing

Reference frames and transformations describe where a robot and the objects around it are located. A robot must also obtain information about those objects, decide what to do, and produce physical motion. These activities connect the geometric tools of the preceding chapters to the operation of a robot system.

## 6.1 Sensing, processing, and action

A useful working description of a robot is an intelligent interconnection between sensing and action. Sensors collect measurements; computation uses those measurements to select actions; actuators carry out the actions. The resulting interaction with the environment produces new measurements, closing a feedback loop.

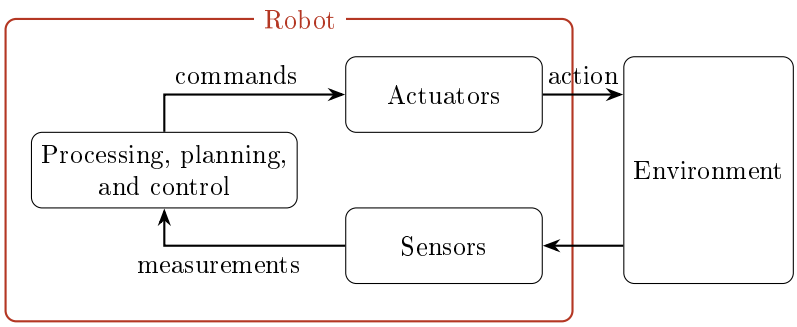



Figure 6.1: A functional view of a robot interacting with its environment. The robot boundary contains sensors, actuators, and the hardware and software that process measurements and generate commands.

## 6.2 Sensors and the environment

The *environment* includes the objects, surfaces, and people with which the robot interacts. A camera measures incoming light, a microphone responds to pressure variations, and a contact sensor detects

physical contact. Sensors may also measure the robot's own state; sensing is not limited to observing external objects.

A measurement is not yet an interpretation of the scene. An image records light arriving at the camera; identifying an object and estimating its location require processing that image. Likewise, a contact measurement indicates an interaction at a surface but does not by itself describe the entire surrounding space. Each type of sensor provides particular information that the robot must interpret in relation to its task.

The coordinate transformations developed earlier enter at this point. An object located in a sensor frame may need to be expressed in a robot frame to determine whether it is within reach, or in a world frame to plan motion. The physical object is the same; the coordinate description is chosen to support the calculation being performed.

## 6.3 Actuators and physical interaction

*Actuators* produce physical action, for example by driving wheels or moving a manipulator. Wheel-driven motion depends on forces exchanged with the supporting surface. An action may change the environment, the robot's pose relative to it, or both.

The distinction between sensing and action is a useful functional one: sensors provide measurements, whereas actuators produce physical effects. A motor command can turn a wheel or move a joint, but the resulting motion also depends on the mechanism and its interaction with the surroundings. Moving a mobile robot changes what its sensors observe, even when the objects in the environment remain fixed in the world frame.

## 6.4 Connecting measurements to commands

The connection between sensing and action includes both computing hardware and software. It may involve interpreting measurements, estimating where objects are, selecting a plan, and producing individual actuator commands. Here, "intelligent" does not require a neural network or machine learning: explicit algorithms and feedback control can also provide this connection.

The central block in Figure 6.1 therefore represents several possible levels of computation. A high-level decision might be to approach an object. More detailed decisions specify how to move, and the commands sent to the actuators must ultimately specify actions that the hardware can execute. All of these functions belong to the connection between measurements and physical action, even when they run in different programs or on different processors.

The enclosing boundary in the diagram is also important: the robot includes the sensors and actuators as well as the computing hardware and software. The central block alone is not the entire robot. The arrows trace a loop through the environment: an action changes the physical situation, sensors measure the resulting situation, and computation can use those measurements to decide the next action.

### 6.4.1 Example: reaching an object

Suppose a robot must grasp an object that is initially beyond its reach. Sensor measurements first support an estimate of the object's location. The robot must then determine that it needs to move closer, choose an approach, and command its motion before extending the manipulator and closing the gripper. A high-level objective thus becomes a sequence of physical actions, with measurements available to check progress and adjust the commands.

Consider the distinction between the decisions involved. Recognizing the object answers what the robot should interact with. Estimating its position answers where it is. Comparing that position with the robot's reach determines whether the base must move first. Planning selects a sequence of motions, and control turns those motions into actuator commands. These decisions serve the same task but require different information and computations.

After the robot approaches, the object's coordinates relative to the robot have changed. New measurements can update its estimated location before the grasp. This illustrates why sensing and action are connected repeatedly: the robot does not merely observe once and then execute an unrelated motion.

## 6.5 Proprioceptive and exteroceptive sensing

Sensors can be classified by the quantities they measure. *Proprioceptive* sensors provide information about the robot itself, such as wheel rotation or joint position. A vehicle's speed indication is an example of information about its own motion. *Exteroceptive* sensors provide information about the environment or the robot's relationship to it, such as the distance to an obstacle. A parking-distance sensor illustrates this second category. The distinction concerns the measured quantity, not whether the sensor is physically mounted inside or outside the robot.

The following sections develop general measurement concepts and then focus on exteroceptive distance sensing. These measurements help a mobile robot detect obstacles and express their locations in the reference frames used for planning and control.

## 6.6 From a physical quantity to a measurement

A sensor system converts a physical quantity into information available to software. For a force measurement, a typical chain is

$$F \longrightarrow \text{electrical signal} \longrightarrow \text{ADC code} \longrightarrow \bar{F}.$$

Here $F$ is the actual force and $\bar{F}$ is the reported force in physical units. The analog-to-digital converter (ADC) produces a number representing an input voltage; calibration relates that number to the force.

Different transducers use different physical effects. A conventional resistive strain gauge changes resistance when deformed; a capacitive transducer instead uses a change in capacitance. Signal conditioning turns the relevant electrical change into a suitable input for measurement. Both mechanisms can support force measurement, but use different electrical effects.[1]

[1] See NI, Measuring Strain with Strain Gages.

### 6.6.1 Quantization and conversion to physical units

An 8-bit ADC has $2^8 = 256$ possible codes, numbered 0 through 255. For an ideal uniform converter spanning 0 to 5 V, the nominal voltage step is

$$\Delta V = \frac{5}{256}\,\mathrm{V} \approx 19.5\,\mathrm{mV}.$$

With the illustrative rule $k = \lfloor V/\Delta V \rfloor$ for $0 \le V < 5\,\mathrm{V}$, an input of 2.31 V gives code 118. The exact transition convention depends on the converter, but the principle is unchanged: a finite interval of input voltages maps to one code. The code is not itself a force in newtons. Software must apply the calibrated relationship between the transducer output and the physical quantity.

Quantization, electrical disturbances, and variation in the physical measurement process can all contribute to the difference between $F$ and $\bar{F}$. Increasing the number of digital bits alone does not eliminate all of these errors.

## 6.7 Gain, bias, noise, and calibration

Within a useful operating interval, an affine measurement model is

$$\bar{F}_k = aF_k + b + \nu_k, \tag{6.1}$$

where $a$ is the gain or scale factor, $b$ is a constant offset (bias), and $\nu_k$ is the noise in sample $k$. Here $F$ and $\bar{F}$ are both expressed in newtons, so $a$ is dimensionless and $b, \nu_k$ have units of newtons. The ideal response has $a = 1$, $b = 0$, and $\nu_k = 0$.

Gain changes the slope of the input–output relationship, while offset shifts its intercept. A sensor with $a = 1.1$ and $b = 0$ reads zero correctly but reports 11 N for an actual force of 10 N. Subtracting a zero-load reading cannot correct that gain error. Noise adds sample-to-sample variation even at a fixed input.

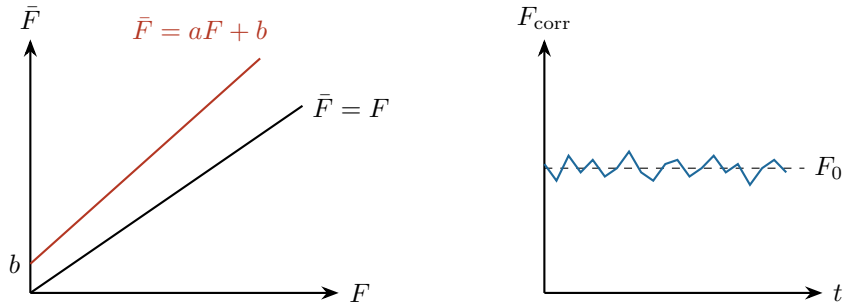



Figure 6.2: Left: gain changes the slope and offset changes the intercept of the mean response. Axis scales differ, so the ideal unit-gain line need not appear at 45°. Right: even with exact gain and offset correction, samples fluctuate around the reference force. Both plots are schematic.

### 6.7.1 Estimating gain and offset

Calibration compares sensor readings with known reference inputs. Apply at least two distinct forces $F_A$ and $F_B$, and collect repeated readings at each. Let $m_A$ and $m_B$ denote the respective sample means. Assuming constant gain and offset and zero-mean noise, the two-point estimates are

$$\widehat{a} = \frac{m_B - m_A}{F_B - F_A}, \qquad \widehat{b} = m_A - \widehat{a}F_A, \qquad F_{\text{corr}} = \frac{\bar{F} - \widehat{b}}{\widehat{a}}. \tag{6.2}$$

The correction requires a nonzero estimated gain. Repeated samples at only one reference force estimate $aF_A + b$; they cannot identify two unknown parameters separately. With more reference levels, fitting a line by least squares uses all the calibration data and helps reveal deviations from an affine response.

For example, suppose mean readings at $F_A = 0\,\text{N}$ and $F_B = 10\,\text{N}$ are $m_A = 0.2\,\text{N}$ and $m_B = 11.2\,\text{N}$. Then

$$\widehat{a} = \frac{11.2 - 0.2}{10 - 0} = 1.1, \qquad \widehat{b} = 0.2\,\text{N}.$$

A later reading of $5.7\,\text{N}$ is corrected to

$$F_{\text{corr}} = \frac{5.7 - 0.2}{1.1}\,\text{N} = 5\,\text{N}.$$

Subtracting the offset alone would give $5.5\,\text{N}$ and leave the scale error uncorrected.

If the gain is already known to be one, calibration reduces to estimating an offset using a known reference $F_0$:

$$\widehat{b} = \frac{1}{M}\sum_{k=1}^{M} \bar{F}_k - F_0.$$

This is a useful special case, rather than the general calibration procedure.

For independent noise samples with equal variance $\sigma^2$, the variance of a sample mean is $\sigma^2/M$. Averaging therefore reduces uncertainty in the reference readings under those assumptions. It does not remove future noise: even with exact calibration parameters, $F_{\text{corr}} = F + \nu/a$. Estimated parameters add their own uncertainty; correlation or drift can also make averaging less effective.

### 6.7.2 Limits of a simple calibration model

The affine model assumes that gain and offset remain constant over the calibrated operating range and conditions. A nonlinear response may require fitting a curve instead. A sufficiently smooth response can often be approximated locally by a line, but that does not justify extrapolating that line indefinitely. Temperature or time-dependent changes may require additional compensation or recalibration.

Sensors have physical operating limits. A compression-only sensing assembly need not measure tensile loading, while other force sensors can measure both directions. At large inputs, a device may saturate or be damaged. Calibration must therefore specify its applicable range and conditions; it cannot make an unsupported input measurable.

## 6.8 Distance sensing with ultrasound

A mobile robot needs information about obstacles early enough to respond. An ultrasonic range sensor emits a sound pulse and detects a returning echo. If $v_s$ is the sound speed in the medium and $\Delta t$ is the round-trip travel time, then

$$\boxed{d = \frac{v_s \Delta t}{2}.} \tag{6.3}$$

The factor two accounts for travel to the obstacle and back. This approximation treats the emitter and receiver as colocated relative to the target distance and assumes the detected echo follows the intended path.

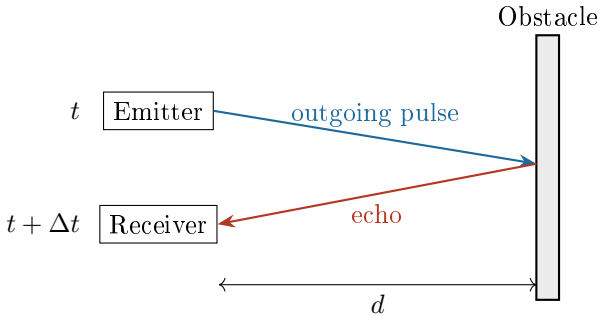



Figure 6.3: Ultrasonic ranging measures the outgoing and return travel time. Emitter–receiver separation is exaggerated for clarity.

For example, using $v_s = 343\,\mathrm{m/s}$ and a measured delay of $10\,\mathrm{ms}$ gives $d = 1.715\,\mathrm{m}$. The sound speed must match the medium and conditions; the same assumed value cannot be used for both air and water.

### 6.8.1 Weak echoes, multipath, and interference

The acoustic wave spreads as it propagates, and only part of the reflected energy returns to the receiver. Absorption and target geometry further affect the echo. A weak or missing return can therefore limit useful range; it does not necessarily establish that no obstacle exists.

An echo may also follow a longer path involving several reflecting surfaces. This *multipath* return corresponds to total path length, not necessarily twice the direct distance to the intended obstacle. With several ultrasonic sensors nearby, one receiver may detect another transmitter's pulse, creating *crosstalk*.

One mitigation is to trigger sensors in sequence and allow earlier echoes to decay. The waiting interval depends on the sensor and environment; there is no universal delay of several seconds. For a maximum intended range $d_{\max}$, $2d_{\max}/v_s$ is the direct round-trip travel time, to which practical timing allowances may need to be added. Sequential operation also means that a set of readings is collected over time rather than instantaneously.

## 6.9 Infrared reflection and laser ranging

An infrared emitter and receiver can detect reflected light from an object. In a simple intensity-based sensor, received signal strength is related to distance through calibration. The difficulty is that signal strength also depends on the target's reflectance, orientation, and other conditions. A less reflective target may therefore be interpreted as farther away if the calibration assumes a more reflective target.

Visible color alone does not determine reflectance at an infrared wavelength. The relevant property is the material's response to the emitted light. At approximately fixed height, an array of downward-facing reflectance sensors can use differences between a line and its background for line following. In that application, contrast is the desired information rather than an unwanted influence on a distance estimate.

Infrared describes a wavelength band, not a unique ranging method. Optical time-of-flight ranging is also possible with suitable electronics; it is not ruled out by the high speed of light.[2] Laser rangefinders can use pulse travel time, phase-based techniques, or other ranging principles. A laser beam is narrow but still has finite divergence, and a useful return depends on the target and receiver.

### 6.9.1 Why scan the beam?

A single narrow beam measures along one direction. An obstacle outside that direction can remain undetected even when the reported distance is correct. A planar laser scanner samples multiple directions, often by rotating a beam or optical assembly. It associates each measured range with a bearing, producing a scan of the surrounding geometry in a plane.

Coverage is limited by the field of view, angular sampling, minimum and maximum usable range, and whether the obstacle intersects the scan plane. Scanning also takes time. A robot or object that moves during acquisition may change position between the first and last readings.

## 6.10 Representing a planar laser scan

Let $S$ be the sensor frame, with bearings measured counterclockwise from $x_S$ toward $y_S$. A scan contains ranges

$$\boldsymbol{r} = \begin{bmatrix} r_1 & r_2 & \cdots & r_N \end{bmatrix}^{\mathsf{T}},$$

together with the information needed to determine each ray's angle and interpret its validity. Suppose $N \geq 2$ uniformly spaced rays include both endpoints $\gamma_{\min}$ and $\gamma_{\max}$. Define the field of view $\alpha$ and angular spacing $\beta$ by

$$\alpha = \gamma_{\max} - \gamma_{\min}, \qquad \boxed{\beta = \frac{\alpha}{N-1}}, \qquad \boxed{\gamma_i = \gamma_{\min} + (i-1)\beta}. \tag{6.4}$$

There are $N-1$ intervals between $N$ endpoint-inclusive samples. In particular, $\gamma_1 = \gamma_{\min}$ and $\gamma_N = \gamma_{\max}$. A full-circle scan that omits a duplicate endpoint uses a different counting convention; always follow the device's actual angular metadata.

[2] For an example of infrared time-of-flight technology, see STMicroelectronics, Proximity Sensing Applications.

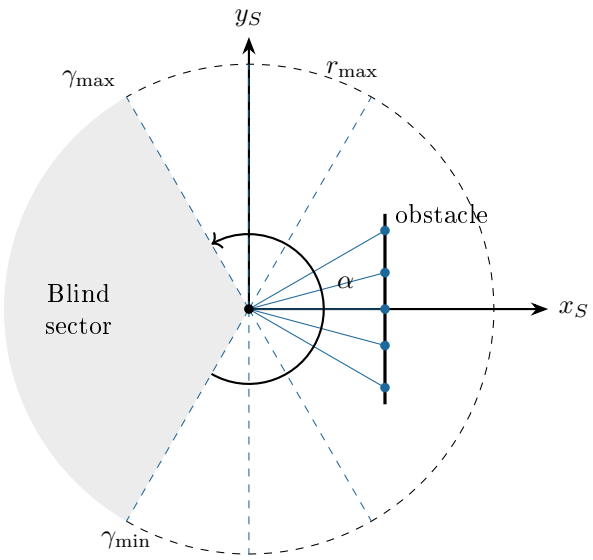



Figure 6.4: A schematic 240° field of view. Solid rays end at detected obstacle points; dashed rays illustrate directions without a depicted return within range. The shaded rear sector is not observed. Only selected rays are drawn.

### 6.10.1 Worked example: 241 rays

For $\gamma_{\min} = -2\pi/3$, $\gamma_{\max} = 2\pi/3$, and $N = 241$,

$$\alpha = \frac{4\pi}{3} = 240^\circ, \qquad \beta = \frac{4\pi/3}{240} = \frac{\pi}{180} = 1^\circ.$$

Thus $\gamma_i = -120^\circ + (i-1)1^\circ$. Ray 1 points at $-120^\circ$, ray 121 at $0^\circ$, and ray 241 at $120^\circ$. The indices here start at one; an array indexed from zero instead uses $\gamma_j = \gamma_{\min} + j\beta$ for $j = 0, \ldots, N-1$.

Angular spacing is distinct from range resolution or measurement accuracy. At range $r$, two directions separated by $\beta$ intersect a circle at points separated by chord length $2r\sin(\beta/2)$. For small angles this is approximately $r\beta$, with $\beta$ in radians. Consequently, the gaps between sampled directions become wider with distance, and a thin obstacle can fall between adjacent rays.

### 6.10.2 Valid returns and missing measurements

A range reading must be interpreted using the device's valid range and status conventions before being converted into an obstacle point. Invalid or missing returns may use special codes, infinity, or other representations. Do not assume that zero or the maximum-range value always has one universal meaning. In particular, a missing return is not an obstacle located at the maximum range, nor proof that the whole direction is unobstructed.

As a concrete software convention, the ROS `LaserScan` message carries angle and range metadata and specifies that ranges below its minimum or above its maximum should be discarded.[3] The sensor and driver documentation determine the remaining status details.

[3] See the official ROS LaserScan message definition.

## 6.11 From ranges to obstacle coordinates

For a valid return at range $r_i$ and bearing $\gamma_i$, the corresponding point in the sensor frame is

$$\boxed{B_i^S = \begin{bmatrix} x_i^S \\ y_i^S \end{bmatrix} = \begin{bmatrix} r_i \cos\gamma_i \\ r_i \sin\gamma_i \end{bmatrix}.} \tag{6.5}$$

This is the polar-to-Cartesian conversion. The inverse is

$$r_i = \sqrt{(x_i^S)^2 + (y_i^S)^2}, \qquad \gamma_i = \operatorname{atan2}(y_i^S, x_i^S). \tag{6.6}$$

At the origin, the range is zero and the bearing is geometrically undefined.

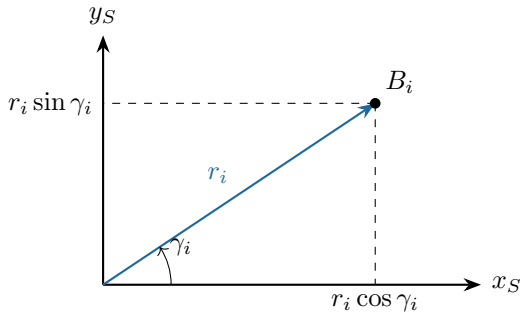



Figure 6.5: A valid range and bearing determine Cartesian coordinates in the sensor frame. The projections give the cosine and sine components.

### 6.11.1 The sensor frame is not necessarily the robot frame

A sensor rigidly attached to a robot has a fixed mounting transform, but its axes and origin need not coincide with the robot's. In the planar case,

$$B_i^R = R_S^R B_i^S + p_S^R, \qquad \widetilde{B}_i^W = H_R^W H_S^R \widetilde{B}_i^S. \tag{6.7}$$

For a spatial mounting, embed the scan point as $[r_i\cos\gamma_i \;\; r_i\sin\gamma_i \;\; 0]^\mathsf{T}$ in $S$ and apply the $4\times 4$ transformations of Chapter 5. A horizontal scan in the sensor frame need not be horizontal in the world frame.

For example, ray 151 in the 241-ray scan has bearing $30°$. If $r_{151} = 2\,\text{m}$, then

$$B_{151}^S = \begin{bmatrix} \sqrt{3} \\ 1 \end{bmatrix} \text{m}.$$

Suppose the sensor is rotated by $90°$ relative to the robot and its origin is at $p_S^R = [0.2 \; 0]^\mathsf{T}\,\text{m}$. Then

$$B_{151}^R = \begin{bmatrix} 0 & -1 \\ 1 & 0 \end{bmatrix} \begin{bmatrix} \sqrt{3} \\ 1 \end{bmatrix} + \begin{bmatrix} .2 \\ 0 \end{bmatrix} = \begin{bmatrix} -.8 \\ \sqrt{3} \end{bmatrix} \text{m}.$$

Treating the original sensor coordinates as robot coordinates would give the wrong location even though the range measurement itself was correct. Finally, all these conversions act on measured data: calibration error and noise remain present in the resulting obstacle coordinates.

## 6.12 Additional exercises

State assumptions and units. Use the endpoint-inclusive scan convention unless otherwise specified.

1. **Classifying measurements.** Classify a wheel-rotation measurement, a joint-angle measurement, and a distance to a wall as proprioceptive or exteroceptive. Explain why the sensor's mounting location does not decide the classification.

2. **ADC conversion.** Use the ideal 8-bit, 0–5 V model in this chapter. Find the code for 2.50 V and the voltage interval represented by code 118. Why is neither number automatically a force?

3. **Gain and offset calibration.** Mean readings at reference forces of 0 and 10 N are 0.3 and 9.3 N. Estimate gain and offset, then correct a later reading of 11.1 N. Explain why one reference force is insufficient when both parameters are unknown, and why corrected samples still vary.

4. **Ultrasonic ranging.** With $v_s = 343\,\mathrm{m/s}$, find the range corresponding to a 12 ms echo delay. Find the direct round-trip time for a 4 m maximum range. Explain how multipath and a neighboring transmitter can each invalidate a naive distance estimate.

5. **Scan indexing.** A scanner samples $[-120°, 120°]$ with 241 rays. Find the bearings of rays 1, 61, 121, and 241. Which ray points at 45°? Repeat the last answer using zero-based indexing.

6. **Polar coordinates and mounting.** Ray 61 of the preceding scan returns 2 m. Find its sensor-frame coordinates. Transform them to the robot frame for mounting rotation 90° and translation $[0.2\ 0]^{\mathsf{T}}$ m. Recover the original range and bearing from the sensor-frame Cartesian coordinates.

7. **What a scan cannot establish.** At range 5 m, compute the chord spacing between rays separated by 1°. Explain why a narrow obstacle might be missed. Distinguish a blind sector, an invalid return, and a valid detection, and explain why plotting every missing return at $r_{\max}$ is misleading.

# Chapter 7

# Cameras and perspective projection

A range sensor reports a distance, whereas an ordinary camera produces an image. The image contains information about the appearance and direction of objects, but does not directly assign a three-dimensional position to each pixel. To use a camera in robotics, we need a model connecting spatial points to their image locations and an understanding of what information that mapping discards.

This chapter first describes images as arrays of measurements, then develops the pinhole camera model. The forward problem predicts where a known point appears in an image. The reverse problem reveals why an image location alone does not determine depth.

## 7.1 Images as arrays of measurements

Light from a scene reaches a photosensitive surface. A digital image sensor contains an array of small sensing sites that convert incident light into electrical signals. Readout electronics and analog-to-digital conversion produce numerical values. Exposure, sensor response, and processing influence those values; they are not direct measurements of object distance.

### 7.1.1 Grayscale images

A grayscale image can be represented as an $H \times W$ matrix, where $H$ is the number of rows and $W$ the number of columns. An 8-bit representation stores values from 0 to 255. With a conventional grayscale display, 0 is black, 255 is white, and intermediate values are shades of gray. Figure 7.1 illustrates this correspondence using a small image. A saturated value of 255 cannot distinguish stronger inputs beyond the represented range.

Image dimensions are often written as width by height. Thus a $640 \times 480$ image has 480 rows, 640 columns, and 307,200 pixels. This convention reverses the order used to state matrix dimensions. Keeping row, column, width, and height distinct prevents indexing errors.

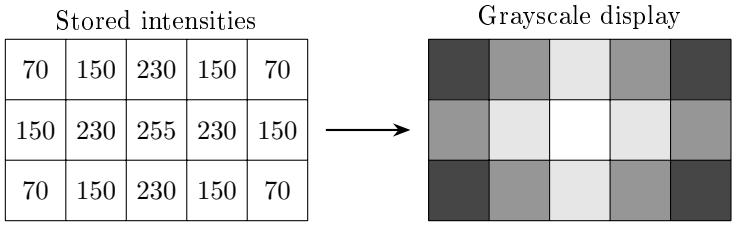



Figure 7.1: A $3 \times 5$ array and its grayscale visualization. Each displayed square corresponds to one matrix entry; larger values appear brighter.

### 7.1.2 Color images and RGB values

An RGB image assigns three values to each output pixel: red, green, and blue. For 8-bit channels, some examples are

| RGB values | Displayed color |
|---|---|
| $(255, 0, 0)$ | Red |
| $(0, 255, 0)$ | Green |
| $(0, 0, 255)$ | Blue |
| $(255, 255, 0)$ | Yellow |
| $(0, 0, 0)$ | Black |
| $(255, 255, 255)$ | White |

These are additive light mixtures. Mixing emitted red and green light is different from mixing red and green pigments. A pixel such as $(126, 8, 180)$ has strong blue and red contributions and appears purple. Channel order is a convention: an application using BGR requires a different interpretation of the same three stored numbers.

### 7.1.3 Color filters and reconstructed pixels

One common sensor design places a color-filter mosaic over the sensing sites. A Bayer pattern repeats a $2 \times 2$ arrangement containing one red, two green, and one blue filter. Figure 7.2 shows one possible arrangement. Each site measures only its filtered channel; it does not independently measure all three RGB components.

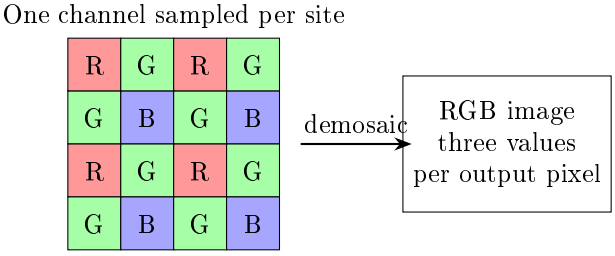



Figure 7.2: An RGGB Bayer mosaic. Each letter denotes the channel sampled at one sensing site. Demosaicing estimates missing channels to form an RGB image.

The missing color components are estimated using neighboring samples in a process called *demosaicing*. A repeated four-site pattern should therefore not be confused with four independent full-color measurements or with a universal rule that four sites produce exactly one output pixel. The raw mosaic and the reconstructed RGB image are different data representations.[1]

## 7.2 The pinhole camera model

An ideal pinhole camera consists of a light-tight enclosure, a small aperture, and an image plane behind it. In geometric optics, the ray from a scene point that passes through the aperture intersects the image plane at the point's image. Rays cross at the aperture, so the physical image is inverted.

The model treats the aperture as a single *projection center*. Actual cameras use finite apertures and lenses to collect and focus light. The pinhole model describes their central projection approximately; it does not model blur, diffraction, or lens distortion. Making a physical pinhole ever smaller is not an unlimited route to a sharper image, because light throughput and diffraction also matter.

Let $C$ be the camera frame with origin $O_C$ at the projection center. Use the optical-axis convention

$$z_C: \text{ forward into the scene}, \qquad x_C: \text{ image-right}, \qquad y_C: \text{ image-down}.$$

These directions form a right-handed frame. In a two-dimensional section, the page may be rotated for readability without changing the frame definition. Write a scene point as

$$B^C = \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}, \qquad Z > 0.$$

The variables $X, Y, Z$ are the point's coordinates in $C$, measured in a common length unit. The quantity $Z$ is depth along the optical axis, not generally the Euclidean distance $\|B^C\|$.

### 7.2.1 Physical and virtual image planes

Place the physical image plane at $z_C = -f$, with $f > 0$. The line through $O_C$ and $B^C$ has coordinates $\lambda[X\ Y\ Z]^\mathsf{T}$. Its intersection with the physical plane has $\lambda = -f/Z$, giving

$$x_{\text{physical}} = -f\frac{X}{Z}, \qquad y_{\text{physical}} = -f\frac{Y}{Z}.$$

The negative signs express the inversion when image axes are parallel to the corresponding camera axes.

For calculations, introduce a *virtual image plane* at $z_C = f$, in front of the projection center. Its coordinates are

$$\boxed{x' = f\frac{X}{Z}, \qquad y' = f\frac{Y}{Z}.} \tag{7.1}$$

This is a modeling convention, not a physical sensor placed in front of the camera. It gives an upright image and removes the two negative signs.

[1] See Basler, Demosaicing Mode.

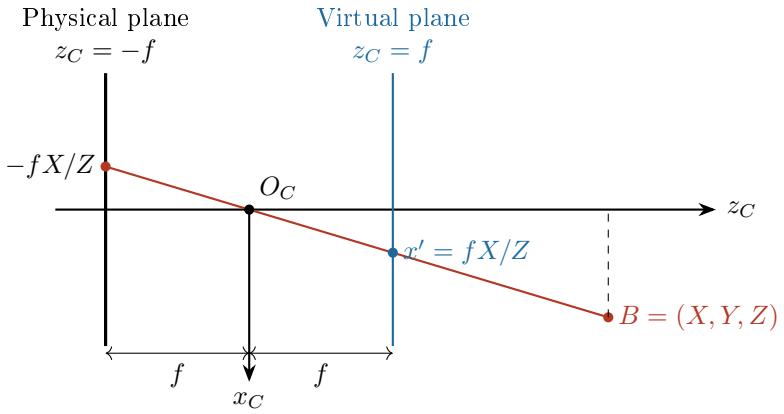



Figure 7.3: A section through the $x_C z_C$ plane, drawn with positive $x_C$ down the page. The physical and virtual image coordinates have opposite signs. The $y_C$ coordinate is omitted from this section.

The same relationship follows from similar triangles. In this section, $X = \ell \sin\theta$ and $Z = \ell \cos\theta$, so $X/Z = \tan\theta$. For the virtual image triangle, $x'/f = \tan\theta$. Equating the ratios gives $x' = fX/Z$. Repeating the construction in the $y_C z_C$ plane gives the second equation.

The parameter $f$ is the projection distance in this ideal model. For a lens camera, an effective calibrated model is used; $f$ should not simply be identified with the distance from the front of the camera housing to the sensor. Changing focus or zoom can change the appropriate calibration.

## 7.3 From metric image coordinates to pixels

The coordinates $x', y'$ are lengths measured from the optical-axis intersection with the virtual plane. Image processing instead uses pixel coordinates measured relative to an image origin. Two operations are needed: convert lengths to pixel units and shift the origin.

Let $\alpha_x, \alpha_y$ be the pixel densities along the two image axes, with units pixels per meter when $f$ is in meters. Define

$$f_x = \alpha_x f, \qquad f_y = \alpha_y f,$$

which are focal lengths expressed in pixel units. Square sampling gives equal densities; the general model permits different scales. Let $(c_x, c_y)$ be the *principal point*, the image coordinates of the optical-axis intersection. The projection equations are

$$\boxed{u = f_x \frac{X}{Z} + c_x, \qquad v = f_y \frac{Y}{Z} + c_y.} \tag{7.2}$$

Here $u$ increases to the right and $v$ downward. They correspond to the horizontal and vertical image coordinates, also denoted $x_B^I$ and $y_B^I$. The formulas assume perpendicular image axes and no distortion.

### 7.3.1 Image center and indexing conventions

For an ideal centered camera, a convenient continuous coordinate convention places the image boundaries at $u = 0, W$ and $v = 0, H$. Then

$$c_x = \frac{W}{2}, \qquad c_y = \frac{H}{2}.$$

Under this convention the pixel centers are at half-integer coordinates. If instead zero-based integer coordinates label pixel centers, the geometric center of the array is $((W-1)/2, (H-1)/2)$. These conventions differ by half a pixel; either can be used consistently. In a calibrated camera, the principal point need not equal the exact geometric center.

A projection generally gives continuous coordinates, not an integer array index. Selecting a pixel or interpolating an image requires a sampling rule. In row–column array notation, the vertical coordinate selects the row and the horizontal coordinate the column. A pixel location is therefore addressed in the order $[\text{row}, \text{column}]$, corresponding to $[v, u]$ after the chosen conversion to indices.

### 7.3.2 What calibration provides

Camera calibration estimates the model parameters from observations of known geometry, such as a checkerboard photographed in several poses. In practice, calibration commonly estimates $f_x, f_y, c_x, c_y$ directly and may also estimate lens-distortion parameters. The separate physical quantities $f$ and $\alpha_x, \alpha_y$ are not needed to use Equation (7.2).[2]

The pinhole equations apply to an ideal or appropriately undistorted image. Using them on raw images with substantial unmodeled distortion introduces systematic position errors. A detailed calibration procedure is outside the scope of this chapter.

## 7.4 Worked projection example

Suppose a camera has

$$W = 640, \quad H = 480, \quad \alpha_x = \alpha_y = 5 \times 10^4 \,\text{px/m}, \quad f = 3 \times 10^{-3} \,\text{m}.$$

Use the centered continuous-image convention, so $(c_x, c_y) = (320, 240)$. The focal lengths in pixels are

$$f_x = f_y = (5 \times 10^4)(3 \times 10^{-3}) = 150 \,\text{px}.$$

For $B^C = [2\ 0\ 10]^\mathsf{T}\,\text{m}$,

$$u = (150\,\text{px})\frac{2\,\text{m}}{10\,\text{m}} + 320\,\text{px} = 350\,\text{px}, \tag{7.3}$$

$$v = (150\,\text{px})\frac{0\,\text{m}}{10\,\text{m}} + 240\,\text{px} = 240\,\text{px}. \tag{7.4}$$

The ratios are dimensionless, leaving pixel units in both equations. The point appears 30 pixels to the right of the principal point and at the same vertical coordinate, as shown in Figure 7.4.

[2] See OpenCV, Camera Calibration.

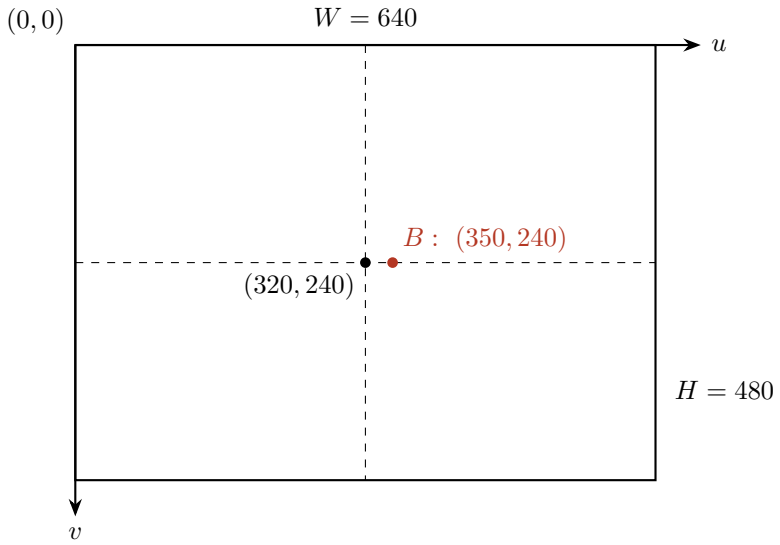



Figure 7.4: The projected location $(350, 240)$ in the $640 \times 480$ example. Dashed lines pass through the principal point. Coordinates refer to the continuous image plane, before selecting discrete pixel indices.

### 7.4.1 When does a projected point appear in an image?

The model assumes $Z > 0$. At $Z = 0$ the division is undefined, while a point with $Z < 0$ is behind a forward-looking camera even if its algebraic projection lies within the image bounds. A point in front must also project inside the image area. In the continuous boundary convention, this means $0 \leq u < W$ and $0 \leq v < H$ for sampling within the image.

These checks establish geometric eligibility, not guaranteed visibility: another surface can occlude the point. Detecting the desired object in an image is also a separate task from computing where a known point projects.

## 7.5 Why one image point does not determine depth

Rearranging the projection equations gives

$$\frac{X}{Z} = \frac{u - c_x}{f_x}, \qquad \frac{Y}{Z} = \frac{v - c_y}{f_y}. \tag{7.5}$$

The right sides are known for a calibrated camera and measured image point, but the left sides are ratios. Two equations constrain three spatial coordinates. Writing the result as a family of points makes the remaining ambiguity explicit:

$$\boxed{B^C = Z \begin{bmatrix} (u - c_x)/f_x \\ (v - c_y)/f_y \\ 1 \end{bmatrix}, \qquad Z > 0.} \tag{7.6}$$

The image determines a ray from the projection center. It does not specify where along that ray the object lies. The vector in brackets is not generally a unit vector: its scale factor here is axial depth $Z$, not range.

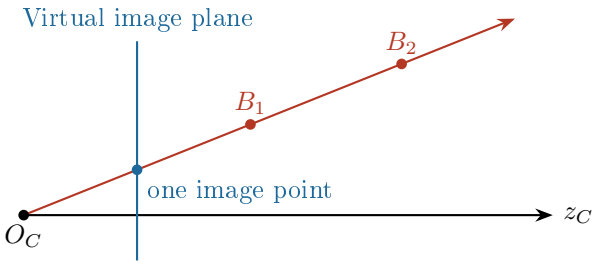



Figure 7.5: Distinct points on one viewing ray have the same image coordinates. Projection removes their common spatial scale.

In the worked example, $(u, v) = (350, 240)$ gives $X/Z = 0.2$ and $Y/Z = 0$. The points $[1\ 0\ 5]^\mathsf{T}$, $[2\ 0\ 10]^\mathsf{T}$, and $[4\ 0\ 20]^\mathsf{T}$ all produce that same location. More generally, multiplying $X, Y, Z$ by any positive constant leaves their ratios unchanged. This geometric argument explains the ambiguity beyond simply counting equations and unknowns.

## 7.6 Additional information for recovering position

Recovering a three-dimensional point requires information beyond one unconstrained image location. Several sources are possible.

### 7.6.1 Two cameras

A stereo pair observes the same scene from distinct positions. If a point is matched between the images and the cameras' calibration and relative pose are known, the two viewing rays can be used to triangulate its position. The additional image coordinates are useful because they come from another viewpoint, not merely because more equations have been written down. Degenerate geometry and measurement noise can prevent a unique or accurate intersection. Stereo also requires identifying the same scene point in both images.

### 7.6.2 Known object geometry

Known size can supply scale information. For example, a segment of known length $L$ parallel to the image plane at constant depth $Z$ spans

$$s = f_x \frac{L}{Z}$$

pixels when aligned with the horizontal axis. Hence $Z = f_x L/s$. This simple relation assumes the stated alignment and a known correspondence between the measured image endpoints and the object's endpoints. Unknown orientation or occlusion makes an apparent-size argument more complicated.

### 7.6.3 A known surface or depth

If $Z$ is known independently, Equation (7.6) determines the point directly. A downward-looking camera above a known level ground plane provides one useful special case. When its optical axis is perpendicular to the ground, points on that plane have axial depth equal to the camera's height above it. If the camera is tilted, height is not generally equal to each point's $Z$; the viewing ray must instead be intersected with the ground plane using the camera pose. A point above the ground does not satisfy the ground-plane assumption.

These methods add hardware, known scene geometry, or constraints on the task. They do not remove the need to state which extra information makes depth recovery possible.

## 7.7 Connecting camera geometry to robot frames

The camera optical frame usually differs from a robot frame whose forward axis is $x_R$. The mounting transform includes both the position of the camera origin and the orientation of its axes. Once a point's camera-frame position has been obtained, its world coordinates are

$$B^W = R_C^W B^C + p_C^W, \qquad H_C^W = H_R^W H_C^R. \tag{7.7}$$

The mounting calibration $H_C^R$ is distinct from the internal projection parameters $f_x, f_y, c_x, c_y$.

Conversely, to predict the image of a known world point, first compute

$$B^C = (R_C^W)^\mathsf{T}(B^W - p_C^W),$$

then apply Equation (7.2). Coordinate transformation and perspective projection are different operations: the former preserves the spatial point, while the latter divides by depth and discards scale. Transforming a viewing ray into the world frame does not by itself resolve the missing depth.

## 7.8 Additional exercises

Use the virtual image plane, zero distortion, and the stated continuous-image convention. Keep depth and Euclidean range distinct.

1. **Image representation.** How many rows, columns, and pixels are in a $640 \times 480$ image? How many bytes store an uncompressed 8-bit grayscale image and an 8-bit-per-channel RGB image, ignoring padding and metadata?

2. **Color sampling.** Sketch a $4 \times 4$ RGGB Bayer mosaic. How many sites measure each channel? Explain why each raw site does not already contain an RGB triple and what demosaicing supplies.

3. **Projection and visibility.** With $f_x = f_y = 150$ and principal point $(320, 240)$, project $[2 \ -1 \ 5]^\mathsf{T}$ and $[12 \ 0 \ 5]^\mathsf{T}$. Which lies inside a $640 \times 480$ image? Explain why the algebraic projection of $[0 \ 0 \ -5]^\mathsf{T}$ does not establish visibility.

4. **Units and scale.** Let $f = 4\,\text{mm}$ and $\alpha_x = \alpha_y = 10^5\,\text{px/m}$. Find $f_x, f_y$. Using principal point $(320, 240)$, project $[1 \ 0.5 \ 4]^\mathsf{T}\,\text{m}$. What changes if all three point coordinates are doubled?

5. **Back-projection.** For $f_x = f_y = 150$, principal point $(320, 240)$, and image location $(350, 255)$, write the full viewing ray. Find its point at depth $Z = 8\,\text{m}$ and compute that point's range. Explain why the image alone cannot select this depth.

6. **Known size.** A horizontal segment of length $0.4\,\text{m}$ lies parallel to the image plane and spans 20 pixels with $f_x = 150$. Find its depth. Name two assumptions that could fail when this method is applied to an arbitrary object in a photograph.

7. **Camera and world frames.** A camera has $R_C^W = I_3$ and $p_C^W = [1 \ 0 \ 2]^\mathsf{T}\,\text{m}$. For the world point $B^W = [3 \ 0 \ 12]^\mathsf{T}\,\text{m}$, compute $B^C$ and project it using $f_x = f_y = 150$, $(c_x, c_y) = (320, 240)$. Explain what additional information would be needed to reverse this calculation from the image point alone.

# Chapter 8

# Motors, transmissions, and speed control

A robot must turn a numerical command into physical motion. This requires more than a motor: a controller generates a command, a driver supplies power, a transmission adapts the motion, and a sensor measures the result. This chapter follows that chain for a motor-driven wheel, then closes the loop by comparing measured speed with desired speed.

## 8.1 From a command to motion

An electric motor converts electrical energy into mechanical motion. Its shaft rotates at angular velocity $\omega_m$. A transmission connects that shaft to a wheel or another mechanism, whose angular velocity $\omega$ may be quite different. The controller determines how the motor should operate, but the energy that moves the load comes from a power supply.

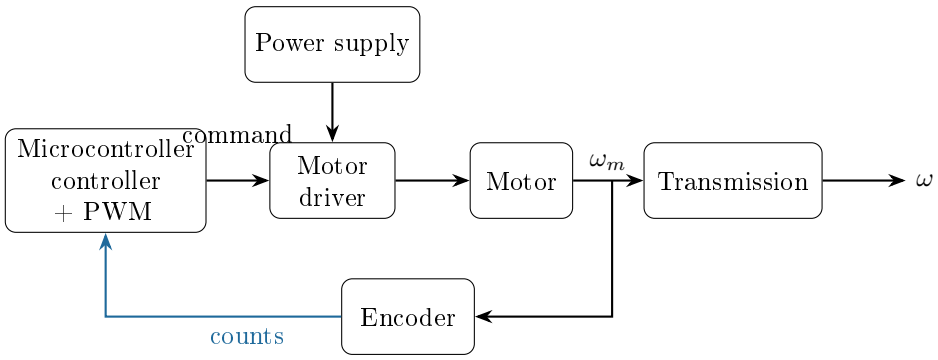



Figure 8.1: The actuation chain. The encoder measures the motor shaft before the transmission. Its signals return to the microcontroller for speed estimation and feedback. The motor power supply feeds the driver directly.

The driver, also called a power amplifier, bridges the gap between a small control signal and the power required by the actuator. A microcontroller's digital output is intended to communicate a logic state; it is not a motor power source. Motor current, particularly during starting or stall, can exceed what a logic pin can supply. The driver switches energy from its supply to the motor according to the controller's command. The controller and driver must also have compatible signal levels and, in an ordinary nonisolated connection, a common electrical reference.

A driver does not create energy or necessarily reproduce the input voltage amplitude. For example, a logic-level command can control switching of a separate motor supply at a different voltage. Driver and supply ratings must match the motor and load; connector size alone does not establish those ratings.

## 8.2 Pulse-width modulation

### 8.2.1 Representing an intermediate command with two levels

A digital output can hold a low or a high voltage. To obtain an adjustable average without generating every intermediate voltage directly, it can switch periodically between those two levels. In *pulse-width modulation* (PWM), the duration of the high part of each period sets the command.

Let $T$ be the period and $t_{\text{on}}$ the time spent high. The *duty cycle* and switching frequency are

$$D = \frac{t_{\text{on}}}{T}, \qquad 0 \le D \le 1, \qquad f_{\text{PWM}} = \frac{1}{T}. \tag{8.1}$$

For levels $V_L$ and $V_H$, the average over one period is

$$\overline{V} = DV_H + (1 - D)V_L. \tag{8.2}$$

With $V_L = 0$ and $V_H = 5\,\text{V}$, a 50% duty cycle has average $2.5\,\text{V}$; a 75% duty cycle has average $3.75\,\text{V}$. The instantaneous voltage still switches between 0 and $5\,\text{V}$.

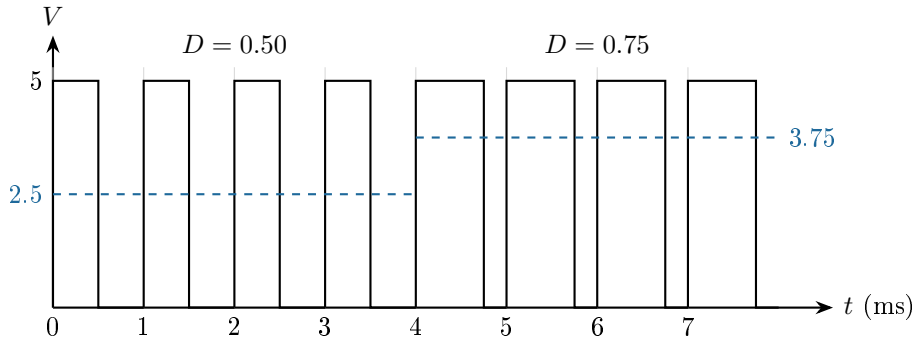



Figure 8.2: PWM with a fixed period and a change in duty cycle. Dashed blue lines show the per-period averages, not additional voltage levels produced by the digital pin. Here $T = 1\,\text{ms}$ and $f_{\text{PWM}} = 1\,\text{kHz}$.

### 8.2.2 Why the shaft does not start and stop at every pulse

The motor and load have inertia, so their speed cannot change instantaneously. Winding inductance also affects how rapidly current changes. With a suitable switching frequency, the mechanical response is much smoother than the electrical waveform. An averaged description is then useful, although current and torque can still have ripple.

This does not mean that angular velocity equals average voltage: they have different units. Speed depends on the motor, supply, load, friction, and driver. Increasing duty cycle generally increases

the drive available in a fixed direction, but a duty cycle alone does not specify an exact speed. Likewise, zero duty does not necessarily stop a moving wheel immediately; the result depends on inertia and the driver's braking or coasting behavior.

### 8.2.3 Digital resolution

An 8-bit command has $2^8 = 256$ possible values, commonly numbered 0 through 255. In a simple endpoint-inclusive command model,

$$D \approx \frac{n}{255}, \qquad n \in \{0, 1, \ldots, 255\}.$$

For example, $n = 128$ requests approximately 50% duty. Exact timing depends on the timer and its configuration. PWM-capable pins, frequency, voltage, and resolution are hardware-dependent; they are not universal properties of all microcontrollers.[1]

## 8.3 Other forms of actuation

Electrical motors are common in mobile robots, but the same separation between command and power applies to fluid-powered actuators. Hydraulic systems use a liquid; pneumatic systems use a gas. A controlled valve routes pressurized fluid to an actuator, rather than an electrical driver supplying motor current.

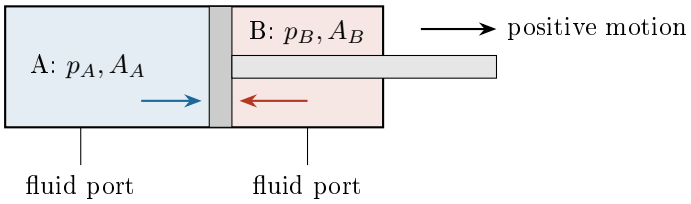



Figure 8.3: A double-acting cylinder, shown schematically. Fluid pressure acts on the two faces of the piston. The rod makes their effective areas unequal.

For the cylinder shown, pressure in chamber A pushes the piston to the right, while pressure in chamber B pushes it to the left. Using pressures relative to the surrounding pressure, the ideal fluid force to the right is

$$F = p_A A_A - p_B A_B.$$

External load and friction also enter the force balance. Motion therefore does not simply continue until the chamber pressures are equal: the effective areas and applied load matter. The remaining sections focus on a rotary electric motor driving a wheel.

## 8.4 Transmissions and reduction ratio

A motor may rotate faster than the wheel needs to rotate. A gear train can reduce this speed and adapt the available torque. Transmissions can also relocate an axis, change its direction, or convert rotation into translation.

[1] For a concrete implementation, see Arduino's PWM documentation.

Define the positive reduction ratio by the speed magnitudes,

$$R = \frac{|\omega_m|}{|\omega|}. \tag{8.3}$$

For a reduction, $R > 1$. Once the positive directions of the two shafts have been chosen to correspond through the transmission, the signed relation is

$$\omega = \frac{\omega_m}{R}. \tag{8.4}$$

This convention absorbs any reversal caused by the gear train into the shaft coordinate definitions. If both axes instead use the same physical viewing direction, a reversing transmission requires a minus sign.

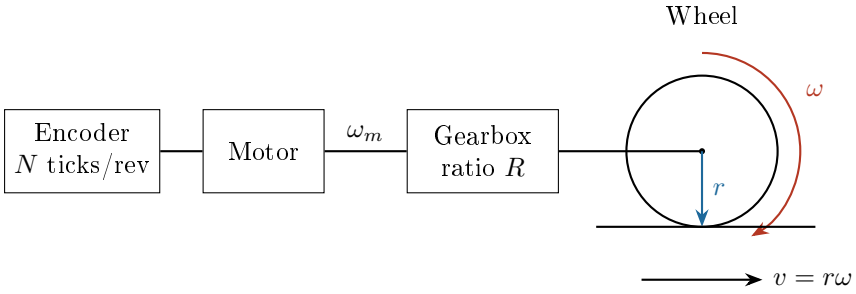



Figure 8.4: Motor-side sensing and wheel-side motion. The gearbox represents the entire transmission, not a particular number or arrangement of gears. The encoder resolution refers to the shaft on which it is mounted.

For $R = 100$, one wheel revolution requires 100 motor revolutions. Reduction does not provide free mechanical power: ideally, the decrease in speed is accompanied by an increase in torque, while real transmissions dissipate some energy. Backlash and compliance can also make wheel motion differ slightly from the motion inferred at the motor shaft.

## 8.5 Encoders and speed measurement

An incremental encoder produces transitions as its shaft turns. A magnetic encoder can use a rotating magnet and Hall-effect sensors; an optical encoder uses a different sensing principle to obtain similar position increments. Counting these increments over a known time interval gives a speed estimate.

One periodic channel provides information about the amount of rotation but does not, by itself, distinguish the two directions. Two suitably phased channels allow direction to be inferred from their transition order. In a quadrature encoder, channels A and B are offset by a quarter of an electrical cycle. Reversing rotation reverses which channel leads. This phase offset need not equal a 90-degree mechanical separation of the sensors.

### 8.5.1 Keeping the counting convention explicit

Let $N$ denote the number of *counted events per motor revolution*, also called ticks per revolution (TPR) here. It must match the decoding method: counting rising edges of one channel is different

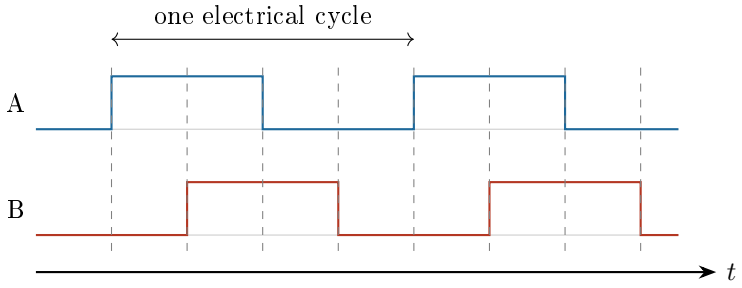



Figure 8.5: Ideal quadrature signals for one direction. Channel A rises before channel B. Reversing the direction reverses the sequence. Counting all edges of both channels gives four counts per electrical cycle.

from counting both edges of both channels. Manufacturer terms such as pulses and counts must be interpreted together with the stated convention.

If $\Delta n$ signed counts are accumulated over an interval $\Delta t$, then

$$\overline{\omega}_m = \frac{2\pi \Delta n}{N \Delta t} \quad \text{in radians per second.} \tag{8.5}$$

The overbar denotes an estimate obtained from the encoder. In particular, the count rate $q = \Delta n / \Delta t$ in ticks per second is converted to angular velocity by multiplying by $2\pi/N$.

An estimate is not the exact instantaneous speed. Finite counts cause quantization, and the counting window averages motion over time. A longer window can improve count resolution while delaying the response to speed changes. Missed transitions and mechanical uncertainty introduce further errors. At low speed, measuring the time between transitions is another possible approach.

## 8.6 From encoder counts to wheel speed

For a wheel of radius $r$ rolling without slip along a stationary surface, its center's velocity component in the rolling direction satisfies

$$v = r\omega. \tag{8.6}$$

The same quantity is the magnitude of the rim's tangential velocity relative to the wheel center. It is not the ground-frame velocity of the material point touching the ground: that point is instantaneously at rest in ideal rolling. For a robot with multiple wheels, their velocities must be combined using the robot's kinematic model to obtain its overall motion.

Combining the encoder conversion, reduction, and rolling relation gives

$$\boxed{\overline{v} = \frac{2\pi r}{NR} \frac{\Delta n}{\Delta t}.} \tag{8.7}$$

This is a wheel-based estimate. If the wheel slips, encoder measurements can still describe shaft rotation accurately while failing to describe travel over the ground.

### 8.6.1 Worked example: reconstructing motion

Suppose the motor encoder provides $N = 8$ counted ticks per revolution, the reduction is $R = 100$, and the wheel radius is $r = 0.05\,\mathrm{m}$. The measured count rate is $q = 40\,\mathrm{ticks/s}$. First convert units:

$$\begin{aligned}\overline{\omega}_m &= 40\,\frac{\mathrm{ticks}}{\mathrm{s}}\frac{1\,\mathrm{rev}}{8\,\mathrm{ticks}}\frac{2\pi\,\mathrm{rad}}{\mathrm{rev}} \\ &= 10\pi\,\mathrm{rad/s} \approx 31.416\,\mathrm{rad/s}.\end{aligned}$$

The transmission then gives

$$\overline{\omega} = \frac{10\pi}{100} = 0.1\pi\,\mathrm{rad/s} \approx 0.31416\,\mathrm{rad/s}.$$

Finally,

$$\overline{v} = 0.05(0.1\pi) = 0.005\pi\,\mathrm{m/s} \approx 0.015708\,\mathrm{m/s}.$$

Thus the estimated rolling speed is about $1.57\,\mathrm{cm/s}$. The first step changes units; the second describes the transmission; the third converts angular motion into linear motion using the wheel radius.

## 8.7 From desired wheel speed to desired motor speed

The same relationships can be used in reverse. A desired rolling speed $v_d$ implies

$$\omega_d = \frac{v_d}{r}, \qquad \boxed{\omega_{m,d} = \frac{R}{r}v_d.} \tag{8.8}$$

Using the same hardware and $v_d = 0.1\,\mathrm{m/s}$,

$$\omega_d = \frac{0.1}{0.05} = 2\,\mathrm{rad/s}, \qquad \omega_{m,d} = 100(2) = 200\,\mathrm{rad/s}.$$

The corresponding expected encoder count rate would be

$$q_d = \frac{N}{2\pi}\omega_{m,d} = \frac{800}{\pi}\,\mathrm{ticks/s} \approx 254.65\,\mathrm{ticks/s}.$$

This calculation specifies a target motor speed. It does not yet determine which PWM command will achieve it under a particular load.

## 8.8 Closing the speed-control loop

In open-loop operation, a chosen command is applied without correcting it from the measured outcome. Feedback instead compares the desired motor speed with the measured motor speed:

$$e(t) = \omega_{m,d}(t) - \overline{\omega}_m(t). \tag{8.9}$$

Both terms must use the same units and shaft reference. Subtracting a wheel speed from a motor speed, or radians per second from ticks per second, would not produce the intended error.

For a fixed forward direction, a positive error indicates that the motor is too slow; a negative error indicates that it is too fast. A controller uses this information to adjust the command. The encoder return path in Figure 8.1 completes this feedback loop. A motor operated with such feedback is part of a servo system; this does not imply that it is an integrated hobby position servo.

### 8.8.1 Setting a command versus changing a command

Let $u$ be a normalized drive command. A proportional controller directly sets the command from the current error:

$$u(t) = K_P e(t), \qquad K_P > 0. \tag{8.10}$$

For a forward-only drive, the applied duty is limited to $[0, 1]$. A driver that supports reversal can instead interpret a signed command through separate direction and duty settings.

There is a useful distinction between Equation (8.10) and the instruction "increase the existing command while the error is positive." If the latter is implemented as

$$\dot{u}(t) = ke(t), \qquad \text{then} \qquad u(t) = u(0) + k \int_0^t e(\tau)\, d\tau. \tag{8.11}$$

It accumulates error and is integral action on the command. With sample interval $T_s$, a corresponding update is $u_{k+1} = u_k + kT_s e_k$, followed by the appropriate command limits. The distinction matters: a dot over the command changes the controller.

### 8.8.2 A brief introduction to PID

A conventional ideal proportional–integral–derivative controller combines three contributions to the command itself:

$$u(t) = K_P e(t) + K_I \int_0^t e(\tau)\, d\tau + K_D \frac{de(t)}{dt}. \tag{8.12}$$

The proportional term responds to present error, the integral term to accumulated error, and the derivative term to the rate of change of error. The gains have different units because these three inputs have different units.[2]

Proportional control can leave a steady speed error when a nonzero command is needed to sustain motion. Integral action can remove a constant offset when the closed loop is stable and the requested speed is achievable. Derivative action is sensitive to measurement noise and is usually filtered in practical implementations. Command saturation also needs attention: continued accumulation while the drive is already at its limit can produce integral windup.

Choosing gains determines how quickly the system responds and whether it oscillates or becomes unstable. A fast processor alone does not guarantee good control. The feedback update period and the PWM switching period are distinct: many switching cycles may occur between feedback updates. Detailed controller design is beyond the present introduction; the essential idea is to measure the outcome and use the error to correct the drive.

[2] See the parallel PID form in MathWorks' PID overview.

## 8.9 Additional exercises

Use positive directions that correspond through the transmission. Unless otherwise stated, the encoder is on the motor shaft and the wheel rolls without slip.

1. **PWM timing.** A signal switches between 0 and 5 V at 1 kHz. Find its period, high time, and average voltage for duty cycles of 20%, 50%, and 80%. Explain why the average voltage is not an angular velocity.

2. **Command resolution.** Use $D = n/255$ for an 8-bit command. Choose the nearest integer command for 60% duty and calculate the resulting average of a 0–5 V signal. How many command values are available?

3. **Encoder conversion.** A motor encoder produces 12 counted events per revolution. During 0.2 s, the controller records 36 forward counts. Find the count rate, motor speed in revolutions per second, and motor speed in radians per second. For $R = 50$ and $r = 0.04\,\text{m}$, find the wheel's rolling speed.

4. **Desired motion.** For the hardware in Exercise 3, determine the desired motor speed and expected count rate for $v_d = 0.12\,\text{m/s}$. Why does this not determine the required duty cycle uniquely?

5. **Counting conventions.** A quadrature encoder produces eight complete electrical cycles per motor revolution on each channel. Determine $N$ when counting only rising edges of A and when counting every edge of both A and B. What speed error results if software uses the first value of $N$ while actually counting with the second method?

6. **Resolution and slip.** Using $N = 8$, $R = 100$, and $r = 0.05\,\text{m}$, find the inferred travel per count. Find the change in estimated speed caused by one extra count in windows of 0.01 and 0.1 s. Explain the resolution–delay tradeoff and why neither window can correct wheel slip by itself.

7. **Feedback.** Let $\omega_{m,d} = 200\,\text{rad/s}$ and $\overline{\omega}_m = 170\,\text{rad/s}$. Compute the error. With $K_P = 0.01\,\text{s/rad}$, find the proportional command $u = K_P e$. Separately, starting from $u_k = 0.4$, evaluate the incremental update $u_{k+1} = u_k + kT_s e_k$ for $k = 0.2\,\text{rad}^{-1}$ and $T_s = 0.01\,\text{s}$. Explain why these are different controllers.

# Chapter 9

# Differential-drive kinematics and odometry

Controlling one wheel does not yet describe how an entire robot moves. A mobile robot combines the motion of several wheels, and its position changes in a fixed reference frame. This chapter develops that connection for a differential-drive robot, then uses the resulting model to estimate position and orientation from wheel measurements.

## 9.1 Geometry and assumptions

A *differential-drive robot* has two independently driven wheels on a common axle line. Each wheel has its own motor; the axle line need not be a single mechanical shaft. A passive caster can provide support without setting the drive velocities. This arrangement allows the robot to travel straight, follow a curve, or turn in place.

Choose the midpoint $P$ between the drive wheels as the origin of the robot frame. Let $x_R$ point forward, $y_R$ point left, and $z_R$ point upward. In the world frame, the midpoint has coordinates $(x, y)$ and the heading $\theta$ is measured counterclockwise from $x_W$ to $x_R$. The planar pose is

$$q = \begin{bmatrix} x \\ y \\ \theta \end{bmatrix}. \tag{9.1}$$

It is also the state of the kinematic model used here. A model that includes motor or body dynamics would generally require additional state variables.

Assume that the robot moves on a plane, the chassis is rigid, the wheel radii are equal to $r$, and the wheel-center separation is $d > 0$. Both wheels remain in contact with the ground and roll without longitudinal or lateral slip. These are ideal kinematic constraints, not a claim that physical friction is infinite. They are useful when the available traction is sufficient for the motion being considered.

The wheel angular velocities $\omega_R$ and $\omega_L$ describe rotation about the wheel axles. The robot angular velocity $\omega = \dot{\theta}$ describes yaw about the vertical axis. These are different rotations. Choose the sign of each wheel coordinate so that positive wheel rotation produces forward rolling. Then the signed

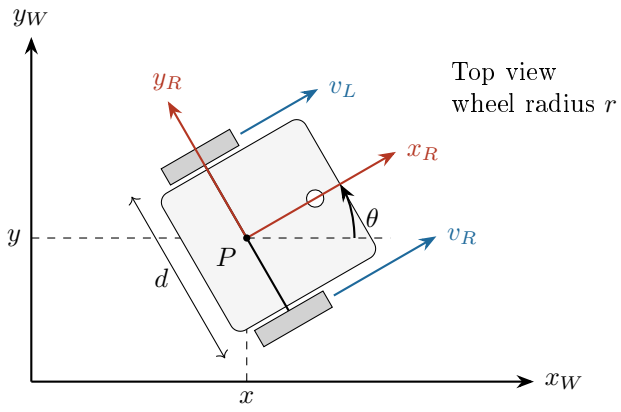



Figure 9.1: A differential-drive robot viewed from above. The frame origin is the axle midpoint. Wheel-center separation is $d$, and each wheel has radius $r$. Both wheel velocities are positive in the robot's forward direction.

wheel-center velocities along $x_R$ are

$$v_R = r\omega_R, \qquad v_L = r\omega_L. \tag{9.2}$$

Here the subscripts R and L mean right and left wheels; they do not denote coordinate frames. Motor-side encoder rates must first be converted through the transmission as in Chapter 8.

## 9.2 From wheel velocities to robot motion

Let $v$ be the forward velocity of $P$. A point at lateral coordinate $y_R$ on the axle has forward velocity $v - \omega y_R$: the contribution from rotation changes sign on opposite sides of the midpoint. Since the right wheel is at $y_R = -d/2$ and the left wheel at $y_R = d/2$,

$$v_R = v + \frac{d}{2}\omega, \qquad v_L = v - \frac{d}{2}\omega.$$

Adding and subtracting these two equations gives

$$\boxed{v = \frac{v_R + v_L}{2} = \frac{r}{2}(\omega_R + \omega_L), \qquad \omega = \frac{v_R - v_L}{d} = \frac{r}{d}(\omega_R - \omega_L).} \tag{9.3}$$

The midpoint velocity is the mean of the two wheel velocities. Their difference determines the turning rate. For the same velocity difference, a larger wheel separation produces a smaller turning rate.

Several limiting cases help check the signs. Equal wheel velocities give $\omega = 0$, so the robot travels straight. Equal and opposite wheel velocities give $v = 0$, so the midpoint remains stationary while the body turns. With both wheels moving forward, a faster right wheel gives positive $\omega$, so the robot turns left.

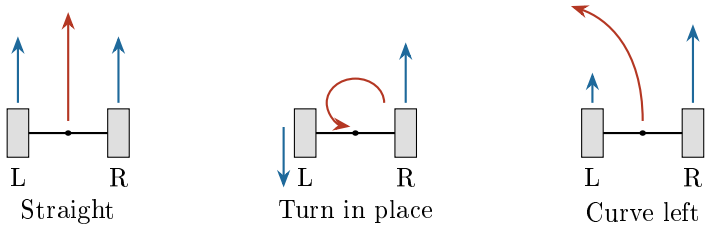



Figure 9.2: Three motions viewed from above, with forward pointing upward. Arrow lengths indicate wheel speeds. In-place turning uses opposite wheel directions; curved travel uses unequal forward speeds.

### 9.2.1 The inverse mapping

A motion planner may specify desired $v$ and $\omega$, while the wheel controllers require wheel angular velocities. Solving Equation (9.3) in the other direction gives

$$\boxed{\omega_R = \frac{2v + d\omega}{2r}, \qquad \omega_L = \frac{2v - d\omega}{2r}.} \tag{9.4}$$

The parameters $r$ and $d$ describe the hardware; $v$ and $\omega$ describe the requested motion. The resulting wheel commands must still lie within the motors' speed and acceleration capabilities. For odometry, measured wheel velocities are preferable to requested ones, since a command is not a guarantee of what the robot actually does.

### 9.2.2 Worked example: wheel and body velocities

Suppose $r = 0.05\,\text{m}$, $d = 0.30\,\text{m}$, $\omega_R = 6\,\text{rad/s}$, and $\omega_L = 2\,\text{rad/s}$. Then

$$v_R = 0.30\,\text{m/s}, \quad v_L = 0.10\,\text{m/s}, \quad v = 0.20\,\text{m/s}, \quad \omega = \frac{2}{3}\,\text{rad/s}.$$

The robot moves forward and turns left. Substituting these body velocities into Equation (9.4) recovers 6 and 2 rad/s.

## 9.3 The unicycle model

Once wheel velocities have been mapped to $(v, \omega)$, the chassis motion can be described by a single oriented point at $P$. This abstraction is called the *unicycle model*. It preserves forward motion and turning, without retaining the two individual wheels in the pose equations.

The midpoint cannot move sideways instantaneously under the no-slip assumption. Its velocity in robot coordinates is therefore $[v\ 0]^{\mathsf{T}}$. Rotating this vector into world coordinates gives

$$\begin{bmatrix} \dot{x} \\ \dot{y} \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} v \\ 0 \end{bmatrix}. \tag{9.5}$$

Together with $\dot{\theta} = \omega$, the model becomes

$$\boxed{\dot{x} = v\cos\theta, \qquad \dot{y} = v\sin\theta, \qquad \dot{\theta} = \omega.} \tag{9.6}$$

Equivalently, with input $u = [v\ \omega]^\mathsf{T}$,

$$\dot{q} = \begin{bmatrix} \cos\theta & 0 \\ \sin\theta & 0 \\ 0 & 1 \end{bmatrix} u. \tag{9.7}$$

The state has three components, while the input has two. A dot denotes a time derivative, so $\dot{x}$ and $\dot{y}$ have units of meters per second, and $\dot{\theta}$ has units of radians per second.

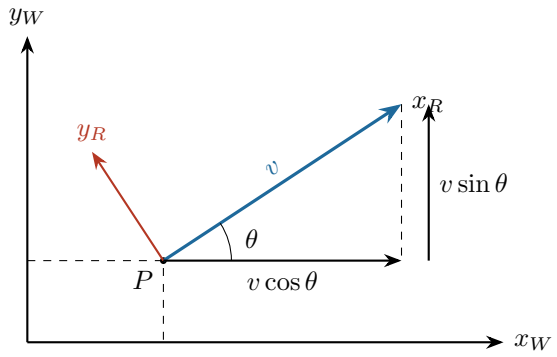



Figure 9.3: The forward velocity expressed in world coordinates. Its components are $v\cos\theta$ and $v\sin\theta$; a zero lateral velocity in the robot frame does not imply a zero world-frame $y$ velocity.

### 9.3.1 The lateral-motion constraint

Projecting the world velocity onto the robot's left-pointing axis gives

$$-\dot{x}\sin\theta + \dot{y}\cos\theta = 0. \tag{9.8}$$

This is a *nonholonomic* constraint: it restricts instantaneous velocities and cannot be replaced by a constraint on pose alone. The robot can still reach a location to its side by turning and driving; it cannot translate directly sideways while holding its heading fixed.

The choice of representative point matters. A point located a distance $a$ ahead of the axle midpoint has robot-frame velocity $[v\ a\omega]^\mathsf{T}$. Thus the simple zero-lateral-velocity statement applies to the chosen axle midpoint, not to every point of a turning chassis.

### 9.3.2 What the abstraction includes

The unicycle equations are kinematic: they relate velocities to changes in pose. They do not predict the forces, torques, balance, or motor transients needed to achieve those velocities. They are useful when low-level control and traction make the commanded motion a reasonable approximation, or when measured velocities are used directly.

Similar abstractions can be useful for planning the paths of more complex systems. Their admissible inputs must still reflect the actual platform. For example, a car-like vehicle generally cannot turn in place, even though an unconstrained unicycle model allows $v = 0$ and $\omega \neq 0$. A simplified path model also does not replace a balance model for a walking robot.

## 9.4 Odometry as integration of measured motion

*Wheel odometry* estimates changes in pose from wheel rotation. Starting from an initial pose $q(t_0)$, the unicycle model implies

$$x(t) = x(t_0) + \int_{t_0}^{t} v(\tau)\cos\theta(\tau)\,d\tau, \tag{9.9}$$

$$y(t) = y(t_0) + \int_{t_0}^{t} v(\tau)\sin\theta(\tau)\,d\tau, \tag{9.10}$$

$$\theta(t) = \theta(t_0) + \int_{t_0}^{t} \omega(\tau)\,d\tau. \tag{9.11}$$

This produces an estimate relative to the initial frame. Setting $q(t_0) = [0\ 0\ 0]^\mathsf{T}$ places that frame at the starting pose; it does not establish the robot's location in an independently defined building map.

On a computer, measurements arrive at discrete times $t_k$. Write $h = t_{k+1} - t_k$ for the sampling interval and $q_k$ for the estimated pose at $t_k$. The index $k+1$ means the next sample, not necessarily one second later. If intervals vary, use their measured durations $h_k$.

### 9.4.1 Forward Euler integration

The derivative definition motivates the approximation

$$\dot{x}(t_k) \approx \frac{x(t_k + h) - x(t_k)}{h}.$$

Using the velocity and heading at the beginning of the interval gives the *forward Euler* update:

$$\boxed{\begin{aligned} x_{k+1} &= x_k + h v_k \cos\theta_k, \\ y_{k+1} &= y_k + h v_k \sin\theta_k, \\ \theta_{k+1} &= \theta_k + h\omega_k. \end{aligned}} \tag{9.12}$$

Each new estimate becomes the starting point for the next step. All three right-hand sides use the old state: updating the heading first and then using that new heading for translation implements a different method.

For a general model $\dot{q} = f(q, u)$, the same construction is

$$q_{k+1} = q_k + h f(q_k, u_k). \tag{9.13}$$

This is numerical integration of a differential equation. The issue is approximating motion over finite sampling intervals, not an inability of computers to manipulate derivatives or limits symbolically.

## 9.5 Which values belong in an integration step?

Over one interval, the exact displacement depends on the velocity throughout that interval. A numerical method specifies how the available information approximates that motion. Choosing

values consistently matters; arbitrary mixtures of old and new variables need not have the intended accuracy.

For translation, the quantities being integrated are $v\cos\theta$ and $v\sin\theta$. For heading, the integrand is $\omega$. The signed area between an integrand and its approximation is the error in its integral. A plot of heading alone does not directly give the error in position.

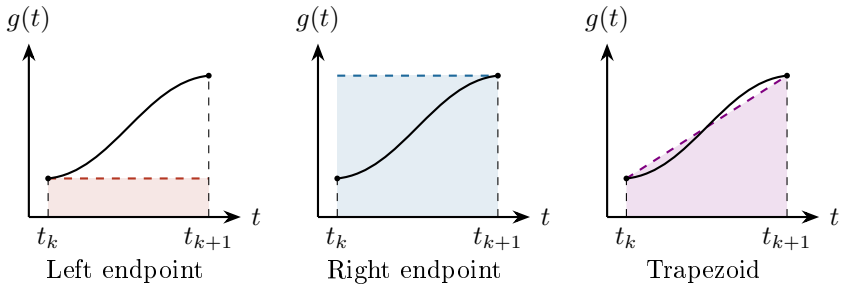



Figure 9.4: Approximating a scalar integrand $g(t)$ over one interval. Left and right endpoint values give rectangles; a straight line between endpoint values gives the trapezoidal rule. Its area equals the interval length times the mean of those endpoint values.

### 9.5.1 Backward Euler

Backward Euler evaluates the derivative at the new state and endpoint input:

$$q_{k+1} = q_k + hf(q_{k+1}, u_{k+1}).$$

In general this is an implicit equation that must be solved. For the unicycle with externally supplied endpoint velocities, it can be evaluated in sequence:

$$\begin{aligned}
\theta_{k+1} &= \theta_k + h\omega_{k+1},\\
x_{k+1} &= x_k + hv_{k+1}\cos\theta_{k+1},\\
y_{k+1} &= y_k + hv_{k+1}\sin\theta_{k+1}.
\end{aligned}$$

Measured endpoint inputs are available only after that endpoint is reached. This timing is compatible with reconstructing past motion, but it must not be confused with predicting future motion from measurements not yet available.

### 9.5.2 Trapezoidal integration

The trapezoidal method averages the two endpoint *derivatives*:

$$q_{k+1} = q_k + \frac{h}{2}\big[f(q_k, u_k) + f(q_{k+1}, u_{k+1})\big].$$

For the unicycle with known endpoint inputs, first compute $\theta_{k+1} = \theta_k + \frac{h}{2}(\omega_k + \omega_{k+1})$, then use

$$\begin{aligned}
x_{k+1} &= x_k + \frac{h}{2}\big[v_k\cos\theta_k + v_{k+1}\cos\theta_{k+1}\big],\\
y_{k+1} &= y_k + \frac{h}{2}\big[v_k\sin\theta_k + v_{k+1}\sin\theta_{k+1}\big].
\end{aligned}$$

In particular,

$$\frac{v_k \cos\theta_k + v_{k+1}\cos\theta_{k+1}}{2} \neq \frac{v_k + v_{k+1}}{2}\cos\left(\frac{\theta_k + \theta_{k+1}}{2}\right)$$

in general. Averaging the variables separately is a different approximation because the velocity model is nonlinear in heading.

### 9.5.3 Runge–Kutta methods

Runge–Kutta denotes a family of methods with specified intermediate derivative evaluations, rather than any operation that takes an average. For example, the explicit second-order method known as Heun's method uses an Euler prediction followed by an average of slopes:

$$a_1 = f(q_k, u_k), \qquad \widetilde{q}_{k+1} = q_k + h a_1,$$
$$a_2 = f(\widetilde{q}_{k+1}, u_{k+1}), \qquad q_{k+1} = q_k + \frac{h}{2}(a_1 + a_2).$$

Here the second slope uses a *predicted* endpoint state, whereas the implicit trapezoidal method uses the final endpoint state. Inputs at the required evaluation times must be supplied or approximated consistently. Detailed solver design is beyond the present scope.[1]

Both forward and backward Euler are first-order integration methods. The trapezoidal and Heun methods are second order for sufficiently smooth problems under the usual convergence assumptions. This refers to error over a fixed time span as the step size decreases. A piecewise constant approximation to an integrand should not be confused with calling Euler a "zeroth-order integration method."

### 9.5.4 Worked example: one interval, different methods

Let $q_0 = [0\ 0\ 0]^\mathsf{T}$, $h = 1\,\text{s}$, and suppose measured endpoint velocities are

$$v_0 = 1\,\text{m/s}, \quad v_1 = 2\,\text{m/s}, \qquad \omega_0 = 0\,\text{rad/s}, \quad \omega_1 = 0.5\,\text{rad/s}.$$

Forward Euler uses the beginning values, so $x_1 = 1$, $y_1 = 0$, and $\theta_1 = 0$. Backward Euler uses the endpoint values and gives $\theta_1 = 0.5$, $x_1 = 2\cos(0.5)$, and $y_1 = 2\sin(0.5)$. The trapezoidal heading is $\theta_1 = 0.25$, giving $x_1 = \frac{1}{2}[1 + 2\cos(0.25)]$ and $y_1 = \sin(0.25)$.

| Method | $x_1$ (m) | $y_1$ (m) | $\theta_1$ (rad) |
|---|---|---|---|
| Forward Euler | 1.0000 | 0.0000 | 0.0000 |
| Backward Euler | 1.7552 | 0.9589 | 0.5000 |
| Trapezoidal | 1.4689 | 0.2474 | 0.2500 |

The endpoint data alone do not uniquely determine the true trajectory between samples. These are three approximations, not three exact answers. If instead $u_0$ is explicitly held constant throughout the interval, the exact motion is straight travel by one meter; the value $u_1$ then begins the next interval. Distinguishing endpoint measurements from held commands is part of specifying the problem.

[1] For the standard Euler, Heun, and implicit trapezoidal formulas, see Marshall Hampton's notes on numerical differential equations.

## 9.6 Sampling interval and missed motion

Reducing the interval can improve the approximation when the motion changes rapidly. Higher-order integration does not recover changes that were never captured by the measurements. Figure 9.5 shows how an apparently smooth interpolation can miss substantial variation.

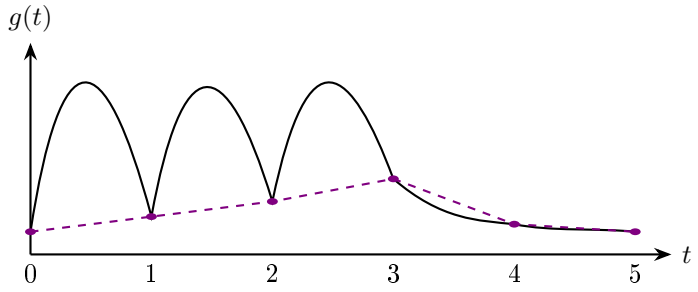



Figure 9.5: Sparse samples can miss motion between sampling instants. The solid curve represents a varying signal; the dashed line interpolates the displayed samples. A more sophisticated solver cannot infer the hidden peaks from these samples alone.

The useful interval depends on speed, turning rate, sensor timing, and required accuracy. There is no universal sampling period for all ground robots or all flying robots. Shorter windows also contain fewer encoder counts and can make count-based speed estimates coarser, as discussed in Chapter 8. Processing cost, measurement resolution, and delay must be considered together. Variable time steps are possible, provided the measurement timestamps and integration intervals remain consistent.

### 9.6.1 A constant-input benchmark

For constant $v$ and $\omega \neq 0$ over an interval, the unicycle equations can be integrated exactly:

$$\theta_{k+1} = \theta_k + h\omega, \tag{9.14}$$

$$x_{k+1} = x_k + \frac{v}{\omega}[\sin(\theta_k + h\omega) - \sin\theta_k], \tag{9.15}$$

$$y_{k+1} = y_k - \frac{v}{\omega}[\cos(\theta_k + h\omega) - \cos\theta_k]. \tag{9.16}$$

For $\omega = 0$, the motion is straight: $x_{k+1} = x_k + hv\cos\theta_k$ and $y_{k+1} = y_k + hv\sin\theta_k$. This special case provides a useful check on numerical integration, not an exact model of arbitrary time-varying or slipping motion.

Starting at zero pose with $v = 0.2\,\mathrm{m/s}$, $\omega = 0.5\,\mathrm{rad/s}$, and $h = 1\,\mathrm{s}$, the exact endpoint is approximately $(0.19177\,\mathrm{m}, 0.04897\,\mathrm{m}, 0.5\,\mathrm{rad})$. A single forward Euler step gives $(0.2\,\mathrm{m}, 0, 0.5\,\mathrm{rad})$: it updates heading but approximates the whole translation using the initial direction. Smaller steps follow the curved path more closely.

## 9.7 From wheel counts to an odometry update

An encoder naturally measures rotation accumulated over an interval. For signed counts $\Delta n_R, \Delta n_L$, motor-side count resolutions $N_R, N_L$, and reductions $R_R, R_L$, the inferred wheel rotations are

$$\Delta\phi_R = \frac{2\pi\Delta n_R}{N_R R_R}, \qquad \Delta\phi_L = \frac{2\pi\Delta n_L}{N_L R_L}.$$

For the equal-radius model, define

$$\Delta s = \frac{r}{2}(\Delta\phi_R + \Delta\phi_L), \qquad \Delta\theta = \frac{r}{d}(\Delta\phi_R - \Delta\phi_L). \tag{9.17}$$

Then an Euler-style increment update is

$$x_{k+1} = x_k + \Delta s\cos\theta_k, \quad y_{k+1} = y_k + \Delta s\sin\theta_k, \quad \theta_{k+1} = \theta_k + \Delta\theta.$$

The accumulated counts already describe the completed interval; they are not instantaneous measurements of velocity at its end. With a short interval, using $\theta_k + \Delta\theta/2$ for the translation direction often improves the turning approximation. Under constant $v, \omega$, this mid-heading approximation uses the arc length as a chord length; it is still approximate. Total wheel increments alone do not determine the translation exactly if the turning pattern varies within the interval.

A practical implementation initializes the pose and encoder baselines, reads both count increments over matching intervals, converts counts to wheel motion, and updates the pose using one consistent integration rule. Retain the old pose until all components of the update have been computed. It is convenient to integrate heading without wrapping and wrap it only for display; directly averaging angles near $-\pi$ and $\pi$ can otherwise give a misleading direction.

## 9.8 Why odometry drifts

Each update builds on the preceding estimate, so errors in motion and heading carry into later updates. Sources include encoder quantization, missed counts, incorrect wheel radii or separation, wheel slip, and numerical integration error. A heading error is especially consequential because it changes the direction used to project subsequent forward motion.

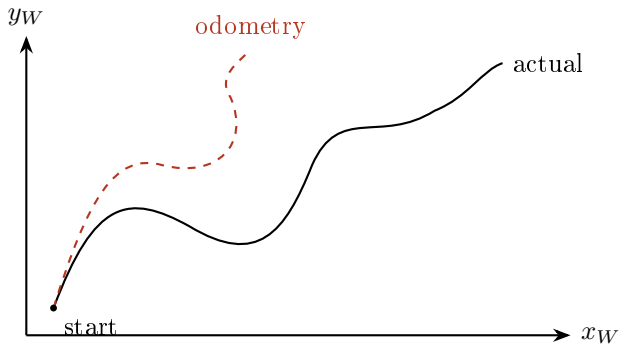



Figure 9.6: Illustrative actual and estimated trajectories starting from the same pose. Wheel odometry has no external position reference to pull the estimate back toward the actual path. Separation need not increase at every step, although uncertainty generally grows without correction.

Reducing the integration step addresses discretization error, but does not remove slip or calibration bias. Conversely, drift is not inevitable solely because integration is numerical: perfect motion information and an exact model would recover the relative trajectory. In practice, wheel odometry is useful over short intervals, while external observations such as satellite positioning or recognized landmarks can constrain accumulated error. Combining such information belongs to the broader problem of localization.

## 9.9 Additional exercises

Assume equal wheel radii, no slip, counterclockwise positive heading, and positive wheel velocities for forward rolling. Use radians in computations.

1. **Wheel-to-body mapping.** Let $r = 0.05\,\mathrm{m}$ and $d = 0.30\,\mathrm{m}$. Find $(v, \omega)$ for wheel rates $(\omega_R, \omega_L) = (4, 4), (4, -4), (6, 2)$ in rad/s. Describe each motion and check the units of both results.

2. **Inverse kinematics.** For the same robot, compute the wheel rates required for $v = 0.15\,\mathrm{m/s}$ and $\omega = 0.4\,\mathrm{rad/s}$. If each wheel is limited to $4\,\mathrm{rad/s}$ in magnitude, is the request feasible? Scale both body commands by a common factor to fit the limit.

3. **World-frame velocity.** At heading $\theta = \pi/3$, the robot has $v = 0.2\,\mathrm{m/s}$ and $\omega = -0.5\,\mathrm{rad/s}$. Compute $\dot{q}$ and verify Equation (9.8). Explain why the robot can have nonzero $\dot{y}$ without moving sideways in its own frame.

4. **Two Euler steps.** Start at $q_0 = [0\ 0\ 0]^\mathsf{T}$ with $h = 0.5\,\mathrm{s}$. Apply $(v_0, \omega_0) = (0.2, 1)$ over the first interval and $(v_1, \omega_1) = (0.2, 0)$ over the second, in SI units. Compute $q_1, q_2$ with forward Euler. Explain why updating heading before position changes the result.

5. **Integration accuracy.** For constant $v = 0.2\,\mathrm{m/s}$ and $\omega = 0.5\,\mathrm{rad/s}$, start at zero pose and integrate for one second using one Euler step and then ten steps. Compare both position estimates with Equation (9.16). Which source of error is being reduced, and which real-world errors are absent from this comparison?

6. **Encoder odometry.** Let $N_R = N_L = 8$, $R_R = R_L = 100$, $r = 0.05\,\mathrm{m}$, and $d = 0.30\,\mathrm{m}$. Over one interval, record 40 right counts and 24 left counts. Find $\Delta\phi_R, \Delta\phi_L$, $\Delta s$, and $\Delta\theta$. Update an initial pose $(1\,\mathrm{m}, 2\,\mathrm{m}, \pi/2)$ using the initial heading, then the mid-heading approximation.

7. **Model and measurement limits.** Explain why decreasing the step size cannot recover a brief wheel slip from encoder data alone. Give an example of an external observation that could help correct pose drift. Why is the final pose not determined uniquely by just two endpoint samples of $v$ and $\omega$?

# Chapter 10

# Robotic arms and direct kinematics

A mobile robot can often be modeled as one rigid body moving through the world. A robotic arm instead consists of several rigid bodies whose relative poses change at joints. Its configuration must describe those internal motions as well as the resulting position and orientation of its tool. The coordinate transformations developed earlier provide a systematic way to connect joint variables to tool pose.

## 10.1 Links, joints, and the end effector

The individual rigid bodies of a manipulator are called *links*. *Joints* connect adjacent links and constrain their relative motion. A sequence of links and joints forms a *kinematic chain*. Here we consider a serial, open chain with a fixed base: there is one path from the base to the terminal link, with no closed loops.

Number the base as link 0 and the moving links as $1, \ldots, n$. Joint $i$ connects link $i-1$ to link $i$. The terminal tool is the *end effector*: it may be a gripper, welding tool, drill, or another device selected for the task. A tool rigidly attached to the last link is part of that link for the arm's kinematic model. Additional tool motions, such as opening gripper fingers, require extra coordinates if they are included in the model.

### 10.1.1 Revolute and prismatic joints

Two common one-degree-of-freedom joints are:

- A *revolute* or rotational joint, denoted R, permits relative rotation about an axis. Its joint variable is an angle, typically measured in radians.
- A *prismatic* joint, denoted P, permits relative translation along an axis. Its joint variable is a displacement, typically measured in meters.

These names describe the joints, not the rigid links themselves. In a schematic, a cylinder can indicate a revolute joint with its axis along the cylinder's length. A sliding block can indicate a prismatic joint, but its translation direction must also be shown.




Figure 10.1: Symbols for one-degree-of-freedom joints. The revolute coordinate measures relative rotation; the prismatic coordinate measures relative translation. Joint zero positions and positive directions are part of the model definition.

A chain described as RRRRP has four revolute joints followed by one prismatic joint. This sequence alone does not determine its geometry: the axes' positions and directions, the distances between joints, and the tool attachment are also needed. Two joint axes that appear parallel in one configuration need not remain parallel when an intervening joint rotates.

## 10.2 Joint variables and degrees of freedom

An unconstrained rigid body in space has six degrees of freedom: three for translation and three for orientation. Five independent free bodies would therefore have 30 degrees of freedom. Connecting those bodies into a fixed-base serial chain changes the count because the joints constrain their relative motion.

For an open chain of $n$ independent one-degree-of-freedom joints, its configuration can be specified by

$$q = \begin{bmatrix} q_1 & q_2 & \cdots & q_n \end{bmatrix}^{\mathsf{T}}. \tag{10.1}$$

Thus the RRRRP chain has five configuration degrees of freedom. Its first four coordinates are angles and its last coordinate is a displacement. The components of a joint vector need not all have the same units.

This count assumes the stated open-chain structure and independent joints. Couplings, closed loops, a moving base, or multi-degree-of-freedom joints require a different count. Joint limits restrict the allowable range of each coordinate. An angle may be periodic for a freely rotating joint, but a physical joint need not allow unlimited rotation.

In a purely kinematic description, $q$ is often called the state. For a dynamic model that predicts motion from torques or forces, position alone is generally insufficient; joint velocities are also needed.

## 10.3 Direct and inverse kinematics

Attach a frame to the end effector and denote it by EE. The *direct kinematics*, also called *forward kinematics*, maps the joint configuration to the end-effector pose:

$$q \longmapsto H^W_{\mathrm{EE}}(q) = \begin{bmatrix} R^W_{\mathrm{EE}}(q) & p^W_{\mathrm{EE}}(q) \\ 0\,0\,0 & 1 \end{bmatrix}. \tag{10.2}$$

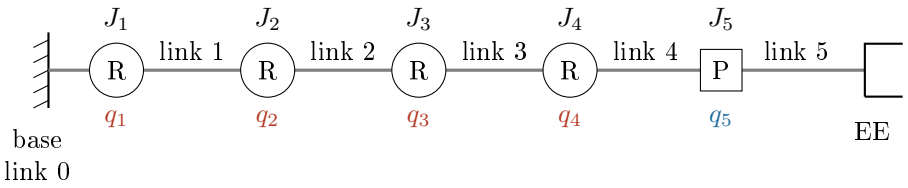



Figure 10.2: Topology of a five-joint RRRRP chain. Joint $i$ connects links $i-1$ and $i$. The drawing identifies connectivity and coordinate dependence; it does not prescribe the spatial directions of the joint axes.

The joint coordinates might be measured by encoders. The result states where the tool is and how it is oriented in the world frame.

Using roll, pitch, and yaw to describe orientation, one can also express the output locally as six scalar functions:

$$\begin{bmatrix} x^W_{\mathrm{EE}} \\ y^W_{\mathrm{EE}} \\ z^W_{\mathrm{EE}} \\ \varphi^W_{\mathrm{EE}} \\ \theta^W_{\mathrm{EE}} \\ \psi^W_{\mathrm{EE}} \end{bmatrix} = \begin{bmatrix} f_1(q) \\ f_2(q) \\ f_3(q) \\ f_4(q) \\ f_5(q) \\ f_6(q) \end{bmatrix}.$$

The homogeneous matrix is often the more convenient output because it avoids choosing among multiple angle representations. For a specified rigid serial-chain geometry and a given admissible $q$, the physical tool pose is unique. Different roll–pitch–yaw triples can nevertheless represent the same orientation, and angle extraction has singular cases.

### 10.3.1 The inverse question

Suppose a sensor detects an object and its pose has been expressed in the world frame. To grasp it, the robot needs joint coordinates that place its tool at a desired pose. This is *inverse kinematics*:

$$\text{given } H^W_{\mathrm{EE},d}, \qquad \text{find } q \text{ such that } H^W_{\mathrm{EE}}(q) = H^W_{\mathrm{EE},d}. \tag{10.3}$$

The desired tool frame may have a known offset from the object frame; their origins and axes do not have to coincide for a grasp to be correct.

Forward kinematics evaluates a function. Inverse kinematics solves equations, usually nonlinear ones. Depending on the geometry, target, and constraints, there may be no solution, one solution, several isolated solutions, or infinitely many solutions. A general closed-form inverse need not be available, even when the forward model is explicit.

The inverse-kinematics problem is not the same as inverting a homogeneous matrix. $(H^W_{\mathrm{EE}})^{-1}$ expresses the world frame relative to the end effector for an already known pose; it does not recover the joint coordinates that produce that pose.

## 10.4 Building direct kinematics from local transformations

Place a frame $i$ rigidly on each link $i$. For now, take the base frame 0 to coincide with the world frame. The essential observation is that the relative pose of adjacent link frames depends only on their connecting joint coordinate and the fixed geometry:

$$H_i^{i-1} = H_i^{i-1}(q_i). \tag{10.4}$$

For example, if joint 2 remains fixed while joint 1 moves, links 1 and 2 move together. Their world poses change, but $H_2^1$ does not. Moving a joint farther along the chain likewise does not alter the relative pose of links 1 and 2. Moving joint 2 itself does change $H_2^1$.

The local transformations compose in the same order as the coordinate transformations studied earlier:

$$\boxed{H_n^0(q) = H_1^0(q_1)H_2^1(q_2)\cdots H_n^{n-1}(q_n).} \tag{10.5}$$

For the five-joint chain,

$$H_5^0 = H_1^0 H_2^1 H_3^2 H_4^3 H_5^4 = \begin{bmatrix} R_5^0 & p_5^0 \\ 0\,0\,0 & 1 \end{bmatrix}.$$

Each factor maps coordinates from its subscript frame to its superscript frame. Applied to a point in frame 5, the rightmost factor acts first. The matching intermediate frame labels help verify the order; in general the factors cannot be reordered.

The same idea also gives the pose of an intermediate link: $H_i^0 = H_1^0 \cdots H_i^{i-1}$. This world-relative pose depends on the preceding joint coordinates, even though each adjacent-frame factor depends on only one coordinate.

### 10.4.1 Base and tool offsets

If the base does not coincide with the world frame, include its mounting transform. If the tool frame does not coincide with frame $n$, include the tool attachment transform:

$$H_{\mathrm{EE}}^W = H_0^W \left(H_1^0 \cdots H_n^{n-1}\right) H_{\mathrm{EE}}^n. \tag{10.6}$$

For a fixed base and a rigidly mounted tool, the outer two factors are constant. Thus the full problem separates into local joint transformations and fixed geometric offsets.

The remaining task is to choose frames and construct each local matrix systematically. Standard frame-assignment conventions make that process repeatable. Before introducing such a convention, a planar arm provides a case simple enough to derive directly from geometry.

## 10.5 A planar two-link manipulator

Consider two revolute joints and two links of lengths $\ell_1, \ell_2 > 0$, moving in the world $xy$ plane. The first joint is at the world origin. Let $q_1$ be the first link's angle from $x_W$, and let $q_2$ be the second link's angle *relative to the first link*. Both angles are positive counterclockwise.

Place the end-effector frame 2 at the tip, with $x_2$ along the second link and $y_2$ perpendicular to it in the plane. The second link's world orientation is $q_1 + q_2$, not $q_2$ alone.

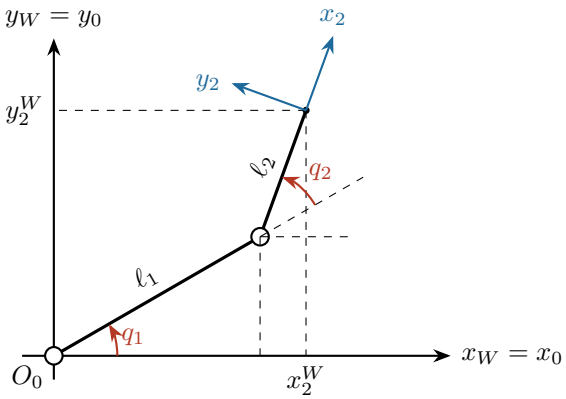



Figure 10.3: A planar RR arm. The second joint angle is measured from the extension of the first link. Dashed projections separate the two link contributions to the end-effector position.

Adding the two link vectors gives

$$\boxed{\begin{aligned} x_2^W &= \ell_1 \cos q_1 + \ell_2 \cos(q_1 + q_2), \\ y_2^W &= \ell_1 \sin q_1 + \ell_2 \sin(q_1 + q_2), \\ \psi_2^W &= q_1 + q_2. \end{aligned}} \tag{10.7}$$

With the frame axes chosen as stated, $z_2^W = 0$ and roll and pitch can both be taken as zero. The yaw is understood modulo $2\pi$ when it represents orientation rather than accumulated joint rotation.

### 10.5.1 Recovering the same result by matrix multiplication

Place frame 1 at the elbow, rigidly attached to link 1, with $x_1$ along that link. In planar homogeneous coordinates,

$$H_1^0(q_1) = \begin{bmatrix} \cos q_1 & -\sin q_1 & \ell_1 \cos q_1 \\ \sin q_1 & \cos q_1 & \ell_1 \sin q_1 \\ 0 & 0 & 1 \end{bmatrix}, \tag{10.8}$$

$$H_2^1(q_2) = \begin{bmatrix} \cos q_2 & -\sin q_2 & \ell_2 \cos q_2 \\ \sin q_2 & \cos q_2 & \ell_2 \sin q_2 \\ 0 & 0 & 1 \end{bmatrix}. \tag{10.9}$$

Multiplying them produces

$$H_2^0 = \begin{bmatrix} \cos(q_1 + q_2) & -\sin(q_1 + q_2) & x_2^W \\ \sin(q_1 + q_2) & \cos(q_1 + q_2) & y_2^W \\ 0 & 0 & 1 \end{bmatrix}, \tag{10.10}$$

where the translation entries agree with Equation (10.7). These are $3 \times 3$ planar transformations. The corresponding spatial matrix is $4 \times 4$, with a $z$ rotation, zero $z$ translation, and the usual homogeneous last row.

### 10.5.2 Worked example

Let $\ell_1 = 1\,\mathrm{m}$, $\ell_2 = 0.5\,\mathrm{m}$, $q_1 = \pi/6$, and $q_2 = \pi/3$. The second link is vertical because $q_1 + q_2 = \pi/2$. Therefore

$$x_2^W = \frac{\sqrt{3}}{2}\,\mathrm{m}, \qquad y_2^W = \left(\frac{1}{2} + \frac{1}{2}\right)\mathrm{m} = 1\,\mathrm{m}, \qquad \psi_2^W = \frac{\pi}{2}.$$

The planar pose matrix is

$$H_2^0 = \begin{bmatrix} 0 & -1 & \sqrt{3}/2 \\ 1 & 0 & 1 \\ 0 & 0 & 1 \end{bmatrix}.$$

The matrix and geometric constructions describe the same tool pose.

## 10.6 Task dimension, reachability, and redundancy

The planar RR arm has two independent joint coordinates. A full planar tool pose has three components $(x, y, \psi)$, so the arm cannot generally set all three independently. Specifying a reachable position can determine one or more joint configurations, each with an associated orientation; there is not generally a freely selectable yaw left over.

This is a limitation relative to the chosen task. If both joints have independent actuators, the arm is *fully actuated in joint space*. Having too few configuration degrees of freedom for an arbitrary task pose is different from *underactuation*, which concerns having fewer independent actuation inputs than configuration degrees of freedom.

A planar three-revolute-joint chain can generally control a full planar pose locally away from singular configurations. A planar chain with more than three independent joints is typically *kinematically redundant* for that task: different joint configurations can produce the same tool pose. This extra freedom can allow internal motion while the tool remains fixed, subject to joint limits, collisions, and the particular configuration. For a position-only task, even a three-joint planar arm can be redundant. Redundancy is therefore defined relative to the task, not by counting motors alone.[1]

Counting degrees of freedom is a useful first check, but is not a guarantee that a target can be reached. Geometry, joint limits, and singular configurations still matter even when the number of joints matches the number of task coordinates.

### 10.6.1 Position workspace of the two-link arm

With unrestricted revolute joints and no collision constraints, the distance $\rho$ of the tip from the base satisfies

$$|\ell_1 - \ell_2| \le \rho \le \ell_1 + \ell_2, \qquad \rho = \sqrt{(x_2^W)^2 + (y_2^W)^2}. \tag{10.11}$$

The outer radius occurs when the links are aligned in the same direction; the inner radius occurs when they fold back against one another. Rotating the first joint sweeps these distances around the base, producing an annular position workspace, or a disk when the link lengths are equal.

[1] For terminology and inverse-kinematics context, see Lynch and Park, Modern Robotics, Chapter 6.

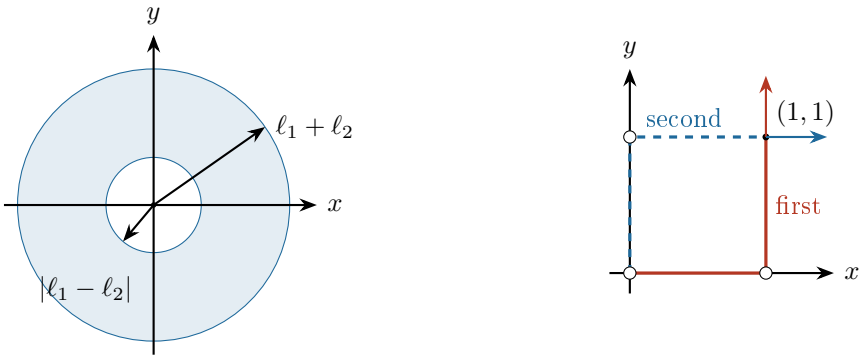



Figure 10.4: Left: the position workspace for unequal link lengths, neglecting joint limits and collisions. Right: two equal-length RR configurations reach the same point but have different end-effector orientations.

For $\ell_1 = 1\,\mathrm{m}$ and $\ell_2 = 0.5\,\mathrm{m}$, a point 2 m from the base is too far away, and a point 0.2 m from the base is too close. Neither has a position inverse-kinematics solution. A point inside the annulus may be position-reachable while a prescribed orientation at that point remains impossible.

### 10.6.2 Same position, different joint configurations

For equal link lengths $\ell_1 = \ell_2 = 1\,\mathrm{m}$, consider the target position $(1, 1)\,\mathrm{m}$. Both

$$(q_1, q_2) = (0, \pi/2) \qquad \text{and} \qquad (q_1, q_2) = (\pi/2, -\pi/2)$$

reach it. The first gives yaw $\pi/2$, while the second gives yaw 0. Thus these are two solutions to the *position-only* task, not two solutions to the same full-pose task. Requiring yaw $\pi/4$ at this point would reject both configurations and is not attainable by this ideal RR arm. The target specification determines which inverse problem is being asked.

The central tool for further analysis is the forward map: establish the geometry, define the frames and joint coordinates, construct the local transformations, and multiply them in the correct order. Workspace and inverse-kinematics questions can then be posed against a precise model.

## 10.7 The Denavit–Hartenberg frame convention

The product in Equation (10.5) reduces the direct-kinematics problem to finding the pose of each link relative to its predecessor. Arbitrary link frames are valid, but they can make those local matrices unnecessarily complicated. The *Denavit–Hartenberg* (DH) convention chooses frames so that each local transformation has just four parameters: two translations and two rotations. For a one-degree-of-freedom joint, only one of these parameters varies with joint motion.

We use the *standard DH convention*. In particular, joint $i$ acts about or along $z_{i-1}$, and the local matrix maps coordinates from frame $i$ to frame $i - 1$. Other conventions, including modified DH, use different frame assignments and matrix formulas. A parameter table must always be used with the convention that defines it.

### 10.7.1 Joint axes and common normals

Let $J_i$ denote the axis of joint $i$: its rotation axis for a revolute joint or its translation direction, represented by a chosen axis line, for a prismatic joint. For the generic construction, consider neighboring axes $J_i$ and $J_{i+1}$ that are *skew*: they neither intersect nor are parallel.

The shortest segment joining two skew lines is perpendicular to both. The line containing that segment is their *common normal*, denoted $\mathrm{CN}_i$. It intersects the axes at their closest points. Extending either joint axis farther does not change these points or the separation. The common normal is a geometric construction; it need not follow the shape of the physical link joining the joints.

### 10.7.2 Assigning an interior link frame

For $i = 1, \ldots, n-1$, place frame $i$ on link $i$ using the following rules:

1. Put $O_i$ at the intersection of $J_{i+1}$ and $\mathrm{CN}_i$.
2. Align $z_i$ with $J_{i+1}$ and choose its positive direction.
3. Direct $x_i$ along $\mathrm{CN}_i$, from $J_i$ toward $J_{i+1}$. The axis starts at $O_i$ and points in that same direction.
4. Complete a right-handed frame with $\hat{y}_i = \hat{z}_i \times \hat{x}_i$.

Here hats denote unit direction vectors. The common-normal construction makes $x_i$ perpendicular to $z_i$. Consequently, their directions determine $y_i$ uniquely; a drawing can omit $y_i$ without leaving the frame undefined.

The index shift is essential: $z_i$ follows joint $i+1$, whereas joint $i$ connects links $i-1$ and $i$ and acts about or along $z_{i-1}$. There are $n$ joints, $n$ moving links plus the base, and $n+1$ link frames numbered $0, \ldots, n$. An interior link $i$ lies between joints $i$ and $i+1$; the terminal link has no physical joint $n+1$.

Either orientation of a joint axis can be chosen initially. Reversing it changes the associated signs and frame assignments, so all parameters and joint-coordinate definitions must be made consistent with that choice. Once chosen, the frames remain attached to their links as the arm moves.

### 10.7.3 End frames and nonunique constructions

The interior rule cannot be applied literally at both ends of the chain. For frame 0, choose $O_0$ on $J_1$, align $z_0$ with $J_1$, and choose a perpendicular $x_0$. This base frame need not coincide with the world frame. At the other end, choose a terminal frame compatible with the DH construction; its $z_n$ is an auxiliary axis, not an additional joint. A desired tool frame can be connected to it by a fixed transform as in Equation (10.6).

The skew-axis rule also needs choices in special geometries. Distinct parallel axes have infinitely many common normals. Intersecting, nonparallel axes have zero separation, with a normal direction perpendicular to both. Coincident axes leave still more freedom. These situations are common in robot designs. The transformation derived below still applies once a consistent DH frame assignment has been made; the frame assignment need not be unique.

## 10.8 Four parameters for one adjacent-frame transformation

To relate frames $i-1$ and $i$, introduce a temporary frame $i'$. Its origin is where $J_i$ meets $\mathrm{CN}_i$. Choose $\hat{z}_{i'} = \hat{z}_{i-1}$ and $\hat{x}_{i'} = \hat{x}_i$, with $y_{i'}$ completing the right-handed frame. This frame combines the incoming joint axis with the outgoing common-normal direction.

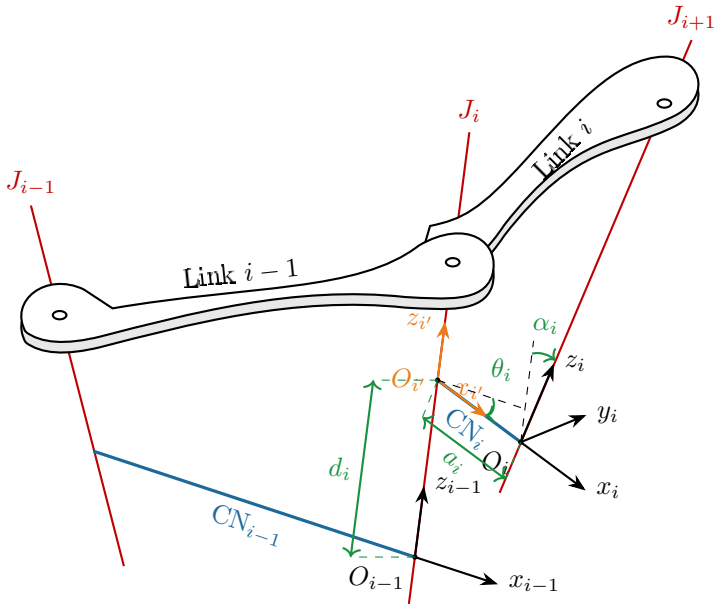



Figure 10.5: DH construction for two neighboring links. Red lines extend the three joint axes; blue lines show their common normals. The intermediate frame $i'$ is orange, and the four parameters are marked in green. Common normals are perpendicular to their joint axes in space, even when the perspective projection does not show a right angle. Omitted $y$ axes follow from the right-hand rule.

The four DH parameters describe two successive changes of frame:

- $d_i$ is the signed displacement from $O_{i-1}$ to $O_{i'}$ along $z_{i-1}$.
- $\theta_i$ is the angle from $x_{i-1}$ to $x_{i'} = x_i$ about $z_{i-1}$.
- $a_i$ is the displacement from $O_{i'}$ to $O_i$ along $x_i$.
- $\alpha_i$ is the angle from $z_{i'} = z_{i-1}$ to $z_i$ about $x_i$.

Angles follow the right-hand rule about their named positive axes. A positive angle appears counterclockwise when viewed from the positive end of that axis toward the origin. This viewing direction matters in a perspective drawing. The lengths $a_i, d_i$ have units of length; the angles $\alpha_i, \theta_i$ are measured in radians unless stated otherwise.

With $x_i$ directed from one skew axis toward the next, $a_i$ is their nonnegative separation. The offset $d_i$ may be positive, zero, or negative: it is a directed displacement, not an unsigned distance.

### 10.8.1 Reading the parameters without the temporary frame

The temporary frame helps derive the matrix, but it need not be drawn for every joint. Since the two displacement directions are perpendicular, the vector between the link-frame origins satisfies

$$\overrightarrow{O_{i-1}O_i} = d_i\hat{z}_{i-1} + a_i\hat{x}_i. \tag{10.12}$$

All vectors in this equation must be expressed in the same reference frame. Taking projections gives

$$d_i = \hat{z}_{i-1}\cdot\overrightarrow{O_{i-1}O_i}, \qquad a_i = \hat{x}_i\cdot\overrightarrow{O_{i-1}O_i}.$$

Thus the phrase "distance between the origins along an axis" means a *signed projection*, not the length of the diagonal joining them. For example, if $a_i = 0.4\,\mathrm{m}$ and $d_i = 0.3\,\mathrm{m}$, the origins are $0.5\,\mathrm{m}$ apart, but neither DH translation equals $0.5\,\mathrm{m}$.

Likewise, $\theta_i$ can be read directly from $x_{i-1}$ to $x_i$ about $z_{i-1}$, and $\alpha_i$ from $z_{i-1}$ to $z_i$ about $x_i$. Directions can be translated to a common point to compare their angles. The auxiliary frame adds no joint, degree of freedom, or extra row to the DH table.

## 10.9 Deriving the standard DH matrix

The temporary frame splits the desired transformation into

$$H_i^{i-1} = H_{i'}^{i-1}H_i^{i'}. \tag{10.13}$$

From frame $i-1$ to frame $i'$, the origin moves by $d_i$ along $z_{i-1}$ and the axes rotate by $\theta_i$ about that same axis:

$$H_{i'}^{i-1} = \begin{bmatrix} \cos\theta_i & -\sin\theta_i & 0 & 0 \\ \sin\theta_i & \cos\theta_i & 0 & 0 \\ 0 & 0 & 1 & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{10.14}$$

From $i'$ to $i$, the origin moves by $a_i$ along $x_{i'} = x_i$ and the axes rotate by $\alpha_i$ about that direction:

$$H_i^{i'} = \begin{bmatrix} 1 & 0 & 0 & a_i \\ 0 & \cos\alpha_i & -\sin\alpha_i & 0 \\ 0 & \sin\alpha_i & \cos\alpha_i & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \tag{10.15}$$

The translation in this second matrix is along $x$, even though the twist changes the direction of $z$. Matrix multiplication gives

$$\boxed{H_i^{i-1} = \begin{bmatrix} c_{\theta_i} & -s_{\theta_i}c_{\alpha_i} & s_{\theta_i}s_{\alpha_i} & a_ic_{\theta_i} \\ s_{\theta_i} & c_{\theta_i}c_{\alpha_i} & -c_{\theta_i}s_{\alpha_i} & a_is_{\theta_i} \\ 0 & s_{\alpha_i} & c_{\alpha_i} & d_i \\ 0 & 0 & 0 & 1 \end{bmatrix}}, \tag{10.16}$$

where $c_\beta = \cos\beta$ and $s_\beta = \sin\beta$. The translation column is expressed in frame $i-1$: the displacement $a_i$ is resolved along its $x$ and $y$ directions by $\theta_i$.

Using $4 \times 4$ elementary rotation and translation matrices, the same factorization is

$$H_i^{i-1} = \mathrm{Rot}_z(\theta_i)\,\mathrm{Trans}_z(d_i)\,\mathrm{Trans}_x(a_i)\,\mathrm{Rot}_x(\alpha_i).$$

This describes successive local frame placements: after the $z$ rotation, the $x$ operations use the rotated $x$ direction. When the matrix acts on a coordinate column, its rightmost factor acts first, as usual. A translation along an axis commutes with a rotation about that same axis, but the $z$ and $x$ groups cannot generally be interchanged.

### 10.9.1 Where the joint variable enters

The parameters $a_i$ and $\alpha_i$ describe fixed link geometry. For a revolute joint, $d_i$ is also constant and $\theta_i$ varies. For a prismatic joint, $\theta_i$ is constant and $d_i$ varies:

| Joint type | Variable parameter | Constant parameters |
|---|---|---|
| Revolute | $\theta_i = q_i + \theta_{i,0}$ | $a_i, \alpha_i, d_i$ |
| Prismatic | $d_i = q_i + d_{i,0}$ | $a_i, \alpha_i, \theta_i$ |

The constants $\theta_{i,0}$ and $d_{i,0}$ allow the joint coordinate's zero to differ from the DH parameter's zero. If the zeros coincide, these reduce to $\theta_i = q_i$ and $d_i = q_i$, respectively. These formulas assume $q_i$ increases in the chosen positive DH direction; an oppositely signed encoder coordinate must be converted accordingly.

This explains why four parameters still give a one-variable local map $H_i^{i-1}(q_i)$. The constants can be obtained from dimensions, a CAD model, or calibration. Moving another joint changes the world pose of these frames but does not introduce another independent variable into their relative transformation.

### 10.9.2 Worked example: checking one DH row

Consider a revolute joint with zero angular offset and

$$a_i = 0.4\,\mathrm{m}, \qquad d_i = 0.3\,\mathrm{m}, \qquad \alpha_i = \pi/2, \qquad q_i = \theta_i = \pi/2.$$

Substitution into Equation (10.16) gives

$$H_i^{i-1} = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0.4 \\ 0 & 1 & 0 & 0.3 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

The origin is at $(0, 0.4, 0.3)\,\mathrm{m}$ in frame $i-1$. The first rotation column shows that $x_i$ points along $y_{i-1}$; the third shows that $z_i$ points along $x_{i-1}$. The columns are orthonormal and right-handed. These geometric checks are often easier than finding a sign error by rereading the entire formula.

If the same constant values $a_i = 0.4\,\mathrm{m}$, $\alpha_i = \pi/2$, and $\theta_i = \pi/2$ instead describe a prismatic joint with $d_i = q_i$, only the third translation entry changes as that joint moves. A DH row must therefore specify which parameter is the joint variable, not merely four numbers at one configuration.

## 10.10 From a robot drawing to direct kinematics

The DH construction turns the geometric problem into a repeatable sequence:

1. Identify and number the $n$ physical joint axes $J_1, \ldots, J_n$. Record each joint's type and positive motion direction.
2. Construct the $n-1$ common normals between neighboring joint axes. Resolve nonunique choices consistently.
3. Assign frames $0, \ldots, n$, including the base and terminal choices. Drawing $x$ and $z$ is sufficient when right-handedness is understood.
4. Make a DH table with one row per joint, $i = 1, \ldots, n$. Record $a_i, \alpha_i, d_i, \theta_i$ and identify the variable entry.
5. Substitute each row into Equation (10.16) to obtain $H_i^{i-1}(q_i)$.
6. Multiply $H_n^0 = H_1^0 H_2^1 \cdots H_n^{n-1}$. Include the base and tool offsets when the requested frames differ from 0 and $n$.

There is no row 0 in this indexing. There are $n$ matrices even though only $n-1$ common normals join physical joint pairs: the terminal-frame choice supplies the remaining geometry. The temporary frames $i'$ are derivation aids and do not change these counts.

Before expanding a long product, check one simple configuration. Verify that each origin lies where the drawing predicts, that the rotation columns describe the intended axes, and that a positive change in $q_i$ moves the model in the intended direction. Different consistent frame choices may produce different DH tables. They should give the same physical tool pose when expressed in the same world and tool frames, with the corresponding joint-coordinate definitions.

The planar RR matrices derived earlier are a useful consistency check. Setting $a_1 = \ell_1$, $a_2 = \ell_2$, $\alpha_1 = \alpha_2 = 0$, $d_1 = d_2 = 0$, and $\theta_i = q_i$ in the spatial DH formula reproduces those planar transformations, with an additional unchanged $z$ coordinate.

## 10.11 Additional exercises

### 10.11.1 Arm kinematics and workspace

Assume ideal rigid links, independent joints, and no joint limits or collisions unless stated otherwise.

1. **Joint descriptions.** An RPR arm has a fixed base. State its number of configuration degrees of freedom, which links each joint connects, and the units of each component of $q$. Explain why the letters RPR alone are insufficient to compute the tool pose.

2. **Local versus world pose.** In a five-joint serial arm, hold $q_2$ fixed and change $q_1$. Which of $H_2^1$ and $H_2^0$ can change? Repeat the question when only $q_4$ changes. Explain your answers physically.

3. **Transform composition.** Write $H_{\mathrm{EE}}^W$ for a three-joint arm with a nonidentity base transform $H_0^W$ and a fixed tool transform $H_{\mathrm{EE}}^3$. State the mapping direction of each factor and explain why inverting the result does not solve inverse kinematics.

4. **Planar direct kinematics.** Let $\ell_1 = 1\,\mathrm{m}$, $\ell_2 = 0.5\,\mathrm{m}$, $q_1 = \pi/2$, and $q_2 = -\pi/2$. Compute the tip position, yaw, and planar homogeneous matrix. Verify the result by multiplying $H_1^0 H_2^1$.

5. **Workspace and orientation.** For the same link lengths, decide whether positions $(2, 0)$, $(0.2, 0)$, and $(1, 0)$ in meters are reachable. If the tip is at $(1.5, 0)$, can its yaw be $\pi/2$? Explain using the fully extended configuration.

6. **Two inverse solutions.** For $\ell_1 = \ell_2 = 1\,\mathrm{m}$, substitute $(q_1, q_2) = (0, \pi/2)$ and $(\pi/2, -\pi/2)$ into the forward equations. Compare their positions and orientations. Which solution meets the full target pose $(x, y, \psi) = (1, 1, 0)$?

7. **Task-dependent redundancy.** Compare planar arms with two, three, and six independent revolute joints for a position-only task and for a full planar-pose task. Assume generic nonsingular configurations. Explain why an arm with two independently driven joints is not necessarily underactuated, even though it cannot set an arbitrary planar tool pose.

### 10.11.2 DH frames and transformations

1. **Indices and frame counts.** A fixed-base arm has five one-degree-of-freedom joints. How many link frames, physical joint axes, common normals between consecutive physical axes, and DH table rows are needed? Which joint acts about or along $z_2$? Explain why $z_5$ does not imply a sixth joint.

2. **Signed displacements.** A DH construction has $\overrightarrow{O_{i-1}O_i} = -0.2\hat{z}_{i-1} + 0.5\hat{x}_i$ in meters. Find $d_i$, $a_i$, and the distance between the origins. Explain why the negative value is compatible with a valid frame assignment.

3. **Deriving the local matrix.** Multiply Equations (10.14) and (10.15) explicitly. Explain why the translation column contains $a_i \cos\theta_i$ and $a_i \sin\theta_i$, even though the second factor translates only along its own $x$ axis.

4. **A revolute DH row.** Let $a_i = 0.2\,\text{m}$, $d_i = 0.1\,\text{m}$, $\alpha_i = -\pi/2$, and $\theta_i = q_i + \pi/2$. Compute $H_i^{i-1}$ at $q_i = 0$. Read the origin and all three axis directions from your answer, and verify that the rotation is right-handed.

5. **A prismatic DH row.** Let $a_i = 0.3\,\text{m}$, $\alpha_i = 0$, $\theta_i = \pi/2$, and $d_i = q_i + 0.1\,\text{m}$. Compute $H_i^{i-1}$ for $q_i = 0.2\,\text{m}$ and $0.4\,\text{m}$. Which entries change? Express the displacement between these configurations in frame $i-1$.

6. **Recovering the planar arm.** Build a two-row DH table for the planar RR arm using the parameter choices at the end of Section 10.10. Multiply the spatial matrices and recover Equation (10.7). Identify the final $z$ direction.

7. **Consistency of conventions.** Explain why reversing a chosen joint-axis direction while leaving every parameter and encoder sign unchanged generally changes the predicted motion. Why can two different DH tables nevertheless model the same physical arm?

# Appendix A

# Rotation representation conversions

This reference collects conversions between rotation matrices, roll–pitch–yaw (RPY) angles, and unit quaternions. The matrix and angle conventions agree with Chapter 4, which introduces the practical use of unit quaternions.

## Conventions

All frames are right-handed, vectors are columns, and

$$R = R_z(\psi)R_y(\theta)R_x(\varphi) \in SO(3), \qquad SO(3) = \{R \in \mathbb{R}^{3\times 3} : R^{\mathsf{T}}R = I_3,\ \det R = 1\}.$$

The angles are roll $\varphi$, pitch $\theta$, and yaw $\psi$, with principal ranges $\varphi, \psi \in (-\pi, \pi]$ and $\theta \in [-\pi/2, \pi/2]$. Quaternions use the scalar-first convention

$$q = \begin{bmatrix} q_w & q_x & q_y & q_z \end{bmatrix}^{\mathsf{T}}, \qquad q_w^2 + q_x^2 + q_y^2 + q_z^2 = 1.$$

The signs in the formulas below specify the quaternion convention; a different component order must be converted before applying them. The quaternions $q$ and $-q$ represent the same rotation. Write $c_a = \cos a$ and $s_a = \sin a$. The function $\operatorname{atan2}(y, x)$ takes its arguments in that order.

## RPY angles to a rotation matrix

$$R = \begin{bmatrix} c_\psi c_\theta & c_\psi s_\theta s_\varphi - s_\psi c_\varphi & c_\psi s_\theta c_\varphi + s_\psi s_\varphi \\ s_\psi c_\theta & s_\psi s_\theta s_\varphi + c_\psi c_\varphi & s_\psi s_\theta c_\varphi - c_\psi s_\varphi \\ -s_\theta & c_\theta s_\varphi & c_\theta c_\varphi \end{bmatrix}.$$

## Rotation matrix to RPY angles

For $R = [r_{ij}]$ and $|\theta| < \pi/2$, the angles are recovered by the formulas below. At $\theta = \pm\pi/2$, yaw and roll are not separately determined; a singular-case convention is required. In particular, $\operatorname{atan2}(0, 0)$ has no unique geometric meaning.

$$\begin{aligned} \varphi &= \operatorname{atan2}(r_{32}, r_{33}), \\ \theta &= \operatorname{atan2}\left(-r_{31}, \sqrt{r_{32}^2 + r_{33}^2}\right), \\ \psi &= \operatorname{atan2}(r_{21}, r_{11}). \end{aligned}$$

## Unit quaternion to a rotation matrix

$$R = \begin{bmatrix} 1 - 2(q_y^2 + q_z^2) & 2(q_x q_y - q_w q_z) & 2(q_x q_z + q_w q_y) \\ 2(q_x q_y + q_w q_z) & 1 - 2(q_x^2 + q_z^2) & 2(q_y q_z - q_w q_x) \\ 2(q_x q_z - q_w q_y) & 2(q_y q_z + q_w q_x) & 1 - 2(q_x^2 + q_y^2) \end{bmatrix}.$$

This expression assumes a unit quaternion. Normalize a nonzero quaternion before using it; a zero quaternion does not describe an orientation.

## RPY angles to a unit quaternion

Using the same intrinsic Z–Y–X convention,

$$\begin{aligned} q_w &= c_{\varphi/2} c_{\theta/2} c_{\psi/2} + s_{\varphi/2} s_{\theta/2} s_{\psi/2}, \\ q_x &= s_{\varphi/2} c_{\theta/2} c_{\psi/2} - c_{\varphi/2} s_{\theta/2} s_{\psi/2}, \\ q_y &= c_{\varphi/2} s_{\theta/2} c_{\psi/2} + s_{\varphi/2} c_{\theta/2} s_{\psi/2}, \\ q_z &= c_{\varphi/2} c_{\theta/2} s_{\psi/2} - s_{\varphi/2} s_{\theta/2} c_{\psi/2}. \end{aligned}$$

## Unit quaternion to RPY angles

Away from the pitch singularities,

$$\begin{aligned} \varphi &= \operatorname{atan2}\big(2(q_w q_x + q_y q_z),\, 1 - 2(q_x^2 + q_y^2)\big), \\ \theta &= \arcsin\big(2(q_w q_y - q_z q_x)\big), \\ \psi &= \operatorname{atan2}\big(2(q_w q_z + q_x q_y),\, 1 - 2(q_y^2 + q_z^2)\big). \end{aligned}$$

For a normalized quaternion, the exact argument of arcsin lies in $[-1, 1]$. In floating-point arithmetic, clamp small roundoff excursions to that interval before evaluating arcsin. This does not resolve the pitch singularity, nor does it replace checking the quaternion's normalization.

## Rotation matrix to a unit quaternion

One conversion branch chooses $q_w \geq 0$:

$$q_w = \tfrac{1}{2}\sqrt{1 + r_{11} + r_{22} + r_{33}},$$
$$q_x = \frac{r_{32} - r_{23}}{4q_w}, \qquad q_y = \frac{r_{13} - r_{31}}{4q_w}, \qquad q_z = \frac{r_{21} - r_{12}}{4q_w}.$$

**This branch requires $q_w \neq 0$.** It fails for an exact half-turn and is numerically sensitive near a half-turn. A robust implementation selects a branch whose denominator is large. The following equivalent identities provide the alternative branches:

$$4q_w^2 = 1 + r_{11} + r_{22} + r_{33}, \quad 4q_x^2 = 1 + r_{11} - r_{22} - r_{33},$$
$$4q_y^2 = 1 - r_{11} + r_{22} - r_{33}, \quad 4q_z^2 = 1 - r_{11} - r_{22} + r_{33}.$$

Choose a largest squared component, assign its positive square root, and recover the other components from

$$4q_w q_x = r_{32} - r_{23}, \quad 4q_x q_y = r_{12} + r_{21},$$
$$4q_w q_y = r_{13} - r_{31}, \quad 4q_x q_z = r_{13} + r_{31},$$
$$4q_w q_z = r_{21} - r_{12}, \quad 4q_y q_z = r_{23} + r_{32}.$$

For example, if $q_x$ is chosen, divide the identities for $q_w q_x$, $q_x q_y$, and $q_x q_z$ by $4q_x$ to obtain $q_w, q_y, q_z$. For a valid rotation at least one component has magnitude at least $1/2$. Normalize the resulting quaternion to remove small numerical drift. These formulas assume the input is a proper rotation matrix; they do not repair an arbitrary matrix.